free study notes

Phase 3
Caching, Scaling & Traffic

48 lessons · notes, diagrams and 289 interview questions

1 · Watch
Scan the QR code on a lesson to open its video.

2 · Read
Notes go deeper than the video: worked examples, trade-offs, mistakes.

3 · Practise
Answer the interview questions out loud before reading the answers.

Traffic and load balancing

1 · Load Balancer Fundamentals	SD-G-001
2 · Layer 4 vs Layer 7 Load Balancing	SD-G-002
3 · Round Robin and Weighted Round Robin	SD-G-003
4 · Least Connections and Least Response Time	SD-G-004
5 · Consistent Hashing	SD-G-005
6 · IP Hash and Sticky Sessions	SD-G-006
7 · Health Checks	SD-G-007
8 · Service Discovery	SD-G-008
9 · Reverse Proxy Configuration	SD-G-009
10 · Traffic Shaping and Load Shedding	SD-G-010
11 · Global Server Load Balancing and Regional Routing	SD-G-011
12 · Failover Strategies	SD-G-012
13 · Blue-Green and Weighted Traffic Splitting	SD-G-013
14 · Autoscaling	SD-G-014

Caching

15 · Why Cache and Where	SD-F-001
16 · Cache-Aside (Lazy Loading)	SD-F-002
17 · Read-Through and Write-Through	SD-F-003
18 · Write-Behind and Write-Around	SD-F-004
19 · TTL and Expiration	SD-F-005
20 · LRU Cache	SD-F-006
21 · LFU and FIFO Eviction	SD-F-007
22 · Cache Invalidation Strategies	SD-F-008
23 · Cache Consistency	SD-F-009
24 · Cache Stampede and Thundering Herd	SD-F-010
25 · Cache Penetration and Avalanche	SD-F-011
26 · Hot Keys	SD-F-012
27 · Redis Fundamentals	SD-F-013
28 · Redis Replication, Sentinel and Cluster	SD-F-014
29 · Memcached vs Redis	SD-F-015
30 · Distributed Caching	SD-F-016
31 · CDN and HTTP Caching	SD-F-017
32 · Multi-Level Caching	SD-F-018
33 · Caching in Python Applications	SD-F-019

Scaling and performance

34 · Stateless Service Design	SD-J-001
35 · Scaling Stateful Services	SD-J-002
36 · Database Bottlenecks	SD-J-003
37 · Asynchronous Processing	SD-J-004
38 · Python Concurrency Models	SD-J-005
39 · Batching	SD-J-006
40 · Compression	SD-J-007
41 · Serialization Formats	SD-J-008
42 · Resource Utilization (USE Method)	SD-J-009
43 · Performance Profiling	SD-J-010
44 · Load Testing and Stress Testing	SD-J-011
45 · Queueing Theory Fundamentals	SD-J-012
46 · Tail Latency	SD-J-013
47 · Performance Optimization Strategy	SD-J-014
48 · Backpressure	SD-J-015

Free to share. Videos: youtube.com/@tapankumarbarik9454 · Notes and practice: backendroadmap.com

Load Balancer Fundamentals

one address in front, many servers behind

Watch the 4-min video

In one line: a load balancer gives clients one address, spreads their requests over a pool of healthy servers, and steers around failed ones; it often terminates TLS too, and it must be redundant itself, or it becomes the single point of failure.

1 Why you need one

One server can't keep up anymore, so you add three more. Two problems appear at once. Clients need to know **which server** to call, and when one of them crashes, someone must **stop sending it traffic**. Hard-coding four addresses in every app doesn't solve either.

A **load balancer** (LB) solves both. It sits between the clients and a **pool** of servers (also called *backends* or *targets*). Clients connect to one address: a **virtual IP** (VIP, an address not tied to one machine) or a DNS name. For each connection or request, the load balancer picks a healthy backend and forwards the traffic to it.

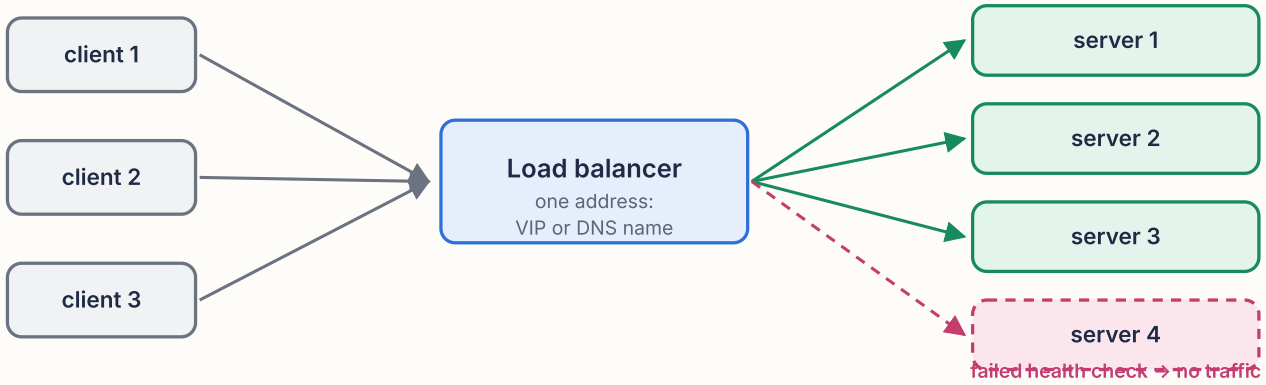


Figure 1. Clients see one address. The load balancer forwards each request to a healthy server; a server that fails its health check gets no traffic.

Because clients only know the load balancer's address, you can add, remove or replace servers behind it and no client notices. That's what makes horizontal scaling (Phase 1, lesson 7) practical.

2 More than spreading load

Most load balancers do several other jobs, which is why nearly every production system has one, and often several:

- **Health checks.** It probes each backend and removes failed ones automatically (lesson 7 of this phase).
- **TLS termination.** It holds the certificate and decrypts HTTPS, so backends don't have to.
- **Connection draining.** During a deploy, it stops sending *new* requests to an old server but lets in-flight ones finish.
- **Timeouts and limits.** One place to cap request time, body size and request rate.
- **Observability.** One place for access logs and metrics: request rate, errors, latency per backend.



Figure 2. The five everyday jobs of a load balancer. Doing them once, in front, keeps every backend simpler.

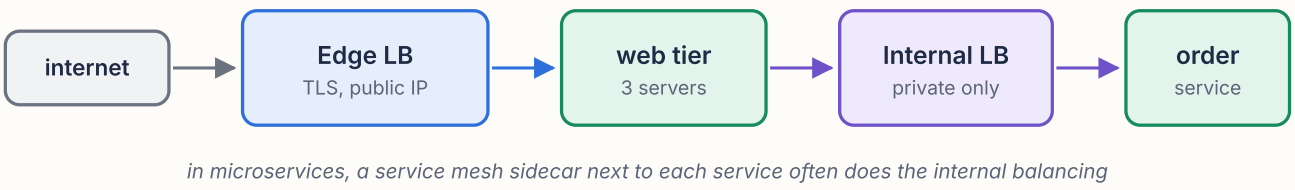
3 Three kinds of load balancer

Kind	Examples	Good	Watch out
Hardware appliance	F5 BIG-IP, Citrix ADC	very fast, dedicated chips	expensive, slow to change, you buy capacity up front
Software	NGINX, HAProxy, Envoy	runs on ordinary servers, flexible, open source	you run, patch and scale it
Cloud, managed	AWS ALB and NLB, Google Cloud Load Balancing, Azure Load Balancer	no servers to manage, scales automatically, multi-zone	provider limits and pricing per hour and per traffic unit

Most new systems use a cloud load balancer at the edge and software proxies (often Envoy) inside.

4 Where it sits: edge and internal

An **edge** load balancer faces the internet: it has the public IP, terminates TLS, and takes traffic from browsers and apps. **Internal** load balancers sit between your own tiers, like the web servers calling an order service; they have private addresses only. In microservices, a **service mesh** (Istio, Linkerd) often does internal balancing in a *sidecar* proxy that runs next to every service.



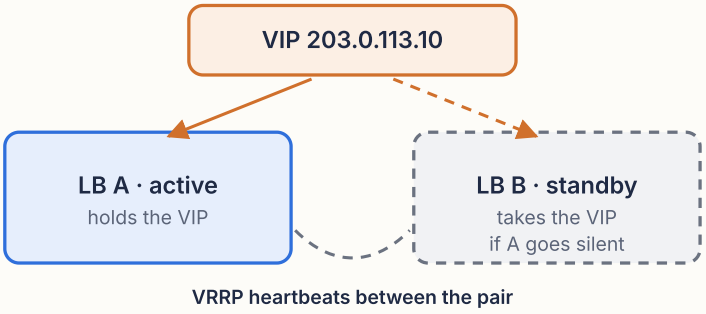
in microservices, a service mesh sidecar next to each service often does the internal balancing

Figure 3. An edge load balancer at the internet boundary and an internal one between tiers. Large systems have many of each.

5 Isn't the load balancer a single point of failure?

Yes, if there is only one. All traffic goes through it, so it must be redundant:

- **Active-passive pair with a floating VIP.** Two load balancers share one virtual IP. The active one holds it; the standby listens for its heartbeats with a protocol like **VRRP** (Virtual Router Redundancy Protocol, used by keepalived). If the heartbeats stop, the standby claims the IP within seconds.
- **Several active nodes.** Many load balancer nodes all take traffic, spread by DNS (several A records) or by **anycast** (the same IP announced from many places).
- **Managed cloud load balancer.** AWS, Google and Azure run it on many machines across availability zones for you.



or: several active nodes behind DNS or anycast · or: a managed cloud LB spread over availability zones

Figure 4. An active-passive pair: the active node holds the virtual IP; if its VRRP heartbeats stop, the standby takes the IP over.

WORKED EXAMPLE: AN ALB IN FRONT OF ECS

An AWS Application Load Balancer is spread across three availability zones, in front of an ECS service (containers). Each new task registers itself in a **target group** automatically. The ALB health-checks every task and sends requests only to healthy ones. During a deploy, it **drains** the old tasks (no new requests, in-flight ones finish, up to a deregistration delay such as 30 seconds) while new tasks start. The ALB itself is managed and redundant across the zones, so it's not your single point of failure.

COMMON MISTAKES

- Drawing one load balancer box in an interview and never saying how it is made redundant.
- No health checks, or checks that only test that the process is up, so broken servers keep getting traffic.
- Deploying without draining, which cuts off requests that were in flight.
- Keeping user sessions in server memory, so the load balancer can't move a user to another server (lesson 6 covers sticky sessions).

6 Key terms

Load balancer: spreads requests over a pool of servers behind one address
Virtual IP (VIP): an address that can move between machines
TLS termination: the LB decrypts HTTPS so backends get plain HTTP
VRRP: protocol that moves a VIP to the standby when the active fails
Target group: the AWS name for a pool of backends
Backend / target: one server in the pool
Health check: a probe that marks a backend healthy or not
Connection draining: finish in-flight requests before removing a server
Anycast: one IP announced from many locations

7 Interview questions

Q1. What does a load balancer do? BASIC

It gives clients one address and spreads their requests across a pool of healthy servers, removing failed ones with health checks. Most also terminate TLS, drain connections during deploys, enforce timeouts and limits, and give one place for logs and metrics.

Q2. How do you make the load balancer itself highly available? BASIC
Run it redundantly: an active-passive pair sharing a floating virtual IP with VRRP, several active nodes behind DNS or anycast, or a managed cloud load balancer that runs across availability zones.

Q3. Hardware, software or cloud load balancer: how do you choose? BASIC
Hardware is fast but expensive and slow to change. Software (NGINX, HAProxy, Envoy) is flexible but you run it. Cloud load balancers are managed and scale automatically, so they are the usual default for new systems, with software proxies inside.

Q4. You deploy a new version and some users see errors for a few seconds. Why, and how do you fix it? SCENARIO
Old servers were removed while requests were still in flight, or new servers got traffic before they were ready. Enable connection draining with a deregistration delay, and use a readiness health check so a new server only gets traffic once it can serve.

Q5. Where do you put load balancers in a three-tier web app? SCENARIO
An edge load balancer at the internet boundary in front of the web tier, terminating TLS, and an internal load balancer (or a service mesh) in front of each internal service the web tier calls. Each is redundant across zones.

Q6. A client keeps one long-lived connection. Does the load balancer still spread its requests? TRICKY
Not at layer 4: the connection is pinned to one backend when it opens. A layer 7 load balancer can spread individual HTTP/2 or gRPC requests from that one connection. That's the next lesson.

Q7. The load balancer is up, but every request times out. What do you check? TRICKY
The health checks (is any backend marked healthy?), the target group's security groups and ports, and the load balancer's idle and request timeouts versus how long the backend takes. A healthy LB with no healthy targets returns errors such as 502 or 503.

Next: SD-G-002 · Layer 4 vs Layer 7 Load Balancing backendroadmap.com

Layer 4 vs Layer 7 Load Balancing

addresses and ports, or the whole HTTP request

Watch the 4-min video

In one line: a layer 4 load balancer sees only IP addresses, ports and TCP/UDP, so it is fast and works with any protocol; a layer 7 load balancer reads each HTTP request, so it can route by path, host or header, terminate TLS and balance individual HTTP/2 and gRPC requests, at more cost per request.

1 Two load balancers, two layers

Both are called load balancers, but they work at different layers of the network stack (Phase 2, lessons 5 to 7). Layer 4 is the **transport** layer: TCP and UDP, with ports. Layer 7 is the **application** layer: HTTP, with paths, headers and cookies. What a load balancer can see decides what it can do.

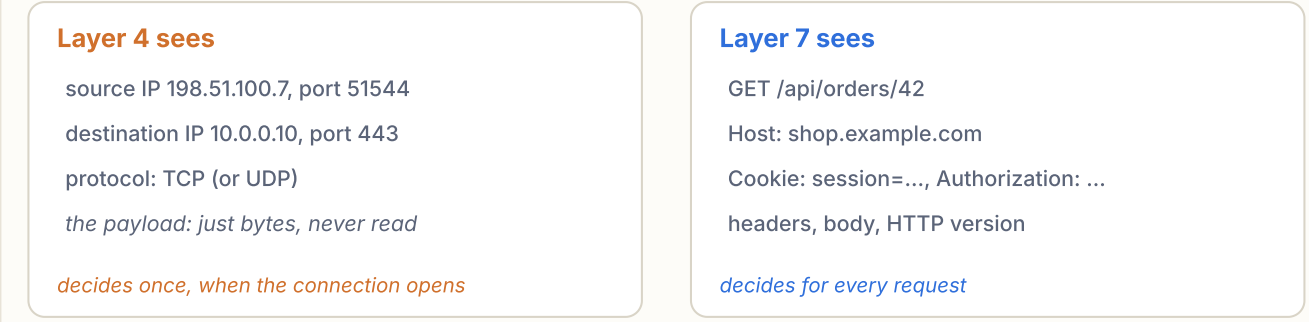


Figure 1. The same request seen by each kind. Layer 4 sees the addresses, ports and protocol; layer 7 parses the HTTP request itself.

2 Layer 4: fast and protocol-blind

A layer 4 load balancer picks a backend when a **connection opens** (using the source and destination IP and port), then just forwards the packets for that connection. It never parses the payload, so:

- it's very fast, adds little latency and handles millions of connections;
- it works with **any protocol** over TCP or UDP: databases, game servers, MQTT, custom binary protocols;
- it can pass encrypted TLS straight through without reading it.

The cost: it is blind to content. It can't route `/api/orders` differently from `/api/users`, and one long connection stays on one backend for its whole life.

3 Layer 7: reads every request

A layer 7 load balancer **terminates** the client's connection, reads the HTTP request and decides **per request**. That allows:

- routing by **path** (`/api/orders/*`) or **host name** (`api.example.com`);
- matching **headers or cookies** (send beta users to version 2);
- rewriting URLs and adding headers such as `X-Forwarded-For` (the client's real IP);
- **retrying** a failed request on another backend, and rate limiting.

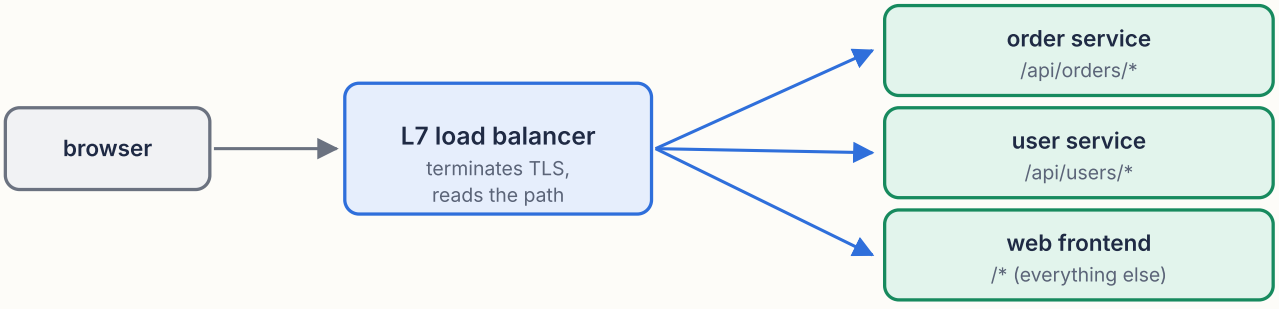


Figure 2. Path-based routing on one domain: the layer 7 load balancer reads the path and sends each request to the right service.

4 TLS: terminate or pass through

To read HTTP, a layer 7 load balancer must decrypt it, so it usually does **TLS termination**: it holds the certificate, decrypts, and sends plain HTTP (or a fresh internal TLS connection, called re-encryption) to the backends. Certificates live in one place, and the balancer can inspect requests.

With layer 4 you can do **TLS passthrough** instead: the encrypted bytes go straight to the backend, and only the backend ever decrypts them. Some compliance rules (end-to-end encryption, mutual TLS to the application) require that.

5 The trade-off, and the HTTP/2 trap

	Layer 4	Layer 7
Decides	once per connection	once per request
Speed and CPU	lowest latency, least CPU	more work per request
Protocols	anything over TCP/UDP	must understand it (HTTP, gRPC, WebSocket)
Routing	IP and port only	path, host, headers, cookies
TLS	pass through (or terminate)	terminates
AWS example	Network Load Balancer (NLB)	Application Load Balancer (ALB)

The trap: **HTTP/2 and gRPC** send many requests over **one long connection**. A layer 4 balancer pins that connection to one backend, so one backend gets every request while the others sit idle. You need layer 7 (or client-side balancing) to spread the individual requests.

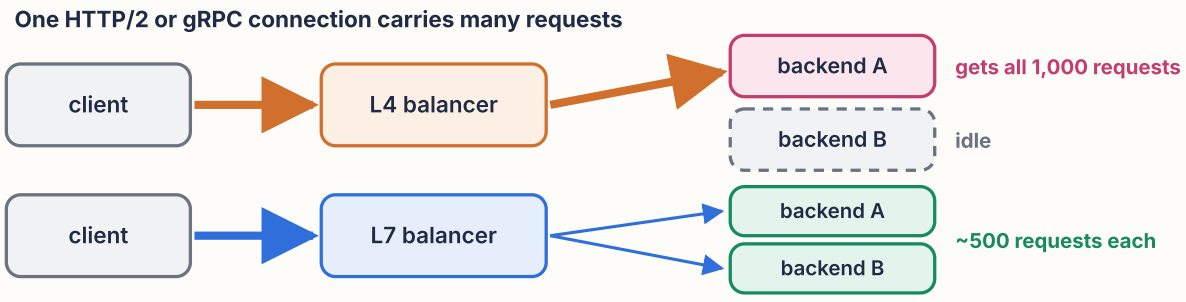


Figure 3. With one HTTP/2 connection, a layer 4 balancer sends all its requests to one backend; a layer 7 balancer spreads them.

WORKED EXAMPLE: A GAME AND A WEB PLATFORM

A multiplayer game uses a custom binary protocol over raw TCP. An AWS **Network Load Balancer** (layer 4) forwards those connections with very low latency, without needing to understand the protocol. Meanwhile, the company's web platform uses an **Application Load Balancer** (layer 7) to route `/api/orders` to the order service and `/api/users` to the user service, all on one domain and one certificate.

6 Using both

Many large systems stack them. A layer 4 balancer at the edge spreads millions of raw connections across a fleet of layer 7 proxies such as Envoy or NGINX, which then do the HTTP routing. The layer 4 tier is cheap and scales to huge connection counts; the layer 7 tier adds the smarts. Inside the cluster, a **service mesh** brings the same layer 7 features (retries, per-request balancing, mutual TLS) between services.

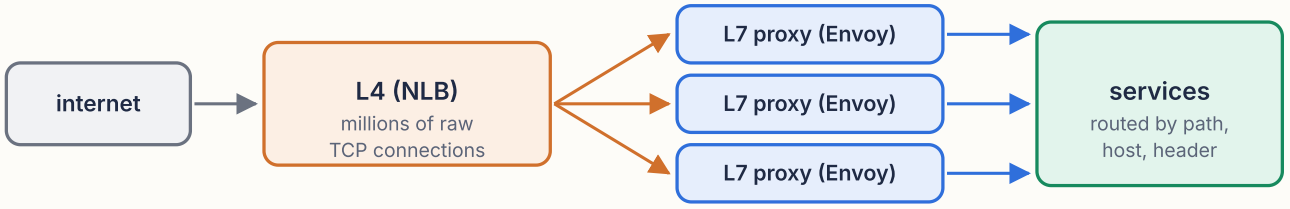


Figure 4. A common pattern: layer 4 at the edge for raw connection volume, layer 7 proxies behind it for HTTP routing.

COMMON MISTAKES

- Putting gRPC services behind a layer 4 balancer and wondering why one pod is at 100% CPU.
- Choosing layer 7 for a non-HTTP protocol it can't parse.
- Forgetting that after TLS termination the backend sees the balancer's IP, not the client's: read `X-Forwarded-For` (or use the PROXY protocol at layer 4).

7 Key terms

Layer 4 (transport): TCP/UDP, IP addresses and ports

TLS termination: the balancer decrypts and holds the certificate

Path routing: pick a service by URL path

NLB / ALB: AWS layer 4 / layer 7 load balancers

Layer 7 (application): HTTP: paths, headers, cookies

TLS passthrough: encrypted bytes go to the backend untouched

X-Forwarded-For: header carrying the original client IP

Multiplexing: many requests sharing one connection (HTTP/2)

8 Interview questions

Q1. When would you use a layer 4 instead of a layer 7 load balancer? BASIC

For non-HTTP protocols such as raw TCP or UDP, when you need the lowest latency and the highest connection counts, or when TLS must pass through to the backend untouched.

Q2. When do you need layer 7? BASIC

For routing by path, host or header, for TLS termination with one place for certificates, for retries and rate limits, and to balance individual HTTP/2 or gRPC requests that share one connection.

Q3. Your gRPC service has 10 pods, but one is at 100% CPU and the rest are idle. Why? SCENARIO

gRPC uses long-lived HTTP/2 connections, and a layer 4 balancer pins each connection to one pod. Use a layer 7 proxy (Envoy, a mesh) or client-side load balancing so each request is balanced, and cap connection age so clients reconnect.

Q4. Security requires that only the application ever sees decrypted traffic. What do you use? SCENARIO

A layer 4 load balancer with TLS passthrough, so the backend terminates TLS. You lose path routing at the balancer, so route by port or SNI, or do routing in the application tier.

Q5. After moving behind an ALB, every request in your logs comes from the same few IPs. Why? TRICKY

The ALB terminates the connection and opens its own to the backend, so the source IP is the ALB's. The real client IP is in the X-Forwarded-For header; log that, and trust it only from your balancer.

Q6. Why not just use layer 7 everywhere? TRICKY

It costs CPU and latency per request, must understand each protocol, and terminates TLS. For huge connection counts, non-HTTP traffic or passthrough needs, layer 4 is simpler and cheaper; many systems put layer 4 in front of layer 7.

Next: SD-G-003 · Round Robin and Weighted Round Robin backendroadmap.com

Round Robin and Weighted Round Robin

take turns, and give bigger servers bigger turns

Watch the 4-min video

In one line: round robin sends each new request to the next server in turn, and weighted round robin gives bigger servers proportionally more turns; both are cheap and fair when requests cost about the same, and both go wrong when request costs vary or connections live long, because they count requests, not work.

1 Taking turns

The simplest way to share work is to take turns: server 1, server 2, server 3, then back to 1. That's **round robin**, and it's the default in many load balancers (NGINX, HAProxy, AWS ALB). The balancer keeps a **pointer** into the list of healthy backends; each new request goes to the next one, and the pointer wraps around at the end.

It needs no measurements and no knowledge of the servers, so it costs almost nothing. With identical servers and similar requests, each server gets an equal share.



Figure 1. Round robin with three servers: nine requests, three each, in strict rotation.

2 Weighted round robin: unequal servers

Servers often differ: an old instance type and a new one, or a mix of sizes. **Weighted round robin** gives each server a **weight** proportional to its capacity. A server with weight 2 gets two requests for every one sent to a server with weight 1.

A naive version sends the weight-2 server two requests in a row, a little burst. **Smooth weighted round robin**, the algorithm in NGINX, interleaves them so the big server's turns are spread out (C, A, C, B rather than C, C, A, B).

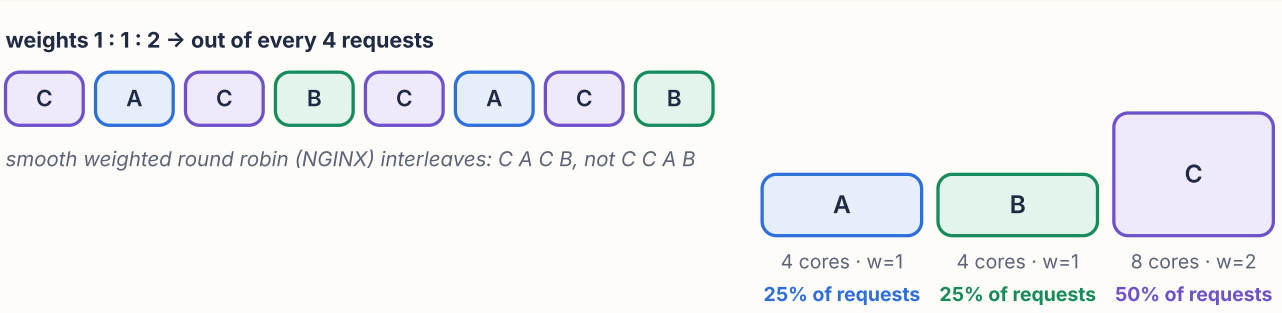


Figure 2. Two 4-core servers with weight 1 and one 8-core server with weight 2: the big one gets half the requests, so CPU stays roughly even.

WORKED EXAMPLE: A MIXED POOL

The pool has two 4-core instances and one 8-core instance. Give the small ones weight 1 and the large one weight 2. Out of every 4 requests, the large instance gets 2 and each small one gets 1, so each core sees about the same load. At 1,200 requests per second: 600 to the big server, 300 to each small one.

Weights are also handy for **canary releases**: give the new version weight 1 and the old fleet weight 19, and about 5% of traffic tries the new code (lesson 13 goes further).

3 Where round robin breaks

It counts requests, not work. If most requests take 10 milliseconds but some take 10 seconds (a report export, an LLM answer), one server can draw several slow ones in a row. A queue builds there while its neighbours sit idle, and round robin keeps sending it its fair share anyway.

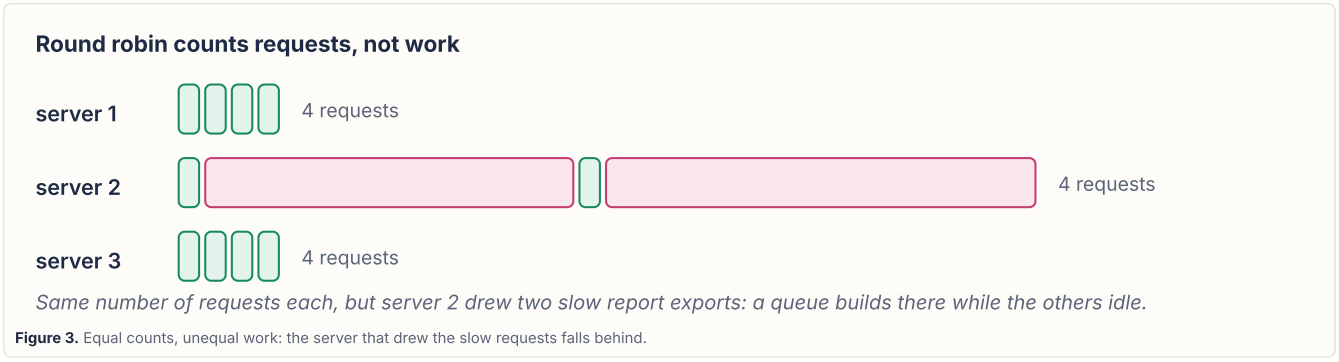


Figure 3. Equal counts, unequal work: the server that drew the slow requests falls behind.

Other traps:

- **Long-lived connections.** WebSockets and gRPC streams are balanced once, when they open. After a scale-up, the old servers keep their connections and the new ones stay nearly empty.
- **Static weights are blind.** A server that is slow because of garbage collection, a bad disk or a noisy neighbour keeps getting its full share.
- **Many balancer nodes.** Each node runs its own rotation; combined, the pattern any one server sees can be lumpy rather than smooth.

4 When to use which

Situation	Choose
Similar servers, short requests of similar cost (typical stateless web APIs)	**Round robin**
Mixed instance sizes, or a gradual rollout / canary	**Weighted round robin**
Request cost varies a lot, or long-lived streams	**Least connections** or latency-aware (next lesson)

COMMON MISTAKES

- Keeping round robin for an endpoint mix that includes slow exports or AI calls, then blaming the servers for high p99 latency.
- Setting weights once and forgetting them after the instance types change.
- Scaling out WebSocket servers and expecting existing connections to move to the new ones.

5 Key terms

Round robin: next request to the next server, wrapping around

Smooth weighted RR: interleaves turns so big servers get no bursts

Long-lived connection: WebSocket or gRPC stream, balanced only once

Weight: a number proportional to a server's capacity

Canary release: a small share of traffic to a new version

Load-aware algorithm: chooses by current load, e.g. least connections

6 Interview questions

Q1. How does round robin work? BASIC

The balancer keeps a pointer into the list of healthy backends and sends each new request to the next one, wrapping around at the end. It needs no measurements, so it is cheap and gives equal shares when servers and requests are alike.

Q2. When does round robin distribute load poorly? BASIC

When request costs vary widely, when connections are long-lived, or when servers differ in capacity or health, because it counts requests, not actual load.

Q3. How does weighted round robin help? BASIC

It gives larger servers proportionally more requests, for example twice the traffic to an instance with twice the cores. Smooth weighted round robin interleaves the turns to avoid bursts.

Q4. You scale a WebSocket fleet from 4 to 8 servers, but the new ones get almost no load. Why, and what do you do? SCENARIO

Existing connections were balanced when they opened and stay put. Use least connections for new connections, and gradually rebalance by asking clients to reconnect (connection max age, or closing a share of connections on the busy servers).

Q5. How would you send 5% of traffic to a new version with round robin? SCENARIO

Weighted round robin: put the new version in the pool with weight 1 against weight 19 for the old fleet (or use a balancer feature such as weighted target groups), watch errors and latency, then raise the weight.

Q6. Your p99 latency is high, yet CPU is low on most servers. The balancer uses round robin. What's going on? TRICKY

Probably uneven work: a few slow requests queue on whichever servers drew them, while others idle. Average CPU hides it. Switch to least connections or power of two choices, or move slow work to an async queue.

Next: SD-G-004 · Least Connections and Least Response Time

Least Connections and Least Response Time

route by the load a server actually has

Watch the 4-min video

In one line: least connections sends each new request to the backend with the fewest active requests, so slow requests stop piling up; least response time also prefers recently fast backends (measured with a moving average), and power of two choices (pick two at random, take the less loaded) keeps many balancers from stampeding onto one server.

1 The problem round robin can't see

Your servers stream answers from a language model. Some responses finish in 2 seconds, others take a full minute. Round robin keeps handing out requests in turn, and soon one server juggles twenty long streams while another has two. To spread *work*, the balancer has to look at the load each server actually has.

2 Least connections

Least connections tracks how many requests (or connections) each backend is handling right now. Each new request goes to the backend with the **fewest**. When slow requests tie up a server, its count stays high, so new work flows elsewhere. It adapts to variable request durations automatically, without knowing them in advance.

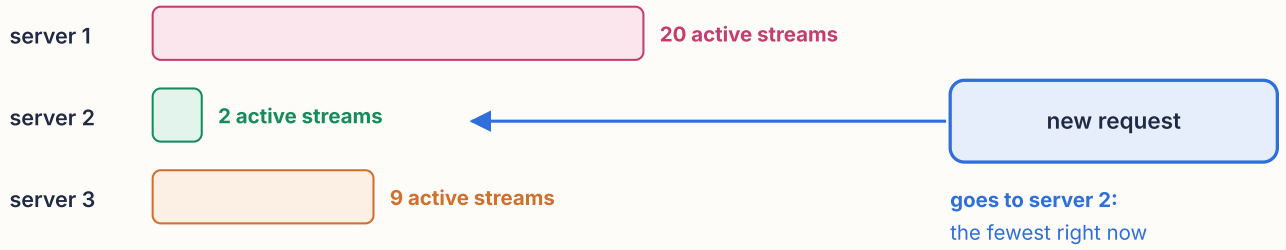


Figure 1. Least connections: the new request goes to the server with the fewest active requests right now, not the next one in turn.

Weighted least connections divides the count by capacity: a server with weight 2 may hold twice as many before it counts as equally busy.

3 Least response time and EWMA

Least response time goes further: it prefers the backend with the lowest *recent latency*, usually combined with its active request count. A server that is slow because of garbage collection, a hot disk or a noisy neighbour gets less traffic until it recovers.

To measure recent latency, balancers keep an **exponentially weighted moving average** (EWMA). Each new sample nudges the average, and old samples fade away:

```
average = α × new sample + (1 − α) × previous average (e.g. α = 0.3)
```

It reacts quickly to a real slowdown but doesn't overreact to one slow request. Envoy, Linkerd and Finagle all offer latency-aware balancing like this.

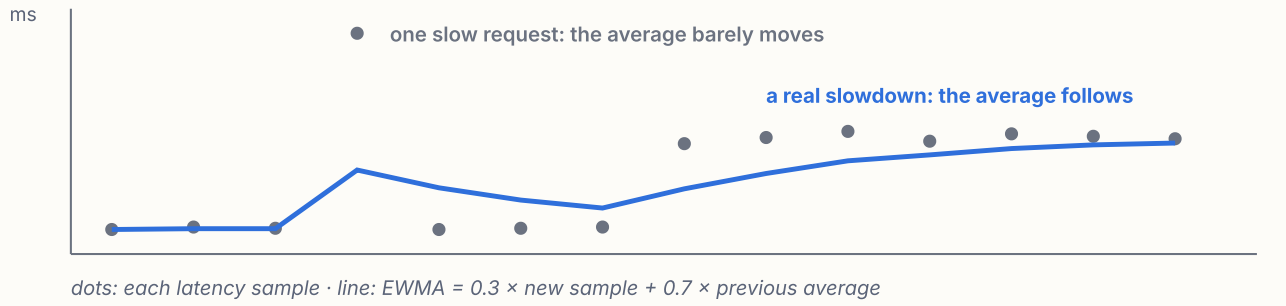


Figure 2. An EWMA of latency samples: a single slow request barely moves it; a lasting slowdown pulls it up within a few samples.

4 The herd problem and power of two choices

Always picking the global minimum has a catch. With many balancer nodes, they all see the same least loaded server, and all send their next requests there at once. It's suddenly the busiest server. Their view of load is also slightly out of date, which makes the stampede worse.

Power of two choices fixes it: pick **two** backends at random, and send the request to the **less loaded** of the two. Surprisingly, this nearly matches a full search. In the maths, the most loaded server goes from about log n / log log n above average with pure random choice to about log log n with two choices: an exponential improvement. It is cheap, needs only local information, and scales to thousands of backends. Envoy's default least-request balancer works this way.

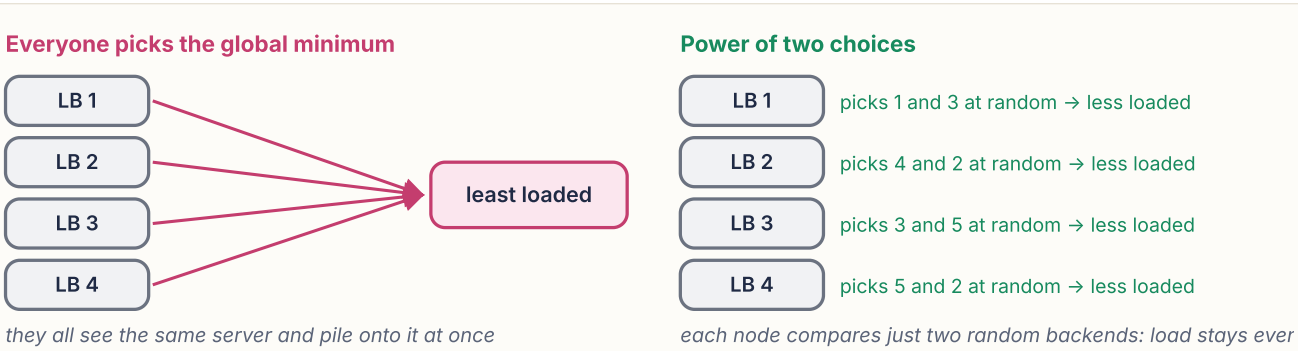


Figure 3. Left: every balancer chases the same minimum and overloads it. Right: each compares two random backends, so the choices spread out.

WORKED EXAMPLE: AN LLM GATEWAY

A gateway streams tokens from a pool of GPU inference servers. Each stream lasts between 2 seconds and 2 minutes. With least connections, each new chat goes to the server with the fewest active streams, so long generations don't pile up on one GPU. Several gateway nodes run power of two choices, so they don't all pick the same quiet server, and the number of active streams stays within a few of each other across the pool.

TRADE-OFFS

- The balancer must track state per backend (counts, latencies), so it's slightly more work than round robin.
- Counts are local to each balancer node; with many nodes, each sees only its own traffic. That's fine with power of two choices.
- A server that fails fast (instant errors) looks lightly loaded and attracts traffic: combine with health checks and outlier ejection.

5 Key terms

Least connections: pick the backend with the fewest active requests

Least response time: prefer the backend with the lowest recent latency

Herd behaviour: many balancers choosing the same server at once

Outlier ejection: temporarily remove a backend that keeps failing

Weighted least conns: count divided by capacity weight

EWMA: moving average where old samples fade

Power of two choices: pick two at random, use the less loaded

6 Interview questions

Q1. Why is least connections better for variable request duration? BASIC

It routes by current active requests, not by turn, so a server busy with slow requests automatically gets fewer new ones, without knowing request durations in advance.

Q2. What is power of two choices? BASIC

Pick two backends at random and send the request to the less loaded one. It nearly matches a full search for the minimum, needs only local information, and stops every balancer from stampeding to the same server.

Q3. How does least response time measure latency? BASIC

With an exponentially weighted moving average of recent response times, often combined with the active request count, so a server that slows down gets less traffic until it recovers.

Q4. A buggy backend returns 500 errors in 1 ms. Least connections sends it more and more traffic. Why, and how do you fix it? SCENARIO

Fast failures keep its active count low, so it looks idle. Add outlier ejection (remove backends with high error rates for a while) and health checks, so a failing backend leaves the pool.

Q5. You have 20 gateway nodes in front of 200 GPU servers serving long streams. Which algorithm? SCENARIO

Least connections (least requests) with power of two choices on each gateway node: it balances active streams without a global view and without the herd effect of every node choosing the same minimum.

Q6. Why not always pick the server with the absolute lowest load? TRICKY

With many balancers and slightly stale load data, they all pick the same server simultaneously and overload it, then all move to the next. Randomising between two candidates breaks that synchronisation.

Next: SD-G-005 · Consistent Hashing

Consistent Hashing

add a server, move only a small share of the keys

Watch the 5-min video

In one line: with `hash(key) mod n`, changing n moves almost every key; consistent hashing puts servers and keys on a ring and gives each key to the next server clockwise, so adding or removing a server moves only about 1/n of the keys, and virtual nodes keep the load even.

1 The problem with modulo

You spread cache keys across four servers with `server = hash(key) mod 4`. It works well, until you add a fifth server. Now `mod 5` sends most keys to a different server than before. For a cache that's almost a cold restart: the hit rate collapses, and every miss goes to the database.

The reason: a key's server depends on the **total number** of servers. Going from 4 to 5 moves about 80% of the keys; in general, going from n to n + 1 moves about n/(n + 1) of them. What we want is that only the keys the new server should own move: about 1/(n + 1).

key hash	hash mod 4	hash mod 5
17	server 1	server 2
26	server 2	server 1
39	server 3	server 4
45	server 1	server 0
60	server 0	server 0

4 → 5 servers with modulo:
about 80% of keys change server
a cache becomes nearly cold;
the database takes the misses

Figure 1. With modulo hashing, changing the server count changes the answer for most keys.

2 The ring

Take the output range of a hash function (say 0 to $2^{32} - 1$) and bend it into a circle. Hash each **server** (by name or address) to a position on the ring. To find a key's server, hash the **key** to a position, then walk **clockwise** to the first server you meet. That server owns the key.

Now add a server. It lands at one point and takes over only the keys between it and the previous server counter-clockwise. Every other key stays where it was. Remove a server, and only its keys move, to the next server clockwise. On average only **1/n** of the keys move: the minimum possible.

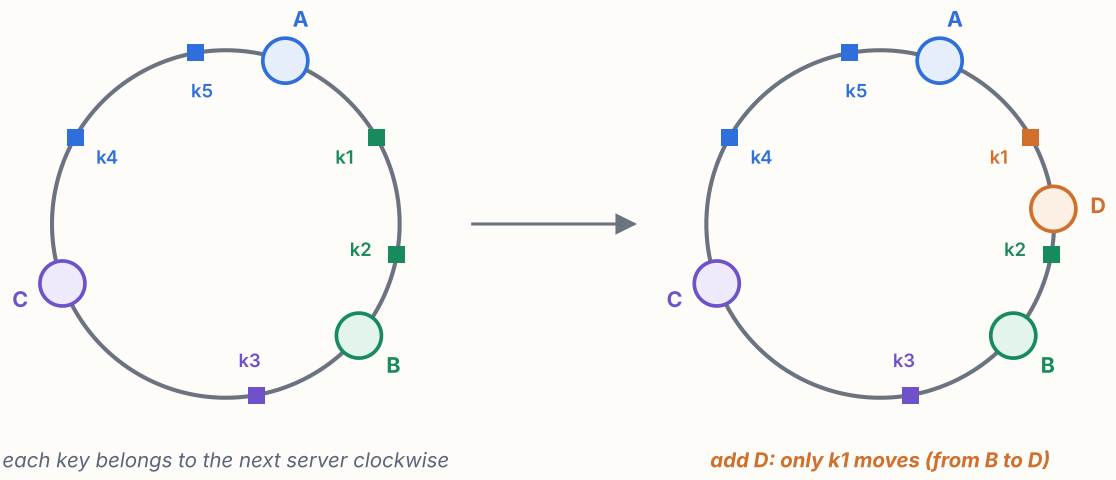
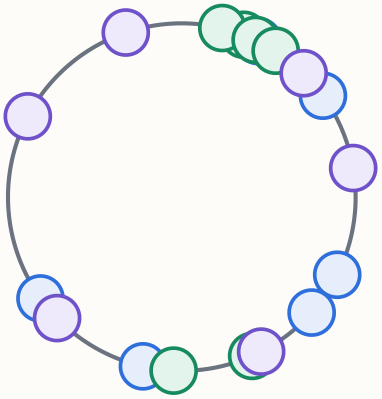


Figure 2. Left: three servers on the ring, each key owned by the next server clockwise. Right: adding D takes over only the arc between B's predecessor and D.

3 Virtual nodes

With only a few servers, their random positions can be very uneven: one server might own a huge arc, another a tiny one. And when a server leaves, all its keys land on just one neighbour, which may then be overloaded.

Virtual nodes fix both. Each physical server is hashed to **many** points on the ring, say 100 to 200 (hash `A#1`, `A#2`, ...). The many small arcs average out, so each server gets close to its fair share. A departing server's keys scatter across many servers. And **weights** become easy: give a bigger server more virtual nodes.



3 servers × 6 virtual nodes

Figure 3. Virtual nodes: each server owns many small arcs instead of one big one.

Without virtual nodes, 3 random points can give one server 60% of the ring and another 10%.

With 100–200 virtual nodes per server:

- the arcs average out, so load is even
- a failed server's keys spread over many servers
- weights are easy: a bigger server gets more points

each physical server appears at many positions

WORKED EXAMPLE: ADDING A MEMCACHED NODE FOR THE HOLIDAY PEAK

A Memcached cluster of four nodes uses a client library with consistent hashing (the classic one is **ketama**). You add a fifth node for the holiday peak. Only about 20% of keys move to the new node; the other 80% keep hitting the same servers. The database sees a small bump in misses, not a stampede. With plain modulo, about 80% of keys would have moved instead.

The same idea places data in **DynamoDB** and **Cassandra** (each node owns token ranges on a ring) and powers hash-based load balancing such as Envoy's ring hash and Maglev.

4 Alternatives

Method	How	Good	Catch
Ring + virtual nodes	next server clockwise	minimal movement, weights	needs many vnodes for even load; ring memory
Rendezvous hashing (highest random weight)	score every server with hash(key, server), pick the highest	no ring, very even, minimal movement	O(n) per lookup, slower with many servers
Jump consistent hash	a few lines of arithmetic map a key to a bucket 0..n−1	tiny, fast, near-perfect balance, no memory	servers can only be added or removed at the end

COMMON MISTAKES

- Using modulo hashing for a cache or shard map that will ever change size.
- One point per server, then wondering why one node holds 40% of the data.
- Different clients using different hash functions or server lists, so they disagree about where a key lives.
- Forgetting hot keys: consistent hashing spreads keys evenly, not traffic, so one very popular key still lands on one server (lesson 26).

5 Key terms

Modulo hashing: hash(key) mod n; moves most keys when n changes

Owner: the first server clockwise from the key

Ketama: classic consistent hashing for Memcached clients

Jump hash: fast arithmetic consistent hash for numbered buckets

Hash ring: the hash range bent into a circle

Virtual node: one of many ring positions for a server

Rendezvous hashing: pick the server with the highest hash(key, server)

Token range: the arc of the ring a Cassandra node owns

6 Interview questions

Q1. Explain consistent hashing. BASIC

Servers and keys are hashed onto a ring, and each key belongs to the next server clockwise, so adding or removing a server only moves about 1/n of the keys, instead of nearly all of them as with modulo.

Q2. Why use virtual nodes? BASIC

They give each server many positions on the ring, which evens out the load, spreads a failed server's keys across many others instead of one neighbour, and allows weights for bigger servers.

Q3. You go from 4 to 5 cache nodes. How many keys move with modulo, and with consistent hashing? SCENARIO

With modulo, about 4/5, 80% of keys. With consistent hashing, about 1/5, 20%: just the share the new node should own.

Q4. Where is consistent hashing used in real systems? SCENARIO

Distributed caches (Memcached clients with ketama, Redis client-side sharding), databases such as Cassandra and DynamoDB that assign token ranges to nodes, and load balancers that need stable key-to-backend mapping (Envoy ring hash, Maglev).

Q5. Consistent hashing gives each node the same number of keys, yet one node is overloaded. Why? TRICKY

Keys are spread evenly, but traffic isn't: a hot key (a celebrity's profile) sends all its requests to one node. Fix with replication of hot keys, local caching, or key splitting.

Q6. When would you pick rendezvous or jump hashing instead of a ring? TRICKY

Rendezvous when the server list is small and you want very even balance without vnodes; jump hash when buckets are numbered and only grow or shrink at the end, such as shards 0..n−1, because it needs no memory and is very fast.

Next: SD-G-006 · IP Hash and Sticky Sessions

IP Hash and Sticky Sessions

pin a user to one server, and what it costs

Watch the 4-min video

In one line: sticky sessions (session affinity) send every request from one client to the same backend, by IP hash or by cookie, so in-memory state keeps working; the price is uneven load, lost sessions when a server dies, and harder deploys and scaling, so the real fix is to keep servers stateless and put state outside.

1 The empty cart

A user adds three items to their cart. The next request lands on a different server, and the cart is empty. The old app kept sessions in each server's memory, so only server 1 knows about the cart. The quick fix is to make the load balancer always send that user to the same server. That's a **sticky session**.

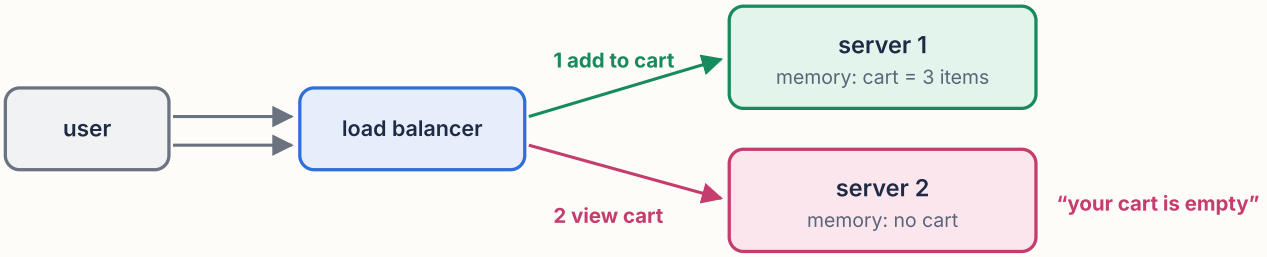


Figure 1. With sessions in server memory, a round-robin load balancer sends the second request to a server that has never seen the cart.

2 Session affinity

Session affinity, or stickiness, means the load balancer routes all requests from one client to the same backend. You need it when a server keeps **per-user state locally**: an in-memory session, a shopping cart, a multi-step upload, or a WebSocket's context. There are two common ways to do it.

3 IP hash, and cookie stickiness

IP hash hashes the client's IP address and picks a backend from the hash. Same IP, same server, and the balancer keeps no state. But many users can share one public IP behind a corporate proxy or a mobile carrier's NAT (carrier-grade NAT puts thousands of phones behind a few addresses), which creates hot servers. And a phone that switches from Wi-Fi to 4G gets a new IP and lands somewhere else.

Cookie-based stickiness is more precise. On the first response the load balancer sets a cookie naming the backend (AWS ALB's `AWSALB`, an NGINX `sticky cookie`), or the app sets its own session cookie that the balancer reads (application-based stickiness). Later requests carry the cookie and go back to the same server, usually until the cookie expires.

IP hash

backend = hash(client IP) mod n (or a ring)

+ no state on the balancer
+ works for any TCP protocol
− 5,000 office users behind one proxy IP → one hot server
− phone moves Wi-Fi → 4G: new IP, new server, session lost

Cookie stickiness

Set-Cookie: AWSALB=... (names the backend)

+ per browser, not per IP
+ survives IP changes
+ ALB, NGINX, HAProxy support it, usually with an expiry
− HTTP only (needs layer 7)
− cookie must be allowed by the client

Figure 2. The two ways to stick: by source IP (stateless, any protocol, but coarse) or by cookie (per browser, but HTTP only).

4 The bill

- **Uneven load.** A few heavy users pin themselves to one server, and new servers only get *new* users, so scaling out helps slowly.
- **Failover loses state.** If that server dies, every session on it is gone: users are logged out or lose their carts.
- **Harder deploys and scale-in.** You must drain sessions before removing a server, so rolling deploys and autoscaling take much longer.
- **Hidden coupling.** The app quietly depends on the load balancer's behaviour; change one setting and users see bugs.

WORKED EXAMPLE: A LEGACY APP WITH IN-MEMORY SESSIONS

The short-term fix is cookie-based stickiness, so the app keeps working behind the load balancer. The real fix is to move sessions out:

- into **Redis** (key `session:abc123` with a TTL), or a database table;
- or into a **signed token** (a JWT or a signed cookie) that the client carries, so no server needs to look anything up.

Then any server can handle any request, stickiness can be turned off, and a server can die or be replaced without anyone losing a cart.

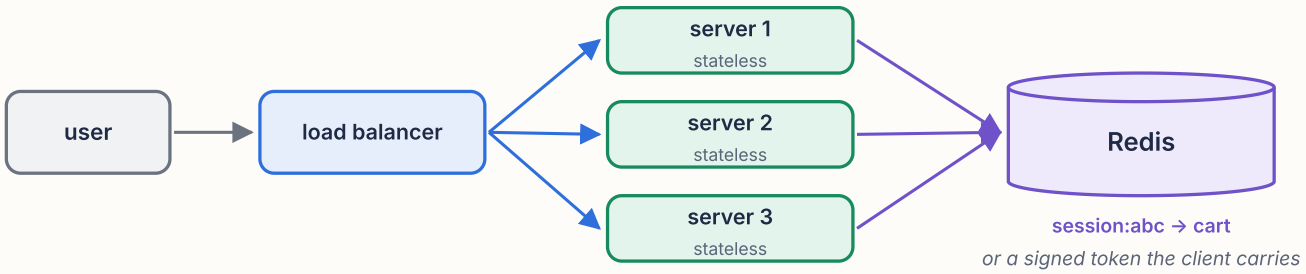


Figure 3. The fix: stateless servers and an external session store. The load balancer can send each request anywhere.

5 When stickiness is fine

Stickiness isn't always bad. It can be a **performance optimisation**: a server's local cache warms up for the users it sees, so pinning them helps, as long as losing that pinning is harmless. Long-lived WebSocket connections are naturally pinned anyway. The rule: **affinity may help speed, but correctness must never depend on it.**

COMMON MISTAKES

- Using stickiness to hide in-memory sessions, then losing sessions on every deploy.
- IP hash for a mobile or enterprise audience, with many users behind one IP.
- Forgetting a cookie expiry, so users stay pinned to a server long after it's overloaded.
- Storing whole carts or large objects in a JWT: tokens travel on every request and can't be revoked easily.

6 Key terms

Sticky session: all requests from one client go to one backend

IP hash: choose the backend from a hash of the client IP

Duration-based cookie: a balancer-made cookie naming the backend

Session store: external place for sessions, e.g. Redis

Session affinity: another name for stickiness

Carrier-grade NAT: many phones sharing a few public IPs

Stateless server: keeps no per-user state between requests

Signed token: state the client carries, protected by a signature

7 Interview questions

Q1. Why are sticky sessions problematic? BASIC
They cause uneven load, lose user sessions when a server fails, and make deploys and autoscaling harder; IP hash also breaks with shared or changing IP addresses.

Q2. What's the better approach? BASIC
Store session state externally, in Redis, a database, or a signed token the client carries, so any server can handle any request and stickiness isn't needed.

Q3. IP hash or cookie stickiness: which is more precise? BASIC
Cookie stickiness: it identifies the browser rather than the network, so users behind one proxy spread out and a phone changing networks keeps its server. IP hash works for non-HTTP protocols, though.

Q4. After a deploy, many users are logged out. The app uses sticky sessions. Why? SCENARIO
Sessions lived in the memory of the servers that were replaced. Move sessions to a shared store or tokens, and meanwhile drain servers (wait for sessions to expire) before removing them.

Q5. You doubled your servers, but the old ones are still at 90% CPU. Why? SCENARIO
Stickiness keeps existing users on the old servers; only new users reach the new ones. Shorten the stickiness duration, or remove stickiness by externalising state.

Q6. When is stickiness a good idea? TRICKY
When it's only an optimisation, such as warm per-user caches, or for long-lived connections like WebSockets, and the system stays correct if a user moves to another server.

Next: SD-G-007 · Health Checks

Health Checks

who gets traffic, and how not to remove everyone at once

Watch the 4-min video

In one line: health checks decide which backends get traffic: active checks probe an endpoint, passive checks watch real errors; keep liveness shallow, use readiness to hold traffic until an instance can really serve, and beware deep checks on shared dependencies, which can remove every server at once.

1 Two ways to fail

A server is up and answers pings, but every real request returns an error, and the load balancer keeps sending it a third of your traffic. Or the opposite: one database hiccup, and the balancer marks *every* server unhealthy at once. Health checks decide who gets traffic, so both mistakes hurt.

2 Active and passive checks

Active checks: the load balancer calls an endpoint such as `GET /health` every few seconds. After a few failures in a row (the *unhealthy threshold*) it removes the server; after a few successes in a row (the *healthy threshold*) it adds it back.

Passive checks, also called **outlier detection**, watch real traffic: if a backend returns too many errors or timeouts, it is **ejected** for a while, then tried again. Passive checks catch problems a health endpoint can't see, such as one bad code path. Use both.

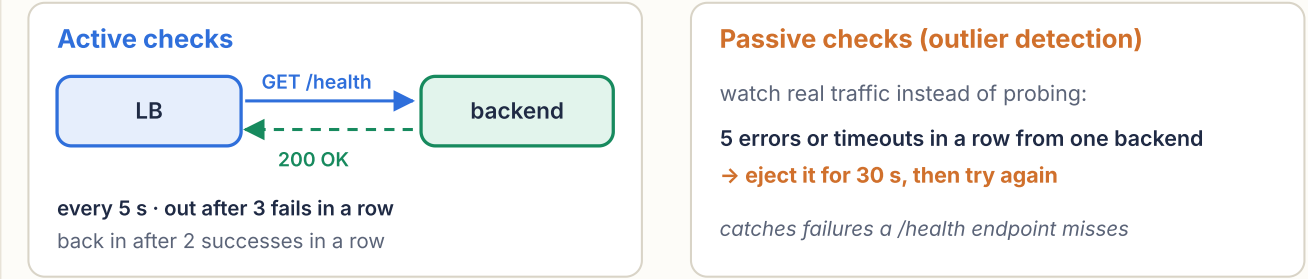


Figure 1. Active checks probe an endpoint on a schedule; passive checks judge each backend by the real responses it sends.

3 Liveness, readiness and startup probes

Kubernetes makes a distinction that is useful everywhere:

- **Liveness**: is this process alive, or stuck (deadlocked, out of memory)? If it fails, the container is **restarted**.
- **Readiness**: can this instance serve traffic *right now*? If it fails, the pod is **removed from the Service's endpoints**, but not restarted.
- **Startup**: gives a slow-starting app time; liveness and readiness only begin once it passes.

WORKED EXAMPLE: A 40-SECOND CACHE WARM-UP

A service starts and spends 40 seconds loading a large cache into memory. Its liveness probe passes, because the process is fine. Its readiness probe fails until the cache is warm, so it receives no traffic yet. Once loaded, readiness passes and traffic flows in. If you had used liveness for this check, Kubernetes would kill and restart the container every time, forever: a crash loop.

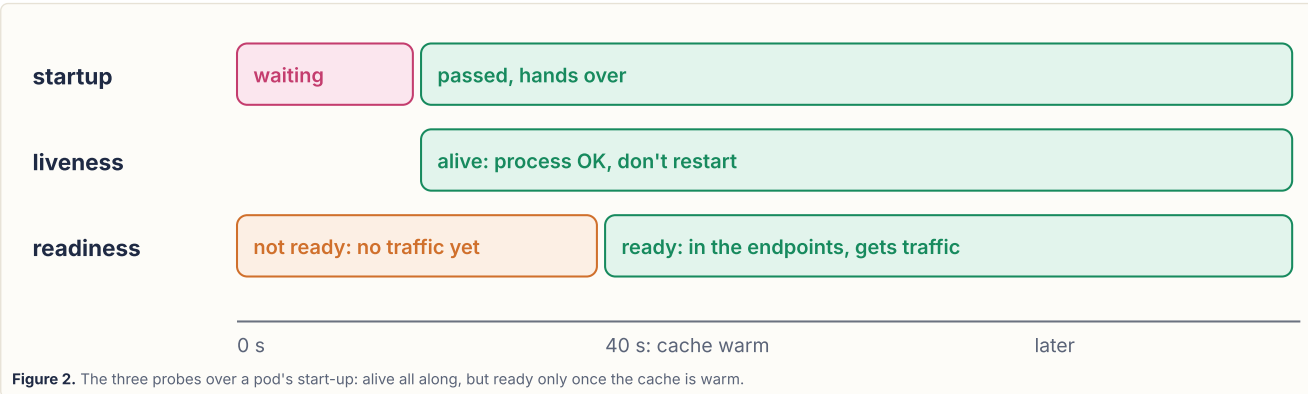


Figure 2. The three probes over a pod's start-up: alive all along, but ready only once the cache is warm.

4 How deep should a check go?

A **shallow** check confirms the process responds. A **deep** check also queries the database, the cache and downstream services. Deep checks catch more problems, but they are dangerous: if the **shared** database slows down, every instance fails its check at the same moment, the load balancer removes all of them, and a partial slowdown becomes a total outage. Worse, the app could still have served cached pages or a friendly error.

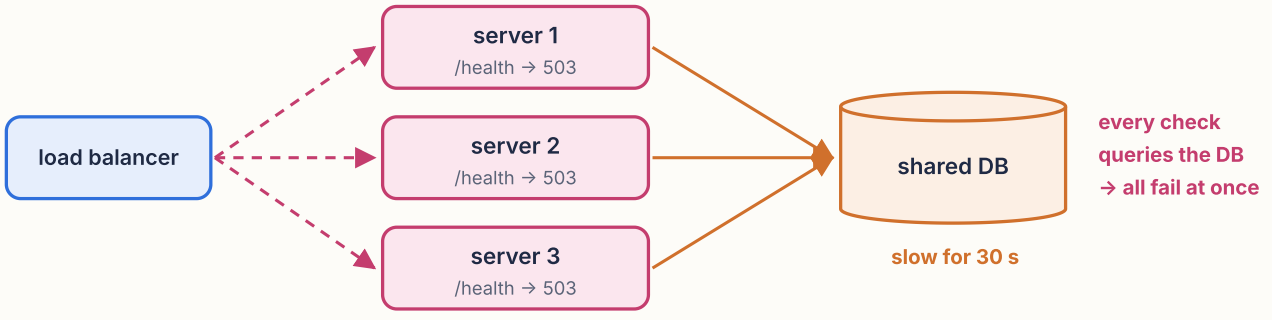


Figure 3. Deep checks on a shared dependency: one slow database makes every server look unhealthy at once.

5 Guidelines

- **Liveness: shallow.** Only the process itself. Never the database.
- **Readiness: what the instance owns.** Its warm cache, its config, its local disk. Be careful with shared dependencies.
- **Handle a failing dependency inside the app** with errors, timeouts and degraded responses, instead of disappearing from the pool.
- **Fail open.** If too many hosts look unhealthy, send traffic to all of them anyway. Envoy's *panic threshold* (default 50%) does this.
- **Tune the numbers.** Intervals of a few seconds and thresholds of 2 or 3 avoid *flapping* (in, out, in, out).
- **Make health endpoints cheap**, and exclude them from rate limits and auth.
- **Graceful shutdown:** on SIGTERM, fail readiness first, finish in-flight requests, then exit.

COMMON MISTAKES

- A liveness probe that checks the database, so a DB blip restarts every pod.
- `/health` that always returns 200, even when the app can't serve.
- No startup or readiness probe on a slow starter, so it gets traffic before it's ready, or is killed while booting.
- Thresholds of 1, so one slow response removes a server.

6 Key terms

Active check: the balancer probes an endpoint on a schedule

Outlier ejection: temporarily removing a misbehaving backend

Readiness probe: can it serve now? fail → no traffic

Flapping: a backend bouncing in and out of the pool

Passive check: judging backends by real responses

Liveness probe: is the process alive? fail → restart

Startup probe: extra time for slow starters

Panic threshold: too many unhealthy → send to all

7 Interview questions

Q1. Should a health check verify the database? BASIC
Usually not for liveness, and only carefully for readiness, because the database is shared by every instance; a DB problem should be handled in the app, not by removing every server.

Q2. Why can deep health checks be dangerous? BASIC
If a shared dependency slows down, every instance fails at once and the load balancer removes them all, turning a partial problem into a full outage.

Q3. What's the difference between liveness and readiness? BASIC
Liveness asks if the process is alive; failing it restarts the container. Readiness asks if it can serve traffic now; failing it removes the pod from the endpoints without restarting it.

Q4. A new pod takes 60 s to start and keeps restarting. Why? SCENARIO
The liveness probe fails before the app is up, so Kubernetes kills it. Add a startup probe (or a longer initial delay) and use readiness to hold traffic until it's ready.

Q5. Health checks pass, but users see errors from one server. How do you catch that? SCENARIO
Add passive health checking (outlier detection) so a backend with a high error or timeout rate on real traffic is ejected, and make the active check exercise the real serving path.

Q6. How do you shut down a server without dropping requests? TRICKY
On SIGTERM, start failing readiness so the balancer stops sending new requests, keep serving in-flight ones (connection draining, a grace period), then exit.

Next: SD-G-008 · Service Discovery

Service Discovery

how services find each other when addresses keep changing

Watch the 4-min video

In one line: service discovery keeps a live registry of which instances exist and where, so callers can find them by name; in client-side discovery the caller looks up and balances itself, in server-side discovery a load balancer does it, DNS is the simplest form, and a service mesh combines both.

1 Addresses that never sit still

Your order service needs to call the payment service. But payments runs as twelve containers, their IP addresses change on every deploy, and autoscaling adds and removes them all day. Hard-coding addresses is impossible. Services need a way to find each other by **name**: **service discovery**.

2 The service registry

At the centre is a **service registry**: a live directory of which instances exist and where. Instances **register** when they start (name, address, port, maybe a version tag) and **deregister** when they stop. Health checks or heartbeats with a TTL remove the ones that die silently.

Common registries: **Consul**, **etcd**, **ZooKeeper** and Netflix **Eureka**. In Kubernetes, the API server and its **EndpointSlices** play this role, updated automatically as pods become ready or go away.

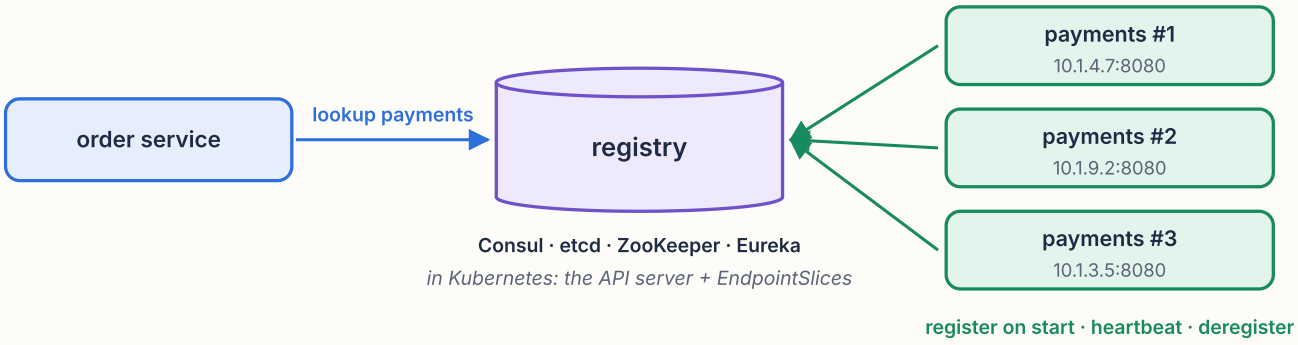


Figure 1. Instances register and send heartbeats; callers ask the registry where a service lives.

3 Client-side and server-side discovery

In **client-side discovery** the calling service queries the registry itself, gets the list of healthy instances, and picks one with its own load balancing, usually through a library. Netflix's Eureka with the Ribbon client worked this way. It's efficient (no extra hop) and allows smart algorithms such as least requests, but every client language needs the library, and discovery logic spreads into your code.

In **server-side discovery** the client calls a fixed address, a load balancer or router, which queries the registry and forwards the request. AWS load balancers with target groups work this way. Clients stay simple and language-independent, but you add a network hop, and the router must be highly available.

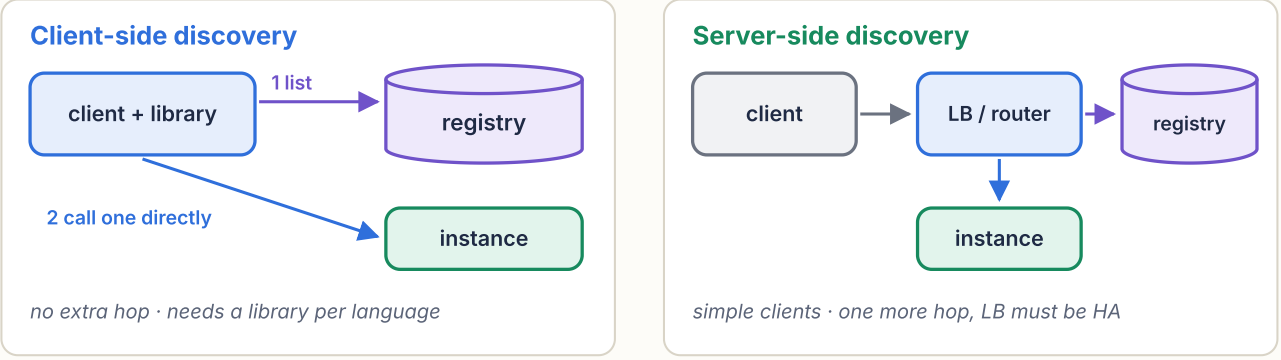


Figure 2. Client-side: the caller asks the registry and calls an instance directly. Server-side: the caller hits a router that does the lookup.

4 DNS-based discovery

The simplest form: a service name resolves to the IP addresses of its instances, and **SRV records** can include ports too. Every language supports DNS, so there is nothing to install. The catch is **caching**: clients and resolvers may keep old answers until the TTL expires (and some runtimes cache longer than the TTL), so removed instances can keep receiving traffic for a while. Keep TTLs short (5 to 30 seconds) and still handle connection errors with retries.

WORKED EXAMPLE: A KUBERNETES SERVICE

You create a Service named `payments`. Cluster DNS gives it the name `payments.default.svc.cluster.local`, which resolves to a stable virtual IP (the ClusterIP). **kube-proxy** programs rules (iptables or IPVS) on every node, so traffic to that IP is spread across the **ready** pods only. Pods come and go; the name and the IP stay the same. A **headless** Service (`clusterIP: None`) skips the virtual IP, and DNS returns the pod IPs directly, which suits clients that balance themselves, such as gRPC.

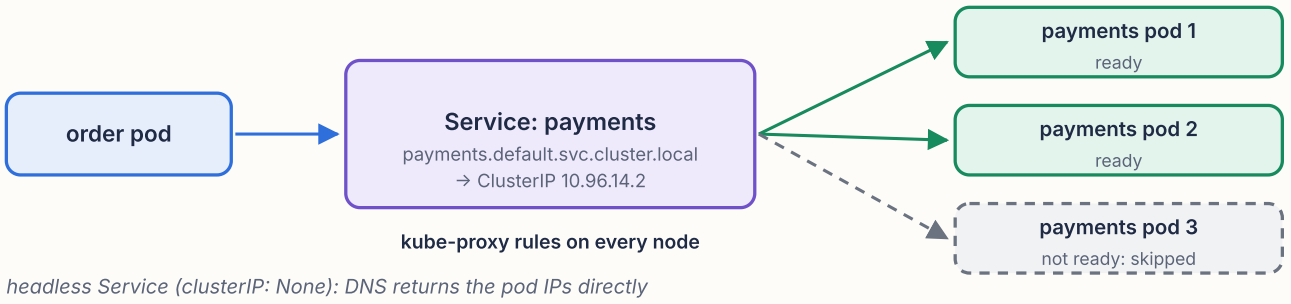


Figure 3. A Kubernetes Service: a stable name and virtual IP in front of the ready pods; a pod that is not ready gets no traffic.

5 Service mesh: both at once

Service meshes such as **Istio** and **Linkerd** combine the styles. A sidecar proxy next to each service gets the endpoint list from the mesh's control plane and balances load locally. Your code just calls a name, as if it were server-side, but the work happens client-side, in the sidecar, with retries, timeouts and mutual TLS included.

COMMON MISTAKES

- Hard-coding IPs or long-lived DNS caches (the JVM's old default cached forever), so callers keep hitting dead instances.
- Instances that crash without deregistering, and no heartbeat TTL to clean them up.
- A single registry node: the registry is critical infrastructure and must be replicated (Consul and etcd run 3 or 5 nodes with Raft).
- Gaps during deploys: a new instance registered before it's ready, or an old one removed while requests are in flight.

6 Key terms

Service registry: live directory of instances and addresses

Heartbeat / TTL: proof of life; silent instances expire

Server-side discovery: a router looks up and forwards

ClusterIP: a Kubernetes Service's stable virtual IP

Registration: an instance announces itself on start

Client-side discovery: caller looks up and load balances itself

SRV record: DNS record with host and port

Headless Service: DNS returns pod IPs, no virtual IP

7 Interview questions

Q1. Compare client-side and server-side discovery. BASIC

In client-side discovery the caller queries the registry and load balances itself, avoiding an extra hop but needing a library in every language; in server-side discovery the caller hits a load balancer or router that does the lookup, keeping clients simple but adding a hop.

Q2. What does a Kubernetes Service give you? BASIC

A stable DNS name and virtual IP that kube-proxy spreads across ready pods: server-side discovery built into the platform.

Q3. What are the downsides of DNS-based discovery? BASIC

Caching: clients and resolvers may use stale answers until the TTL expires, so removed instances still get traffic; DNS also gives no load information and only coarse balancing.

Q4. After a deploy, some requests still go to old, terminated instances for a minute. Why? SCENARIO

Callers cached DNS or a stale registry list. Shorten TTLs, make sure instances deregister before shutting down, drain connections, and retry on connection errors.

Q5. How does a gRPC client balance across pods in Kubernetes? SCENARIO

A normal Service pins its HTTP/2 connection to one pod. Use a headless Service so DNS returns all pod IPs and the gRPC client balances per request, or use a mesh sidecar.

Q6. What happens if the registry goes down? TRICKY

New instances can't be found, but well-built clients keep using their last known list. Run the registry replicated (Raft quorum of 3 or 5) and let clients cache results so a registry outage doesn't become a service outage.

Next: SD-G-009 · Reverse Proxy Configuration

Reverse Proxy Configuration

the settings that matter in production

Watch the 4-min video

In one line: a reverse proxy routes requests to named upstreams, and in production you must set buffering, connect/read/client timeouts that are shorter than the caller's, bounded retries for idempotent requests only, connection and rate limits, body size limits, compression and request logging, because the defaults let one slow dependency take everything down.

1 Defaults are not a configuration

You put NGINX in front of your API, keep the default config, and ship it. Then a slow client ties up your workers, a dead backend makes every request hang for a minute (NGINX's default `proxy_read_timeout` is 60 s), and one retry storm takes everything down. A reverse proxy is only as good as its configuration.

A **reverse proxy** sits in front of your servers and handles requests on their behalf (Phase 2, lesson 27). Behind it are **upstreams** (NGINX) or **clusters** (Envoy): named groups of backend servers. The proxy matches each request by host and path to an upstream, picks a healthy server, and forwards it.

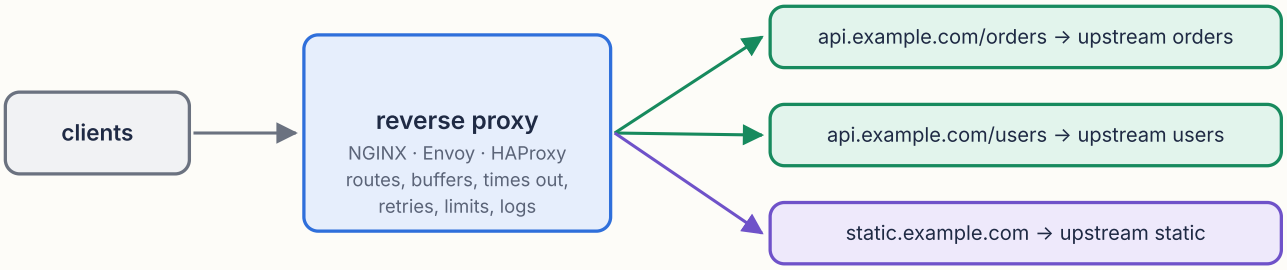


Figure 1. One proxy, many routes: each host and path maps to a named upstream group of servers.

2 Routing and buffering

Buffering protects your application workers from slow networks. With **request buffering**, the proxy reads a slow client's whole upload first, then sends it to the backend in one go. With **response buffering**, the backend writes its response to the proxy quickly and moves on, while the proxy trickles it to a phone on a weak network. Either way, a backend worker isn't held hostage for 20 seconds.

Exception: **streaming**. For server-sent events or LLM tokens, turn response buffering off (`proxy_buffering off`, or `X-Accel-Buffering: no`), or users see nothing until the end.

3 Timeouts

- **Connect timeout**: 1 to 2 seconds. A dead backend should fail fast, not hang.
- **Read (response) timeout**: fits your slowest *normal* request, such as 5 or 10 seconds, not a default of a minute.
- **Client timeouts**: drop idle or extremely slow clients (header and body read timeouts, idle keep-alive timeout).

The rule: **every timeout is shorter than its caller's**. Then the inner call gives up first, and there's still time to return a clean error or a fallback.

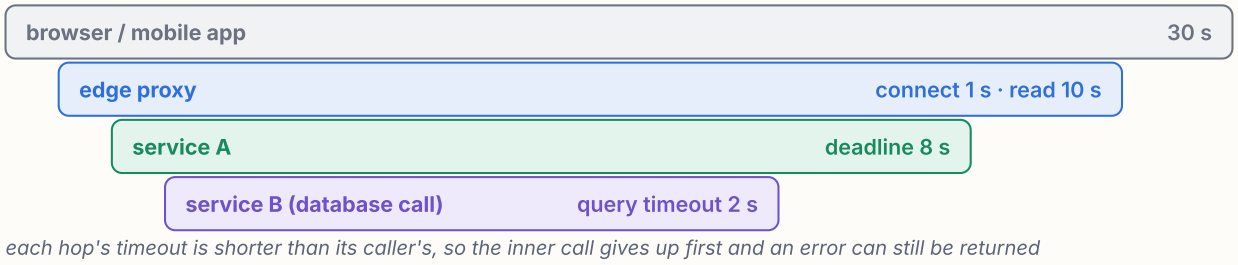


Figure 2. Timeouts nest: each layer gives up before the layer that called it.

4 Retries, with a budget

Retries hide brief failures, but they multiply load. Retry only **idempotent** requests (GETs, or writes with an idempotency key), only on connection errors or specific codes such as 503, cap attempts (1 or 2 retries), give each try its own **per-try timeout**, and use a **retry budget**: retries may never exceed, say, 20% of traffic. Unlimited retries at every layer turn a small outage into a **retry storm**.

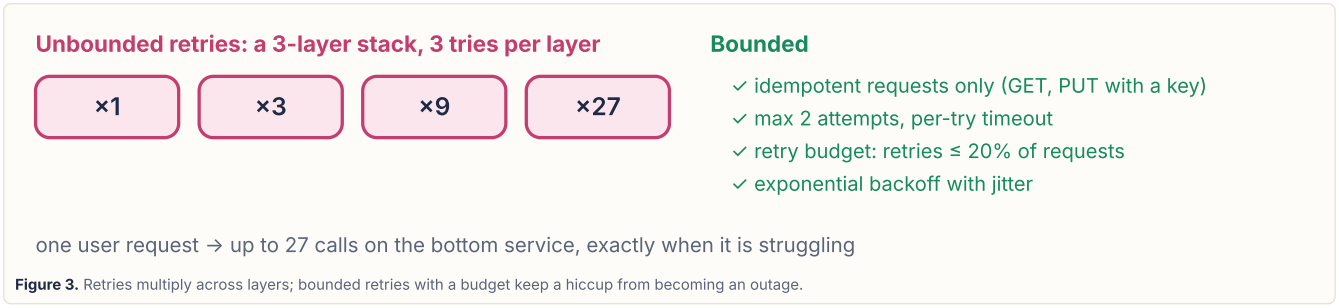


Figure 3. Retries multiply across layers; bounded retries with a budget keep a hiccup from becoming an outage.

5 Protect the backends

- **Connection limits** per upstream, and **keep-alive pools** to upstreams so you don't pay a new TCP and TLS handshake per request.
- **Rate limits** per client IP or API key.
- **Body size limit** (`client_max_body_size`), so one 2 GB upload can't fill your disk or memory.
- **Compression** (gzip or brotli) for text responses such as JSON and HTML.
- **Logging**: a request ID on every request (and passed to the backend), plus upstream connect and response times, so slow requests can be traced.

WORKED EXAMPLE: POLICY IN AN ENVOY SIDECAR

An Envoy sidecar runs next to each service. The calling code makes a plain HTTP call to `http://payments`. The sidecar applies the policy from config: a **2-second per-try timeout**, up to **2 retries** on connection failures and 503s (with backoff), and a **circuit breaker** that caps pending requests and connections to payments. The code stays simple, the policy is the same for every language, and it can be changed without a redeploy.

COMMON MISTAKES

- Inner timeouts longer than outer ones, so the caller gives up while the work continues uselessly.
- Retrying POSTs that create orders or charge cards, causing duplicates.
- Buffering on for streaming endpoints, so tokens arrive all at once at the end.
- No request IDs, so nobody can match a slow request across the proxy and the services.

6 Key terms

Upstream / cluster: a named group of backend servers

Response buffering: take the response fast, deliver it slowly

Read timeout: how long to wait for the response

Retry budget: retries capped as a share of traffic

Request buffering: read the whole request before forwarding

Connect timeout: how long to wait to open a connection

Per-try timeout: a limit on each single attempt

Circuit breaker: stop calling a failing backend for a while

7 Interview questions

Q1. What should you configure on a reverse proxy in production? BASIC
Routing to upstreams, request and response buffering, connect, read and client timeouts, bounded retries for idempotent requests only, connection limits, rate limits, body size limits, compression and request logging.

Q2. Why not leave the defaults? BASIC
Default timeouts are too long and retries can multiply load, so a slow dependency ties up everything and can take the whole system down.

Q3. A backend died and every request hung for 60 seconds. What do you change? SCENARIO
Set a short connect timeout (1 to 2 s) and a realistic read timeout, add health checks or outlier ejection to stop sending to the dead backend, and return a fast error.

Q4. Users of your AI chat see the whole answer appear at once after 20 seconds. Why? SCENARIO
The proxy buffers the response. Turn off response buffering for that streaming route (and make sure compression or other middleware doesn't buffer either).

Q5. Why must inner timeouts be shorter than outer ones? TRICKY
If the inner call waits longer than its caller, the caller has already given up and the inner work is wasted; worse, the caller may retry, doubling the load. Shorter inner timeouts let the service fail fast and still answer.

Q6. How do retries turn a small outage into a big one? TRICKY
Each layer retries, so attempts multiply (3 × 3 × 3 = 27) against the struggling service, exactly when it has the least capacity. Retry budgets, per-try timeouts, backoff with jitter and retrying at only one layer prevent this.

Next: SD-G-010 · Traffic Shaping and Load Shedding

Traffic Shaping and Load Shedding

reject some requests on purpose, so the important ones survive

Watch the 4-min video

In one line: under overload, queues grow, clients time out and retry, and useful throughput collapses; load shedding rejects excess work early and cheaply (a fast 503), drops low-priority traffic first, caps in-flight requests and queues with admission control, and clients cooperate with backoff and retry budgets.

1 Why overload is so dangerous

It's Black Friday. Traffic is five times normal and everything slows down together: checkout, search, analytics beacons. Soon nothing works at all. Why does it get *worse* than just slow?

When a server gets more work than it can do, requests wait in queues. Latency grows, clients time out and **retry**, and the retries add even more load. The server spends its time on requests whose clients have already given up. The CPU is at 100%, but **goodput** (requests completed in time, that someone still wants) collapses.

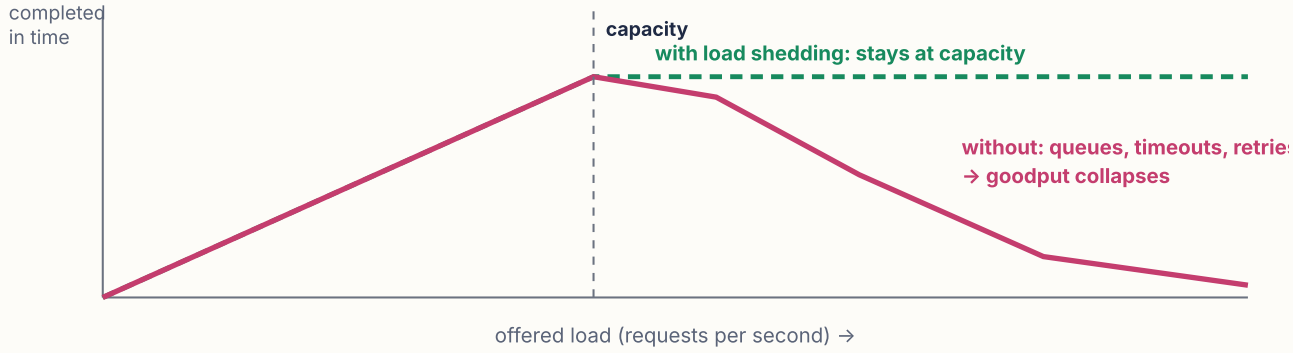


Figure 1. Beyond capacity, an unprotected server completes fewer and fewer requests in time; with load shedding it keeps serving at capacity and rejects the rest.

2 Load shedding: fail fast, on purpose

Load shedding means rejecting excess work early and cheaply, with a fast error such as **503 Service Unavailable** (with a `Retry-After` header), instead of accepting it and doing it badly. A quick rejection costs about a millisecond; the client can back off or try elsewhere; and the requests you do accept finish quickly. It feels wrong to refuse users, but it's the difference between serving 100% of capacity and serving almost nobody.

3 Not all traffic is equal

Give requests a **priority**. Under pressure, drop the sheddable tier first, then the important tier, and protect critical traffic to the end. The gateway can tag each request (by route, client or a header) so every service downstream applies the same policy.

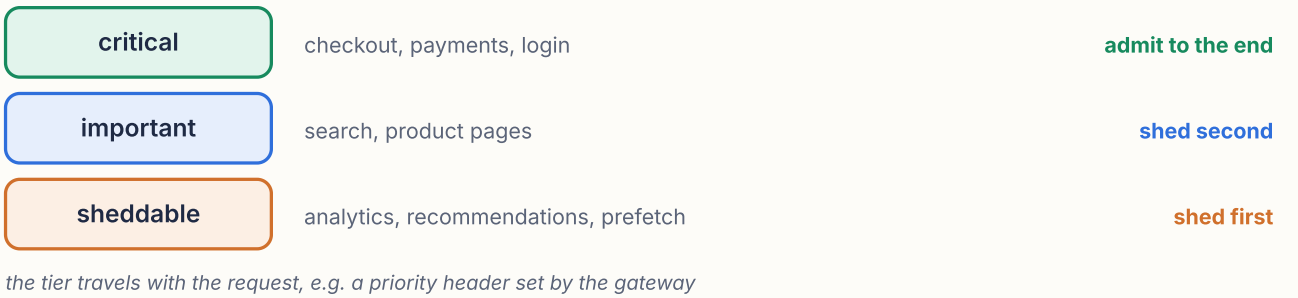


Figure 2. Three priority tiers. Shedding starts at the bottom; critical requests are admitted as long as possible.

4 Knowing you're overloaded: admission control

Admission control checks before accepting work: cap the number of **in-flight** requests and the **queue size**, so requests never wait forever. A request that would wait longer than its client's timeout is better rejected now.

Adaptive concurrency limits go further: they watch latency and shrink the allowed concurrency when latency rises, and grow it when latency is fine, like TCP congestion control. Netflix's *concurrency-limits* library and Envoy's adaptive concurrency filter work this way, so you don't have to guess the right limit.

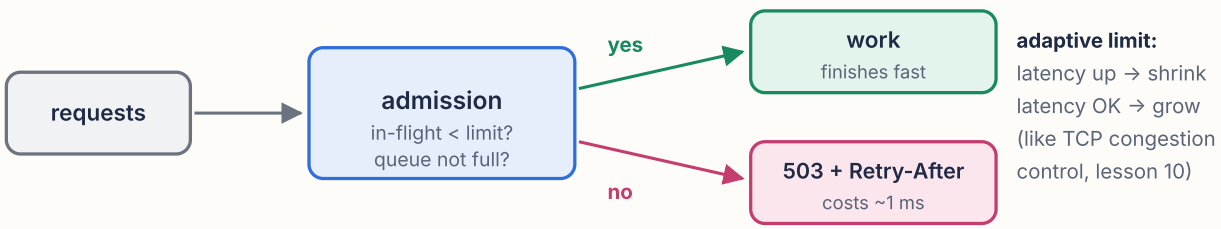


Figure 3. Admission control: work is accepted only if there's room; otherwise the request gets a cheap, immediate 503.

WORKED EXAMPLE: BLACK FRIDAY

CPU on the API fleet crosses 80%. The gateway starts dropping analytics and tracking requests with a 503. Recommendations fall back to a cached "popular items" list. Checkout requests carry a `priority: critical` header and are always admitted. Revenue keeps flowing, latency on checkout stays normal, and nobody notices the missing analytics events.

5 Shaping before overload

- **Rate limits and quotas** cap each client, so one integration can't eat everyone's capacity (Phase 2, API lesson 12).
- **Queues** smooth bursts, so background jobs run at a steady pace instead of spiking.
- **Cooperative clients** honour `Retry-After`, use exponential backoff with jitter, and keep a retry budget, so they don't hammer a struggling service.

COMMON MISTAKES

- Unbounded queues: they hide overload until every request is late.
- Shedding at the database instead of the edge, after the expensive work is done.
- Treating all requests the same, so checkout fails alongside analytics.
- Clients that retry 503s immediately and forever.

6 Key terms

Overload: more offered work than capacity

Load shedding: rejecting excess work early and cheaply

Admission control: deciding whether to accept a request at all

Retry-After: header telling clients when to come back

Goodput: useful requests completed in time

Priority tier: critical, important, sheddable

Adaptive concurrency: a limit that follows latency

Retry budget: cap on retries as a share of traffic

7 Interview questions

Q1. How do you keep a system alive under overload? BASIC

Shed load early with fast rejections, prioritise critical traffic and drop low-priority tiers first, cap in-flight requests and queue sizes, and use adaptive concurrency limits, while clients back off with jitter.

Q2. Why not just accept everything? BASIC

Because queues grow, requests time out, retries add load, and throughput collapses for everyone; the server ends up doing work nobody is waiting for.

Q3. Design load shedding for an e-commerce API on Black Friday. SCENARIO

Tag requests by priority at the gateway (checkout critical, browse important, analytics sheddable), shed from the bottom when CPU or latency crosses thresholds, serve cached fallbacks for recommendations, cap queues, and return 503 with Retry-After.

Q4. Where should load shedding happen? SCENARIO

As early as possible, at the gateway or the service's front door, before expensive work; and in every service, since each one can be overloaded independently.

Q5. Why is an unbounded queue a bad idea? TRICKY

It converts overload into latency: every request waits longer and longer, most time out, and the server still processes them. A bounded queue plus fast rejection keeps latency low for the requests it does accept.

Q6. How do adaptive concurrency limits work? TRICKY

They measure latency and adjust the allowed number of in-flight requests, like TCP congestion control: if latency rises above the baseline, the limit shrinks; if it stays low, the limit grows. No fixed number to guess.

Next: SD-G-011 · Global Server Load Balancing and Regional Routing backendroadmap.com

Global Server Load Balancing and Regional Routing

send each user to the right region, and survive losing one

Watch the 4-min video

In one line: a global server load balancer (GSLB) picks the region for each user, by location, latency, health, load and data-residency rules, using DNS answers (simple, but failover waits for the TTL) or anycast (one IP everywhere, failover in seconds); the other regions need spare capacity and the data must be there too.

1 One region is too far and too fragile

Your app runs in Virginia. A user in Mumbai waits about 200 ms for every round trip, and a page that needs several round trips feels slow. A single region outage takes everyone offline. So you deploy to three regions, and a new question appears: how does each user reach the **right** region, and what happens when one goes dark?

2 The global load balancer

A **global server load balancer** (GSLB) decides which region or data centre serves each user. It sits **above** your regional load balancers. It weighs:

- where the user is, and the latency to each region;
- the health of each region (probes from many places);
- how loaded each region is;
- rules about where data may live (data residency).

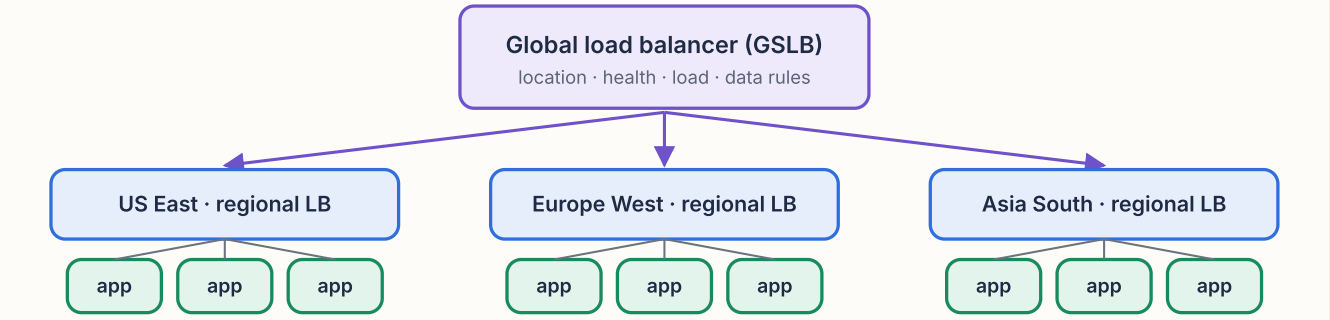


Figure 1. Two levels of load balancing: a global layer chooses the region, a regional load balancer chooses the server.

3 Two ways to steer: DNS and anycast

DNS-based: the user's resolver asks for your domain, and the GSLB answers with the IP of the chosen region. **Geo** routing picks by location, **latency** routing by the lowest measured latency, **weighted** routing splits traffic (Phase 2, lesson 17). AWS Route 53 and Azure Traffic Manager work this way. The catch: **caching**. Resolvers and clients keep the old answer until the TTL expires, so failover takes at least a TTL, often longer.

Anycast: every edge location announces the **same IP address**, and internet routing (BGP) carries each user to the nearest one. When a location withdraws its route, traffic shifts within seconds, with no DNS caching involved. Cloudflare, Google's global load balancer and Azure Front Door use anycast at the edge, then forward to backend regions over their own network.

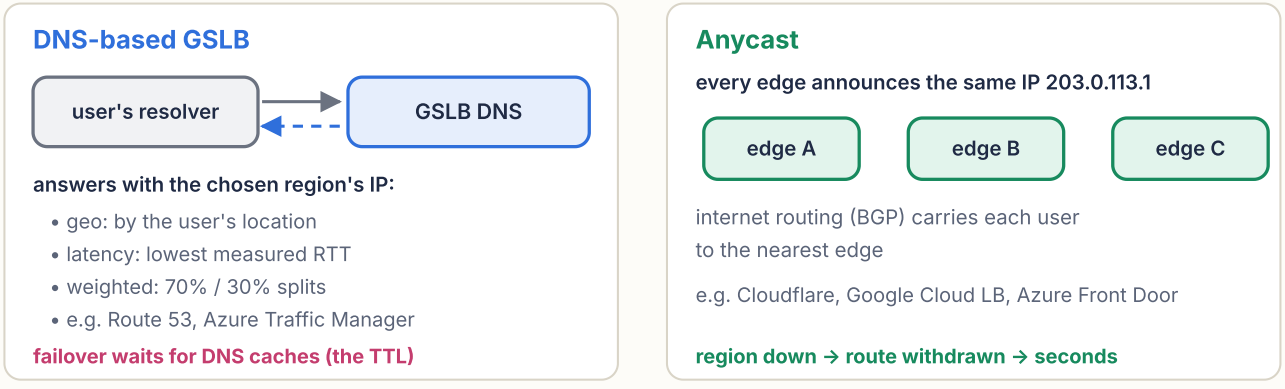


Figure 2. DNS steers with answers and is limited by caching; anycast steers with routing and fails over in seconds.

WORKED EXAMPLE: AZURE FRONT DOOR

Front Door has edge locations worldwide, all on one anycast address. A user in Mumbai connects to a nearby edge, which terminates TLS (saving round trips over the long distance) and forwards the request to the closest healthy backend region, Central India. Front Door probes each region every few seconds. If Central India fails its health probes, requests shift to Southeast Asia automatically, without any DNS change.

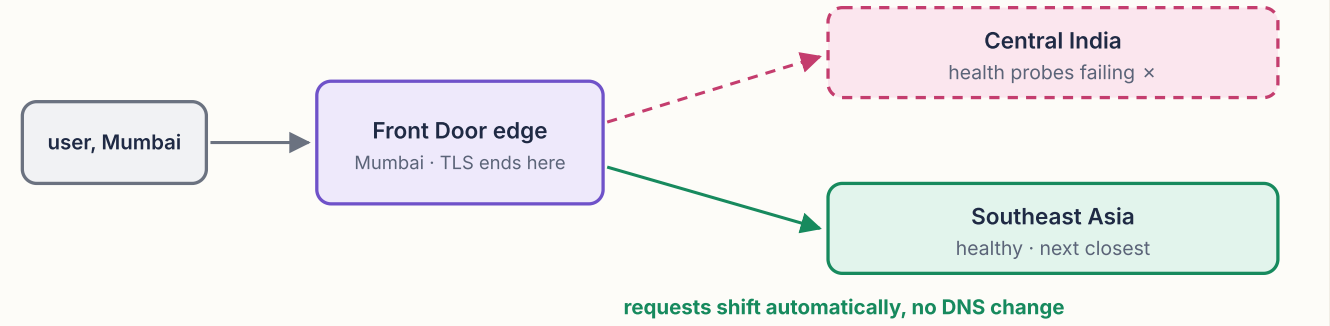


Figure 3. A regional failure behind an anycast edge: the edge stops sending to the failing region and uses the next closest one.

4 It's not only about speed: data

Data residency. Laws such as the EU's GDPR, or customer contracts, may require European users' data to stay in Europe. Those users must route to a European region, even if another one is faster at that moment.

Home regions. A user's data might live in one home region. Requests must go there (route by user, not just by location), or the data must be replicated where the user lands.

5 When a region fails

- **Spare capacity.** Traffic moves to the other regions, so they must be able to absorb it. With three regions, each needs headroom for half of a lost region's load.
- **Failover storms.** If everyone moves at once, the next region can be overloaded too, and fail. Shift gradually where possible, and shed low-priority load (lesson 10).
- **Writes.** If the failed region was the primary for some data, another region must be promoted. That's the next lesson.

COMMON MISTAKES

- Long DNS TTLs (an hour or a day) on records you expect to fail over.
- Routing purely by latency and breaking data-residency rules.
- Regions sized only for their own traffic, so a failover takes down the survivors.
- Health checks from a single location, so a network blip near the checker looks like a region outage.

6 Key terms

GSLB: global load balancer that picks a region

Latency routing: answer with the lowest-latency region

Anycast: one IP announced from many locations

Data residency: rules on where data may be stored

Geo routing: answer by the user's location

Weighted routing: split traffic by percentages

BGP: the routing protocol between internet networks

Failover storm: everyone moving at once and overloading the next region

7 Interview questions

Q1. How do you route users to the nearest region? BASIC
With a global load balancer: DNS-based geo or latency routing that answers with the best region's IP, or anycast IPs that send users to the nearest edge, which forwards to the closest healthy region.

Q2. What happens when a region fails? BASIC
Health checks mark it down and traffic shifts to other regions, within seconds with anycast and after the TTL with DNS, so those regions need spare capacity and the data must be available there.

Q3. Users in Germany must have their data stored in the EU. How does routing change? SCENARIO
Route them to an EU region by account or location rules, even if another region is faster, keep their data and backups in the EU, and fail over only to another EU region.

Q4. You use DNS failover with a 300 s TTL, but some users take 15 minutes to move. Why? SCENARIO
Some resolvers and clients cache longer than the TTL, and long-lived connections stay open. Use short TTLs, close connections on the failed side, or move to anycast at the edge.

Q5. Why might latency-based routing send a user to a far region? TRICKY
It measures from the user's DNS resolver, not the user; a public resolver far away (or without EDNS Client Subnet) makes the far region look closer. Anycast or client-measured latency avoids this.

Q6. Two of three regions must absorb a lost region. How much capacity does each need? TRICKY
Each normally carries a third of the load; after a failure, each carries half. So each region needs headroom for about 1.5× its normal load, plus margin for retries.

Next: SD-G-012 · Failover Strategies

Failover Strategies

active-passive, active-active, and how warm the spare is

Watch the 4-min video

In one line: active-passive keeps one serving site and a hot, warm or cold standby (simple data, slower and riskier failover); active-active serves from every site (no idle capacity, near-instant failover, but multi-site writes need conflict handling or partitioning); either way, detect carefully, avoid split brain, fail back slowly and rehearse.

1 Three in the morning

Your primary region goes down. Do you have a second region already serving traffic, one sitting idle waiting for this moment, or just backups and a runbook? The answer decides whether you're down for seconds, minutes or hours. Two numbers frame the choice (Phase 1, lesson 10): **RTO**, how long recovery may take, and **RPO**, how much recent data you may lose.

2 Active-passive and active-active

In **active-passive**, one site serves all traffic and a standby waits. When the primary fails, you **fail over**: the standby becomes primary. There's only one writer at a time, so data conflicts are rare. The costs: you pay for idle capacity, failover takes time, and it may not work if it's never tested.

In **active-active**, every site serves traffic all the time. If one fails, the others simply absorb its share. There's no idle capacity, and failover is nearly instant, because the other sites are already warm and proven by real traffic. The hard part is data: if several sites accept writes to the same record, you need conflict handling (Phase 2, database lesson 21), or you **partition** data so each record has one home site.

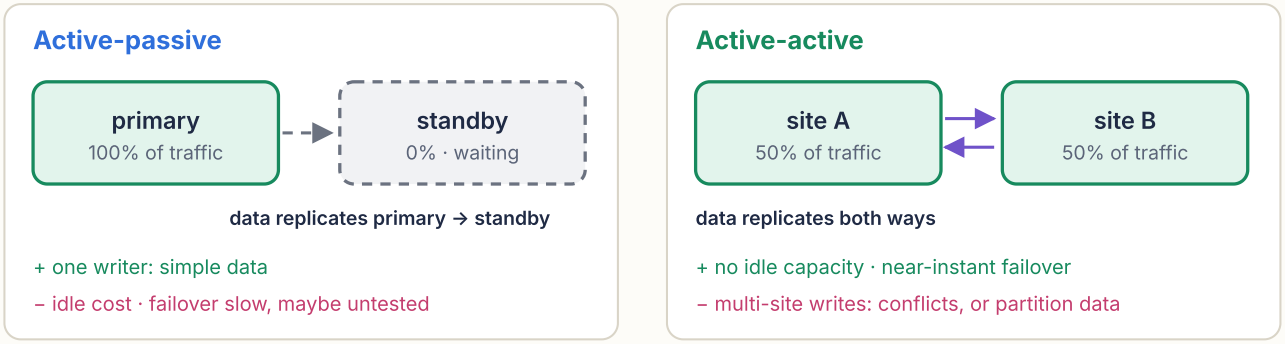


Figure 1. Active-passive keeps a spare; active-active uses every site. The difference shows up in cost, recovery time and data complexity.

3 How warm is the standby?

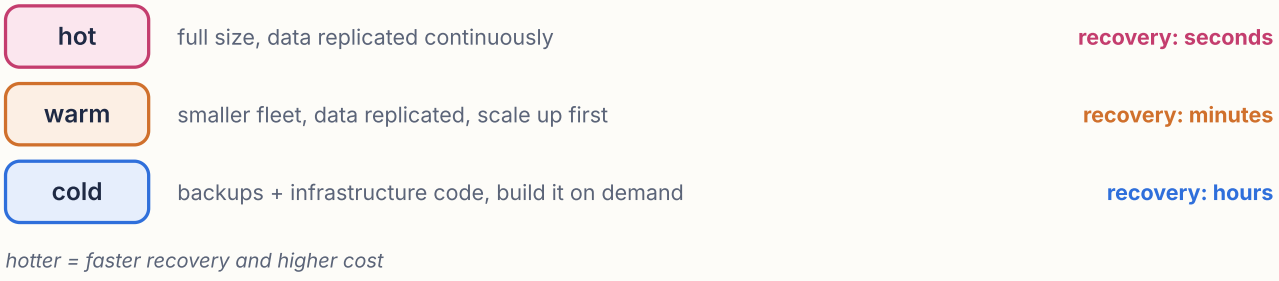


Figure 2. Standby temperatures: cheaper standbys recover more slowly.

Pick the temperature from the RTO: a payments system might need hot; an internal reporting tool can live with cold. Many teams use **warm**: the database replica is always running (that's what limits data loss), while the application fleet runs small and scales up during failover.

4 Detecting failure and moving traffic

Detection: health checks from several locations must agree before you act, so a network blip near one checker doesn't flip regions. **Moving traffic**: DNS failover changes the answer, but clients wait out the TTL; anycast withdraws a route and shifts within seconds (lesson 11).

Automate it where you can, but guard against **split brain**: two sites both believing they're primary and both accepting writes. The standard defence is **fencing**: before promoting the new primary, make sure the old one can no longer write (revoke its role, cut its access, or use a consensus system such as etcd or a cloud's managed failover to hold the leader lock).

WORKED EXAMPLE: EAST US GOES DOWN

Monitoring confirms the outage from three locations. The team, or automation, fences East US, promotes the database replica in **West US** to primary, scales the warm application tier up to full size, and switches traffic with Front Door or DNS. Writes resume in West US. Data written in East US after the last replication may be lost: with asynchronous replication lagging 5 seconds, the RPO is about 5 seconds.



Figure 3. A failover runbook for a warm standby. Fencing comes before promotion, to avoid two primaries.

5 Failback, and practice

When the old primary returns, don't rush back. **Resync** its data from the new primary (it may hold writes that never replicated: reconcile or discard them), verify it, and move traffic back gradually, or simply keep the new primary. And **practise**: run game days that fail over on purpose. A failover plan that's never been tested is a hope, not a plan.

COMMON MISTAKES

- A standby that was never tested, with expired credentials or a missing config, discovered during the outage.
- Promoting the replica without fencing the old primary: split brain and diverging data.
- Automatic failback that flaps between regions.
- Forgetting the other dependencies: DNS, secrets, queues and third-party allow-lists for the standby's IPs.

6 Key terms

Failover: moving service to a standby after a failure

Active-passive: one site serves, one waits

Hot / warm / cold: how ready the standby is

Split brain: two sites both acting as primary

Failback: returning to the original primary

Active-active: every site serves all the time

RTO / RPO: time to recover / data you may lose

Fencing: making sure the old primary can't write

7 Interview questions

Q1. Compare active-active and active-passive setups. BASIC
Active-active serves traffic from every site, so there's no idle capacity and failover is nearly instant, but multi-site writes need conflict handling or data partitioning. Active-passive keeps a hot, warm or cold standby, which is simpler for data but wastes capacity and makes failover slower and riskier.

Q2. How do you fail back safely? BASIC
Resync the old primary's data from the new primary and reconcile any unreplicated writes, verify it, shift traffic gradually (or keep the new primary), and rehearse failovers regularly.

Q3. What decides hot, warm or cold standby? BASIC
The RTO and RPO you need versus cost: hot for seconds of downtime, warm for minutes, cold for hours.

Q4. Your RTO is 5 minutes and RPO 1 minute, on a budget. What setup? SCENARIO
A warm standby: a continuously replicated database replica in the second region (RPO of seconds), a small application fleet that autoscales, infrastructure as code, and an automated, rehearsed runbook.

Q5. What is split brain and how do you prevent it? TRICKY
Two sites both believe they're primary and accept conflicting writes. Prevent it by fencing the old primary before promoting, using a quorum or a single lock holder (consensus) to decide who is primary, and requiring multiple observers to agree on failure.

Q6. Why is active-active failover more trustworthy? TRICKY
Every site already serves real traffic, so its capacity, configuration and dependencies are proven daily; a passive standby's problems only appear when you need it.

Next: SD-G-013 · Blue-Green and Weighted Traffic Splitting backendroadmap.com

Blue-Green and Weighted Traffic Splitting

release safely: flip, or shift a little at a time

Watch the 4-min video

In one line: blue-green runs two full environments and flips all traffic from old to new (rollback is a routing change); weighted splitting sends a small share, say 5%, to the new version as a canary, compares its metrics, and shifts step by step to 100%, or back to 0% in seconds.

1 Don't let every customer find the bug

You're about to deploy a new version of the checkout service. If it has a bug, every customer hits it at once. What if you could send it 5% of traffic first, watch the numbers, and switch back in seconds if anything looks wrong? Load balancers make this easy.

2 Blue-green deployment

Blue-green runs two full environments. **Blue** is the current version, serving all traffic. **Green** is the new version, deployed and tested alongside it (smoke tests, maybe internal users). When green is ready, the load balancer switches traffic from blue to green. If something breaks, switch back: **rollback is a routing change, not a redeploy**, so it takes seconds. The cost is running two full environments during the release.

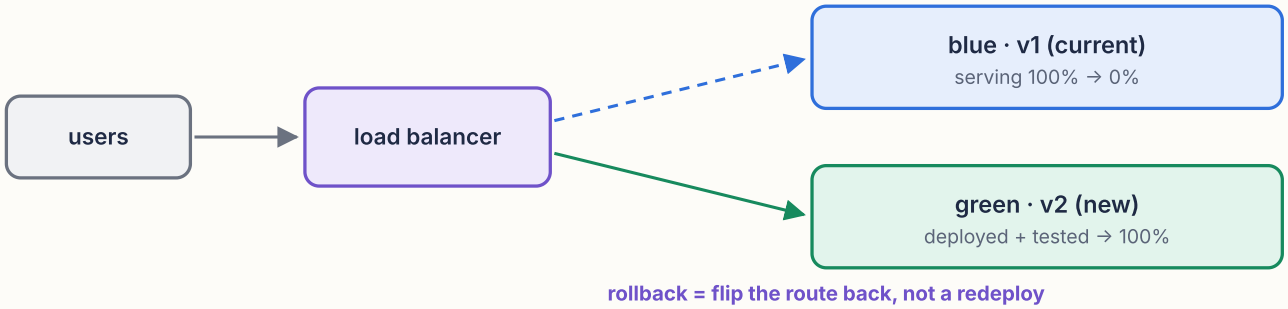


Figure 1. Blue-green: the new version runs beside the old one; the load balancer flips all traffic, and can flip it back.

3 Weighted traffic splitting and canaries

Weighted splitting makes the switch gradual. The load balancer has two target groups, one per version, each with a **weight**. Weights of 95 and 5 send 95% of requests to the old version and 5% to the new one, the **canary** (named after the canaries miners carried to detect gas). AWS ALB weighted target groups, Kubernetes ingress controllers (NGINX canary annotations) and service meshes such as Istio all support this.

WORKED EXAMPLE: V2 OF CHECKOUT

You deploy version 2 next to version 1 and set the weights to 95 and 5. For 15 minutes you compare the canary against the old version: error rate (say 0.2% vs 0.2%), p99 latency (310 ms vs 300 ms) and a business metric (orders completed per visit). They look healthy, so you move to 25%, then 50%, then 100%. If errors spike at any step, the weight goes back to 0, and only that share of users ever saw the problem, for a few minutes.

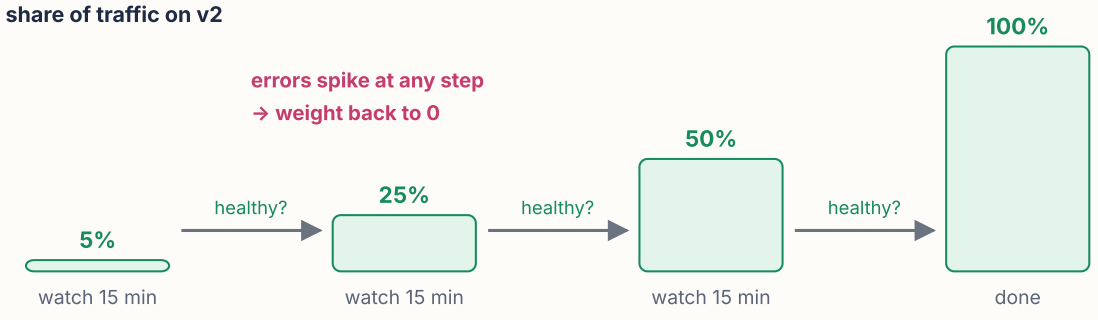


Figure 2. A canary rollout in steps: each step waits, checks the metrics, and only then shifts more traffic.

4 Automate the decisions

Tools like **Argo Rollouts** and **Flagger** shift weights step by step, query metrics (Prometheus, Datadog) after each step, and roll back automatically when a threshold is crossed. That removes the human waiting at 2 a.m., and makes every release follow the same safe path.

5 Traps

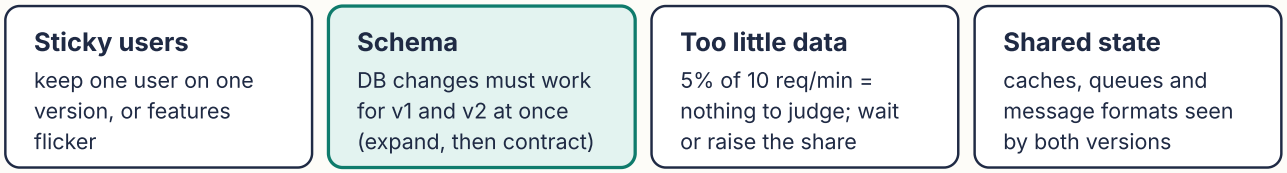


Figure 3. Four things that make canaries misleading or unsafe.

- **Sticky sessions.** A user should stay on one version during the rollout, or features appear and disappear between clicks. Pin by cookie or user ID hash.
- **Database schema.** Both versions run at once, so schema changes must be backward compatible: add the column first (expand), deploy, then remove the old one later (contract).
- **Not enough data.** A 5% canary on a low-traffic service may see a handful of requests; wait longer, or judge on synthetic traffic too.

6 Beyond releases: migrations and mirroring

The same tool moves systems. Breaking up a monolith? Route one path, or a small weight, to the new service, and grow it as confidence builds (the **strangler fig** pattern). Moving to a new region or cloud? Shift weight gradually in DNS or the global load balancer. You can also **mirror** (shadow) traffic: send a copy of real requests to the new system and compare its responses, without users ever seeing them. Mirror only safe, read-only requests, or the new system must not perform side effects.

COMMON MISTAKES

- Judging a canary only on errors, while latency or conversions quietly got worse.
- A destructive migration (dropping a column) in the same release as the canary, so rollback is impossible.
- Mirroring write requests, so the shadow system charges cards or sends emails.

7 Key terms

Blue-green: two full environments, flip all traffic

Weighted target groups: per-version pools with traffic weights

Expand / contract: backward-compatible schema migration

Traffic mirroring: copy requests to a new system, ignore its answers

Canary: a small share of traffic on the new version

Progressive delivery: automated step-by-step rollout

Strangler fig: move traffic to a new system piece by piece

8 Interview questions

Q1. How do load balancers support canary releases? BASIC
By splitting traffic between target groups with weights, like 95 and 5, so a small share hits the new version while you compare metrics, then shifting gradually to 100%, or back to 0 to roll back instantly.

Q2. What's the difference from blue-green? BASIC
Blue-green flips all traffic between two full environments at once; weighted canaries move gradually, so a bad version reaches only a small share of users.

Q3. Which metrics do you compare during a canary? SCENARIO
Error rate, latency percentiles (p95, p99), saturation (CPU, memory), and business metrics such as checkout completion, all compared against the old version over the same period.

Q4. v2 adds a database column. How do you canary it safely? SCENARIO
Expand first: add the column (nullable or with a default) in a migration that v1 ignores, deploy v2 as a canary, and only after v1 is gone run the contract step that removes old columns.

Q5. Why might a canary look healthy but still be bad? TRICKY
Too little traffic to see errors, a share of users that isn't representative (only one region or client type), or problems that show up later, like slow memory leaks or batch jobs. Wait long enough and segment metrics.

Q6. How would you use traffic mirroring to test a rewritten service? TRICKY
Copy read-only production requests to the new service asynchronously, compare its responses and latency with the old one, and discard its answers; never mirror side-effecting requests unless the shadow is isolated.

Next: SD-G-014 · Autoscaling

Autoscaling

add capacity when demand rises, remove it when it falls

Watch the 4-min video

In one line: autoscaling adjusts capacity to demand; target tracking keeps a metric near a target, the right metric depends on the workload (CPU for CPU-bound, requests or concurrency for I/O-bound APIs, queue depth for workers), scheduled and predictive scaling cover the reaction lag, and limits and stabilisation windows stop flapping and runaway bills.

1 Four servers at night, forty at lunch

Your API needs 4 servers at night and 40 at lunchtime. Running 40 all day wastes money; running 4 at lunch takes you down. **Autoscaling** adds and removes capacity automatically as demand changes. It only works for **stateless** tiers (lesson 34) that can add instances freely, and only if it scales on the right signal, fast enough.

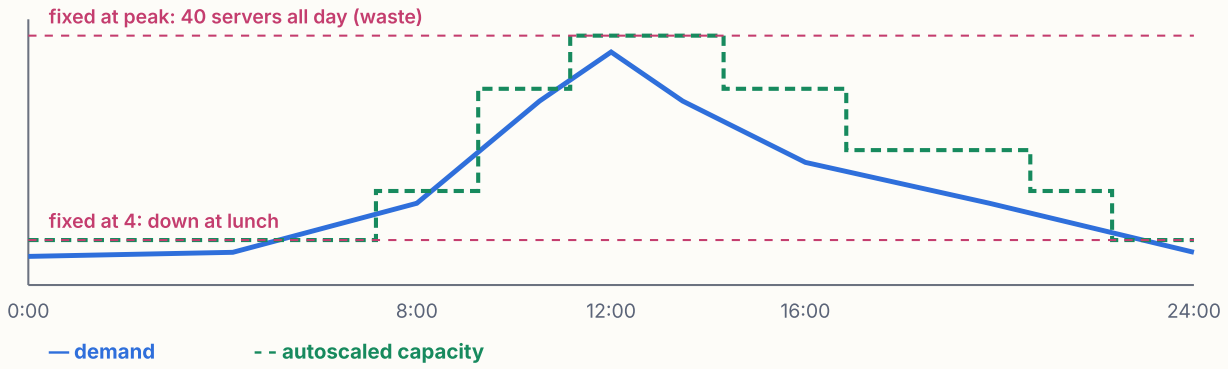


Figure 1. Fixed capacity either wastes money or fails at the peak; autoscaled capacity follows demand in steps.

2 Reactive scaling: target tracking

Target tracking keeps a metric near a target, like average CPU at 60%. If it climbs above, add instances; if it falls below, remove some. The maths is roughly `desired = current × (metric / target)`: 10 instances at 90% CPU with a 60% target → 15 instances. AWS target tracking policies and the Kubernetes **Horizontal Pod Autoscaler** (HPA) both work this way.

3 Which metric?

CPU is a fine default for CPU-bound services. But many APIs are **I/O-bound**: they spend their time waiting on databases and other services. Their CPU stays low even when they're overloaded and latency soars, so a CPU target never triggers. For those, scale on **requests per second per instance**, **concurrent requests**, or latency. For workers, scale on **queue depth** or consumer lag.

CPU-bound service	image resizing, ML inference	CPU utilisation, target ~60%
I/O-bound API	waits on DB and other services	requests/s or concurrency per instance, latency
Queue worker	Celery, Sidekiq, Kafka consumer	queue depth (or consumer lag) per worker

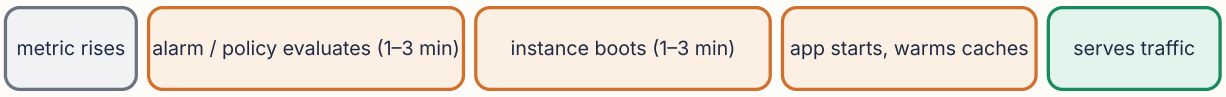
Figure 2. Match the scaling metric to what limits the workload.

4 The lag, and how to beat it

Reactive scaling has lag: the metric must rise, the policy must react, instances must boot and warm up. That can take several minutes, and a sudden spike arrives in seconds.

- **Scheduled scaling** adds capacity before known peaks, like 9:00 every weekday or a planned sale.
- **Predictive scaling** learns daily and weekly patterns and scales ahead of them (AWS predictive scaling).
- **Headroom**: a target of 60% rather than 90% leaves room to absorb a spike while new instances start.

Combine them, and keep reactive scaling as the safety net.



reactive scaling can take minutes: schedule or predict known peaks, and keep headroom for the gap

Figure 3. Why reactive scaling is slow: each step between the spike and new capacity takes time.

5 Stability and limits

- **Cooldowns and stabilisation windows** prevent **flapping** (add, remove, add again). The HPA waits 5 minutes by default before scaling down.
- **Scale up fast, scale down slowly.** Being briefly over-provisioned is cheap; being under-provisioned is an outage.
- **Minimums**, so a quiet night doesn't leave one instance (and no redundancy across zones).
- **Maximums**, so a bug, a retry storm or an attack can't scale your bill to the moon, or overwhelm the database.

WORKED EXAMPLE: CELERY WORKERS WITH KEDA

Celery workers process jobs from a Redis queue. **KEDA** (Kubernetes Event-Driven Autoscaling) watches the queue length, with a target of 20 jobs per worker. 1,000 jobs arrive, so KEDA scales to 1,000 / 20 = **50 workers** (capped by the maximum). When the queue drains, it scales back down, and to **zero** when it's empty, so idle workers cost nothing. The next job waits for a cold start, which is fine for background work.

6 Remember the whole system

- Scaling the app tier sends more load to the **database**, which may not scale at all; 50 workers × 10 connections = 500 connections.
- **Scale to zero** means a cold start for the first request: fine for jobs, often not for user-facing APIs.
- New instances start with cold caches and empty connection pools: add a startup or readiness probe (lesson 7) and warm-up.
- Autoscaling can't fix a bottleneck elsewhere; it can only add more of what you already have.

COMMON MISTAKES

- Scaling an I/O-bound API on CPU, so it never scales while latency explodes.
- No maximum, so a retry storm scales to hundreds of instances and takes the database down.
- Minimum of 1, so there's no redundancy and every scale-up starts from a cold single instance.
- Scaling down too aggressively, then thrashing at every small spike.

7 Key terms

Target tracking: keep a metric near a target value

Scheduled scaling: add capacity at known times

Stabilisation window: wait before acting, to avoid flapping

Scale to zero: no instances while idle; cold start later

HPA: Kubernetes Horizontal Pod Autoscaler

Predictive scaling: forecast demand and scale ahead

KEDA: scales Kubernetes workloads on events like queue length

Headroom: spare capacity for spikes while scaling

8 Interview questions

Q1. Which metric would you autoscale an API on? BASIC

Usually requests per second or concurrency per instance, or latency, rather than CPU alone.

Q2. Why is CPU not always right? BASIC

Because I/O-bound services wait on databases and downstream calls, so they can be overloaded with low CPU; and for queue workers, queue depth is the better signal.

Q3. Traffic jumps 5× at 9:00 every weekday and autoscaling is always late. What do you do? SCENARIO

Add scheduled (or predictive) scaling before 9:00, keep a lower target for more headroom, speed up startup (smaller images, pre-warmed pools), and shed low-priority load during the gap.

Q4. How do you autoscale background workers? SCENARIO

On queue depth or consumer lag, for example with KEDA: desired workers = queue length / target jobs per worker, with min, max and scale-to-zero if cold starts are acceptable.

Q5. After autoscaling to 60 pods, the database falls over. Why? TRICKY

Each pod opens its own connection pool, so connections multiply, and every pod adds queries. Cap the max replicas, use a connection pooler such as PgBouncer, and scale the data tier or cache first.

Q6. Why scale up fast but down slowly? TRICKY

Under-provisioning causes errors and latency for users right away, while over-provisioning for a few minutes only costs a little money; slow scale-down also avoids flapping when traffic is bursty.

Next: SD-F-001 · Why Cache and Where

Why Cache and Where

keep hot data close, and know what not to cache

Watch the 4-min video

In one line: a cache is a fast copy of data that is expensive to fetch or compute; it works because a small hot working set gets most requests, the hit ratio decides how much load it removes, caching can happen at every layer from browser to database, and some data (strictly fresh, rarely reused, one-off personal, sensitive) shouldn't be cached.

1 The same query, thousands of times a second

Every product page on your shop runs the same database query, thousands of times a second, for data that changes once a day. The database sits at 90% CPU and pages take half a second. What if most of those requests never reached the database at all? That's what a **cache** does. Welcome to the caching section of Phase 3.

2 What a cache is, and why it works

A cache is a **fast copy** of data that's expensive to fetch or compute. Reading from memory takes well under a microsecond; a database query takes milliseconds; a call to another service even more (Phase 2, estimation lesson 2).

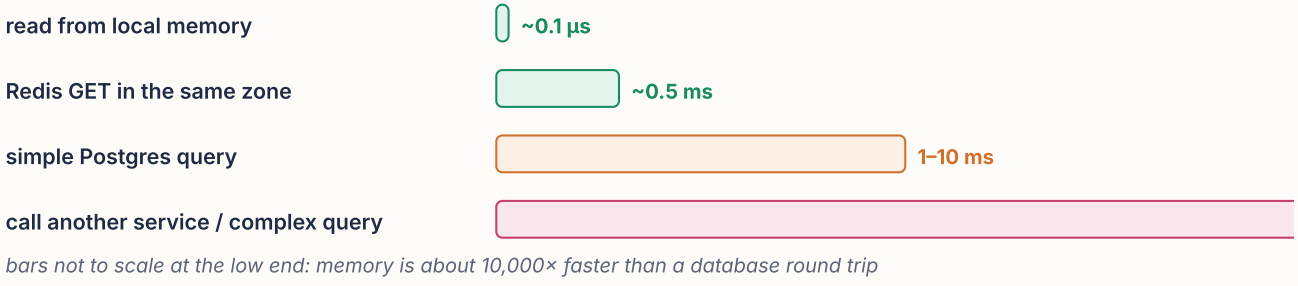


Figure 1. Where the time goes: memory and Redis are orders of magnitude faster than a database or service call.

Caching works because of **locality**: a small set of hot data, the **working set**, gets most of the requests. The top 1% of products might get half the views. Keep that set close, and most reads become fast.

3 The number that matters: hit ratio

The **hit ratio** is the share of requests served from the cache. It matters more than it seems: at 90%, one in ten requests reaches the database; at 99%, only one in a hundred does, **ten times less load**. Every miss also costs extra time: the cache check, then the database trip.

```
average latency ≈ hit ratio × cache time + (1 − hit ratio) × (cache time + database time)
```

With a 1 ms cache and a 20 ms database: 90% → about 3 ms on average; 99% → about 1.2 ms.

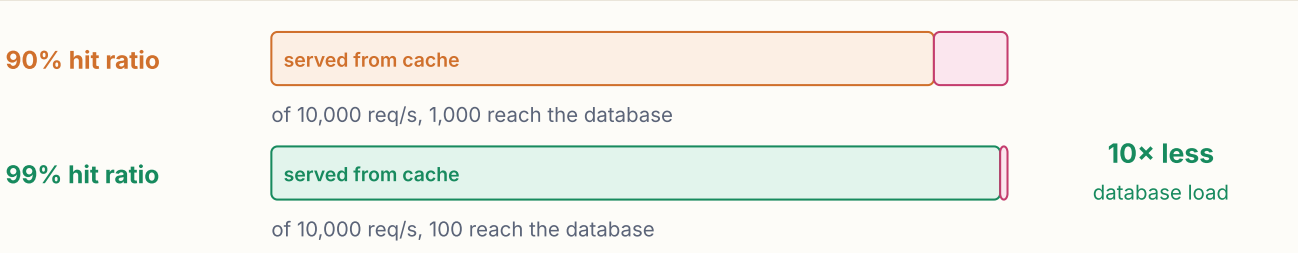


Figure 2. Going from a 90% to a 99% hit ratio cuts database traffic by ten times.

4 Where caching can happen

Along one request there are many places to cache:

- **Browser or mobile app**, controlled by HTTP headers such as `Cache-Control` (lesson 31).
- **CDN** edges cache static files and some API responses near users (Phase 2, lesson 28).
- **API gateway or reverse proxy** can cache whole responses.
- **Application**: objects in process memory, or in a shared cache such as **Redis** or Memcached.
- **Database**: its own buffer pool keeps hot pages in memory.

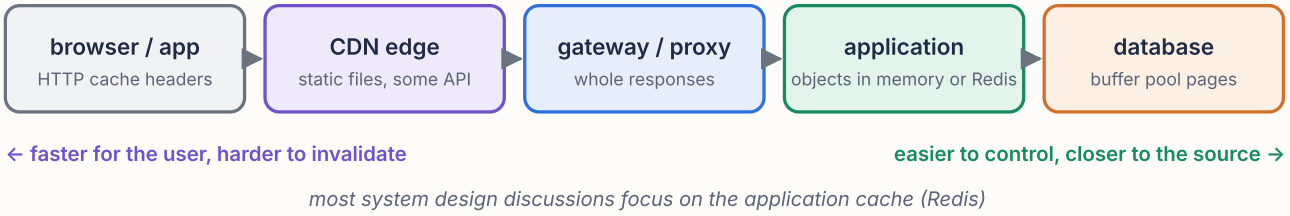


Figure 3. Caching layers along a request, from the user's device to the database.

Each layer trades **freshness for speed**. The closer to the user, the faster, but the harder to invalidate: you can't clear a browser's cache from your side, so keep browser lifetimes short for anything that changes. Application caches in Redis are easiest to control, which is why most system design discussions focus there.

WORKED EXAMPLE: PRODUCT DETAILS IN REDIS

Product details are read constantly but change rarely. The app stores each product in Redis under a key like `product:42` with a 10-minute TTL. With a 95% hit ratio, Postgres handles one twentieth of the reads: CPU drops from 90% to about 20%, and pages respond in a few milliseconds instead of half a second.

5 What not to cache

- **Data that must be exactly current**: account balances, stock levels at checkout. A stale value causes real harm.
- **Data that's rarely reused**: the hit ratio would be low, and entries just take memory.
- **Highly personalised one-off results**: the cache fills with entries read once.
- **Sensitive data**, unless the cache is secured and keys are scoped to the right user, so one user never sees another's data.

WHAT A CACHE COSTS YOU

- **Staleness**: the copy can be out of date (lessons 22 and 23).
- **Another system** to run, monitor and size.
- **New failure modes**: a cold cache after a restart, a stampede on one hot key (lessons 24 to 26).

6 Key terms

Cache: fast copy of expensive data

Hit ratio: share of requests served from the cache

Locality: recent or popular data is likely to be used again

Buffer pool: the database's own in-memory page cache

Hit / miss: found in the cache / not found

Working set: the hot data that gets most requests

TTL: how long an entry stays valid

Staleness: how out of date a cached copy is

7 Interview questions

Q1. Where can caching happen in a request path? BASIC
In the browser or app, the CDN, the API gateway or reverse proxy, the application with in-memory or Redis caches, and the database's own buffer pool.

Q2. What data should not be cached? BASIC
Data that must be strictly fresh, data with low reuse, highly personalised one-off results, and sensitive data without proper isolation.

Q3. Why does a cache help so much? BASIC
Most requests hit a small working set, and memory is thousands of times faster than a database round trip, so serving the hot set from memory removes most of the load and latency.

Q4. Your hit ratio is 80%. How much would reaching 98% reduce database load? SCENARIO
Misses fall from 20% to 2% of requests: ten times fewer database reads.

Q5. Should you cache a user's account balance? SCENARIO
Not for decisions like payments or transfers, which must read the source of truth. For a display-only view you might cache it for seconds, invalidated on every transaction.

Q6. Why can a cache make an outage worse? TRICKY
The database is sized for the cache's hit ratio. If the cache fails or restarts cold, all traffic hits the database at once and can overwhelm it; plan for that with warm-up, rate limiting and request coalescing.

Next: SD-F-002 · Cache-Aside (Lazy Loading)

Cache-Aside (Lazy Loading)

the app checks the cache, and fills it on a miss

Watch the 5-min video

In one line: in cache-aside the application reads the cache first, and on a miss reads the database and stores the result with a TTL; on a write it updates the database and then deletes the key, because deleting is safer than updating the cache, while TTLs bound the small stale window that remains.

1 Who fills the cache?

You've added Redis next to your database. Who puts data into the cache, when, and what happens when the data changes? The most common answer, used in most systems you'll ever build, is **cache-aside**, also called **lazy loading**. The application talks to both the cache and the database; the cache is just storage on the side.

2 The read path

1. Ask the cache for the key.
2. **Hit**: return the cached value.
3. **Miss**: read from the database, store the result in the cache with a TTL, and return it.

Only data that's actually requested gets cached, which is why it's called *lazy*. The cache fills itself with the working set.

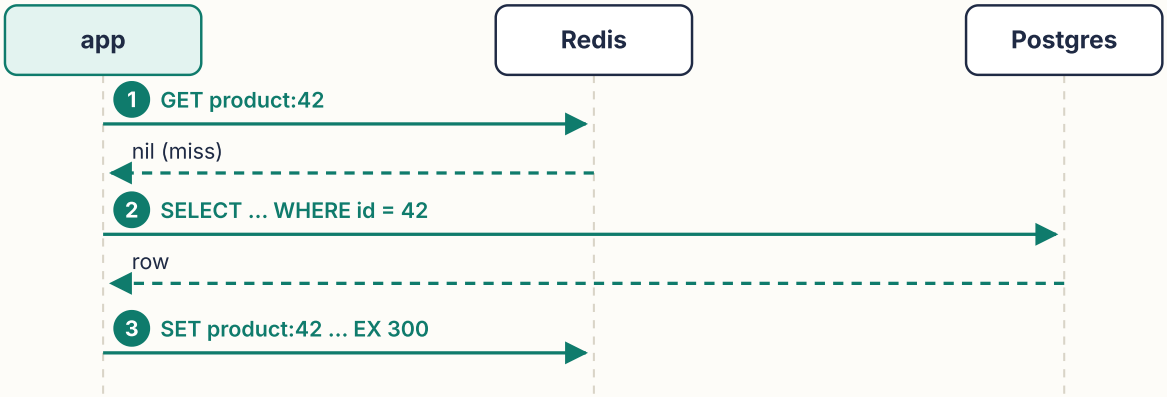


Figure 1. A cache miss in cache-aside: the app gets nothing from Redis, reads Postgres, then stores the row with a 5-minute TTL.

3 The write path: update the database, delete the key

On a write, the app updates the **database first**, then **deletes** the cache key. The next read misses, loads the fresh value, and repopulates the cache.

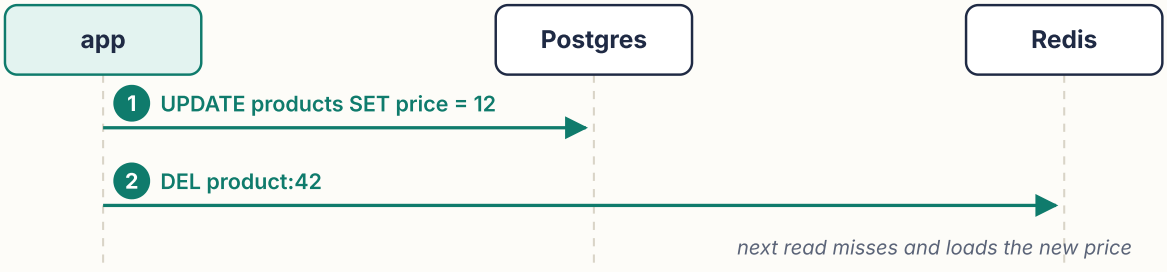


Figure 2. Writes go to the database, then the cache entry is deleted rather than updated.

WORKED EXAMPLE: A FASTAPI ENDPOINT

`GET /products/{id}` calls `redis.get(f"product:{id}")`. If there's a value, it parses the JSON and returns it. If not, it queries Postgres, calls `redis.set(key, json, ex=300)` (5 minutes) and returns the product. `PUT /products/{id}` writes to Postgres, then calls `redis.delete(key)`.

Ten lines of code, any cache product, and the database stays the source of truth.

4 Why delete instead of update?

Two writers can race. Writer A updates the database to price 10, writer B to price 12, but their cache updates arrive in the opposite order: the cache keeps 10, while the database says 12, until the TTL expires. Deleting avoids that: whatever the order, the next read loads the current value. Updating the cache also wastes work if nobody reads the key before it changes again.

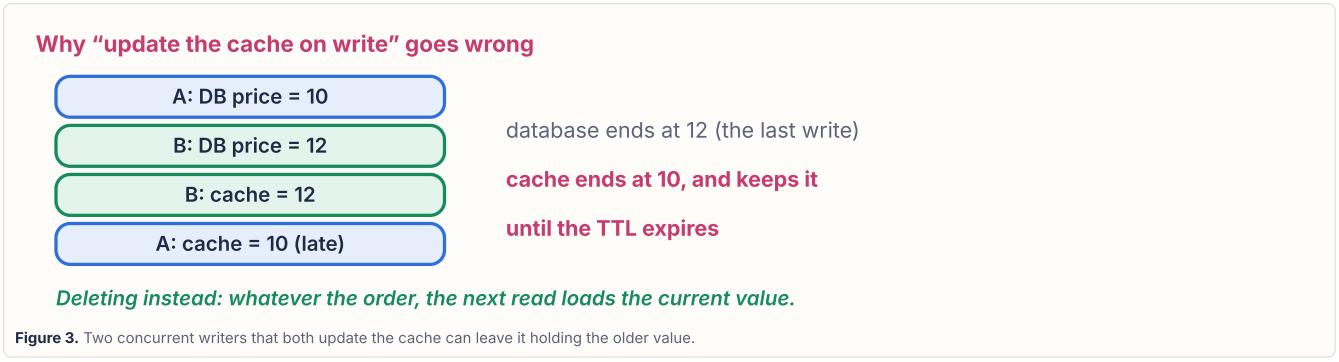


Figure 3. Two concurrent writers that both update the cache can leave it holding the older value.

5 The stale window that remains

Between the database write and the cache delete, readers see the old value: usually a few milliseconds. A rarer race remains too: a slow reader loads an *old* value from the database, the writer commits and deletes the key, and then the slow reader writes its old value into the cache. **TTLs bound how long any stale entry can live.** Some teams also **delete twice**: once immediately, and once again after a short delay (say 500 ms) to catch that race. Lesson 23 covers stronger options.

6 Failure modes

- **Cache down**: every request goes to the database. The database must survive it, or you need rate limiting and a circuit breaker around the cache.
- **Cold cache** after a restart or deploy: a burst of misses. Warm the hottest keys first.
- **Stampede**: many requests miss the same hot key at once and all query the database. A lock or request coalescing fixes it (lesson 24).

COMMON MISTAKES

- Deleting the cache key *before* the database write: a reader can refill the old value in between.
- No TTL, so a missed invalidation leaves wrong data forever.
- Caching "not found" without a short TTL, or not caching it at all and letting repeated misses hammer the database (lesson 25).

7 Key terms

Cache-aside: the app reads the cache and fills it on a miss

Source of truth: the database; the cache is a copy

Stale window: time the cache can show an old value

Lazy loading: cache only what's requested

Invalidation: deleting a cache entry after a change

Delayed double delete: delete again shortly after the write

8 Interview questions

Q1. Walk through cache-aside reads and writes. BASIC
On read: check the cache and return on a hit; on a miss, read the database, store the result with a TTL, and return it. On write: update the database, then delete the cache key.

Q2. Why delete rather than update on write? BASIC
Concurrent updates can leave the cache with an older value, while a delete always forces the next read to load the current data, and avoids caching data nobody reads.

Q3. What are the advantages of cache-aside? BASIC
Simple, works with any cache, caches only what's used, and the database stays the source of truth, so a cache failure only slows things down.

Q4. Redis goes down during peak. What happens with cache-aside, and how do you prepare? SCENARIO
Every read falls through to the database, which may be overloaded. Size the database for some miss traffic, put timeouts and a circuit breaker on cache calls, rate-limit or shed load, and warm the cache when it returns.

Q5. Describe a race that leaves stale data even with delete-on-write. TRICKY
A reader misses, reads the old row, pauses; a writer updates the row and deletes the key; the reader then writes the old row into the cache. TTLs bound it; a delayed second delete or versioned values reduce it.

Q6. Should you delete the key before or after the database write? TRICKY
After. Deleting first opens a gap where a reader misses, reads the old value from the database, and caches it before the write commits.

Next: SD-F-003 · Read-Through and Write-Through backendroadmap.com

Read-Through and Write-Through

let the cache load and store data itself

Watch the 4-min video

In one line: in read-through the cache itself loads missing data from the database through a configured loader, so the app only talks to the cache; in write-through every write goes through the cache, which writes the database synchronously before confirming, keeping them in sync at the cost of slower writes and caching data nobody reads.

1 The same code in every service

With cache-aside (lesson 16), every service writes the same code: check the cache, query the database, fill the cache. What if the cache did that work itself? You'd talk only to the cache, and it would keep itself filled and in sync. That's **read-through** and **write-through** caching.

2 Read-through

The application reads **only from the cache**. On a miss, the cache itself loads the data from the database, using a **loader** function you configure, stores it and returns it. The miss logic lives in one place, the cache provider, instead of being repeated in every service.

The flow on a miss is the same as cache-aside; what changes is **who runs it**. In cache-aside the application owns the logic and can use any cache. In read-through the cache owns it, so app code is simpler, but you need a cache that supports loaders: Caffeine or Guava `LoadingCache` in Java, Ehcache, or a managed product such as Amazon DAX. A nice bonus: a good loader coalesces concurrent misses for the same key into one database call.

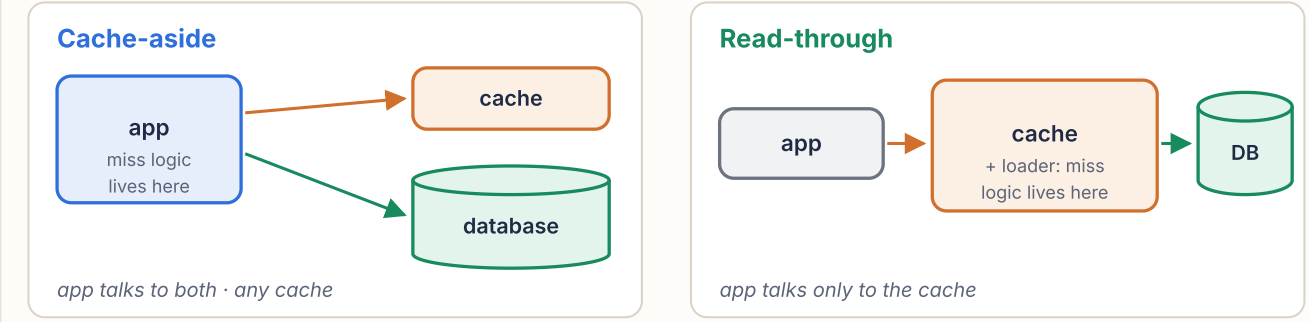


Figure 1. Same miss flow, different owner: in cache-aside the app runs it, in read-through the cache's loader does.

3 Write-through

Every write goes **to the cache**, and the cache **synchronously** writes it to the database before confirming. The write isn't done until both are updated. So the cache is always in sync with the database, and a read right after a write sees the new value, a cache hit.

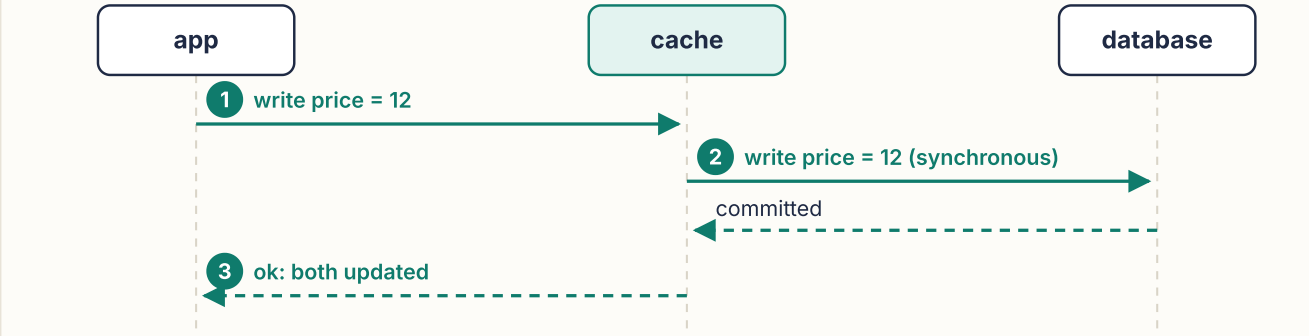


Figure 2. Write-through: the cache forwards the write to the database and confirms only after both are updated.

THE COST OF WRITE-THROUGH

- **Higher write latency**: every write pays for two writes, cache and database, in sequence.
- **Cache pollution**: data that's written but never read still fills the cache. Pair it with TTLs (and eviction) so unused entries leave.
- **Partial failure**: if the database write fails after the cache write, the provider must roll back or invalidate the cache entry, or the two disagree.

Write-through is usually paired with read-through: writes keep the cache fresh, and reads fill in whatever is missing.

WORKED EXAMPLE: DAX IN FRONT OF DYNAMODB

DAX, the DynamoDB Accelerator, is an in-memory cache that speaks the DynamoDB API. The application swaps the DynamoDB client for the DAX client and changes nothing else. **Reads are read-through**: a hit returns in microseconds, a miss is fetched from DynamoDB and cached. **Writes are write-through**: DAX writes to DynamoDB, then updates its item cache. There's no cache code in the application at all. (One subtlety: strongly consistent reads bypass DAX and go straight to DynamoDB.)

4 Choose by workload



most teams default to cache-aside; providers like DAX give read-through + write-through for free

Figure 3. Four patterns and what each one suits.

- **Cache-aside**: maximum control, works with any cache. The common default.
- **Read-through**: simpler code for read-heavy data, when your cache supports loaders.
- **Write-through**: data that's read soon after it's written and must be consistent in the cache, such as a user's profile after they edit it.
- For write-heavy data that's rarely read, write-through wastes effort; **write-around** is better (next lesson).

COMMON MISTAKES

- Calling write-through "consistent" while other services write to the database directly and bypass the cache.
- Write-through with no TTL, so the cache fills with write-only data.
- Ignoring the failure case where the cache write succeeds and the database write fails.

5 Key terms

Read-through: the cache loads misses itself

Write-through: writes go through the cache to the DB, synchronously

Cache pollution: entries that are never read taking space

Loader: the function the cache calls on a miss

DAX: DynamoDB Accelerator, a read/write-through cache

Request coalescing: one load for many concurrent misses

6 Interview questions

Q1. Compare cache-aside with read-through. BASIC
In cache-aside the application checks the cache, loads from the database on a miss and fills the cache itself; in read-through the cache provider does that loading, so application code only talks to the cache.

Q2. What is the cost of write-through? BASIC
Higher write latency, because each write synchronously updates both cache and database, plus caching data that may never be read.

Q3. What's the benefit of write-through? BASIC
The cache always holds the latest value for written keys, so reads right after writes are hits and never stale, as long as all writes go through the cache.

Q4. Users edit their profile and immediately view it. Which pattern? SCENARIO
Write-through (with read-through for misses): the edit updates the cache and database together, so the next view is a fresh hit. With cache-aside, delete-on-write followed by a miss also works but costs a database read.

Q5. You use DynamoDB and want microsecond reads with no cache code. What do you use? SCENARIO
DAX: swap in the DAX client for read-through and write-through caching, and keep strongly consistent reads (which bypass DAX) for the few cases that need them.

Q6. Can write-through still serve stale data? TRICKY
Yes, if anything writes to the database without going through the cache (a batch job, another service, a manual fix), or if the cache entry was evicted and refilled from a lagging replica. TTLs limit the damage.

Next: SD-F-004 · Write-Behind and Write-Around backendroadmap.com

Write-Behind and Write-Around

absorb huge write rates, or keep writes out of the cache

Watch the 4-min video

In one line: write-behind (write-back) acknowledges writes in the cache and flushes them to the database later in batches, turning floods of tiny writes into a few big ones at the risk of losing unflushed data; write-around sends writes straight to the database and skips the cache, so write-heavy, rarely read data doesn't push hot data out.

1 Fifty thousand tiny writes a second

Every video view increments a counter in your database. A viral video gets 50,000 views a second, and the database melts under 50,000 tiny `UPDATE`s on one row (each one locks that row, so they queue). Does every single view need its own database write right now? Not really.

2 Write-behind (write-back)

The application writes to the **cache** and gets an immediate acknowledgement. The cache, or a background job, **flushes** changes to the database later, often in batches. Writes become very fast, and many updates to the same key are **combined** into one database write: a counter incremented 500,000 times becomes one `UPDATE … SET views = views + 500000`.

WORKED EXAMPLE: VIDEO VIEW COUNTERS

Each view runs `INCR views:{video_id}` in Redis, which takes microseconds. Every 10 seconds a worker reads the counters (atomically, with `GETDEL` or a rename), writes the totals to the database in one batch, and starts from zero. 50,000 writes per second become a few thousand batched updates every 10 seconds, one per active video.

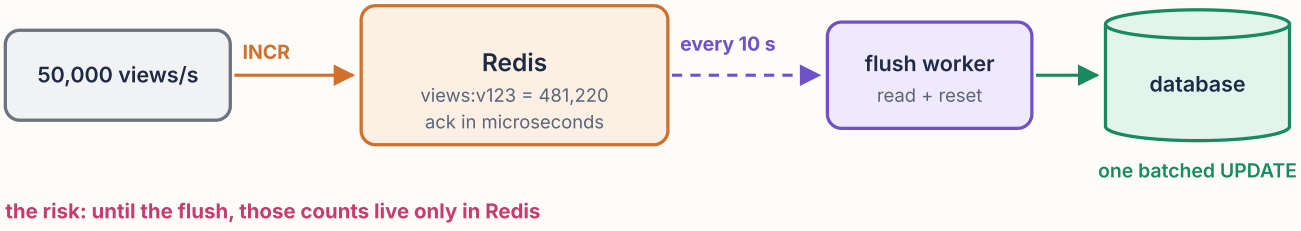


Figure 1. Write-behind: Redis absorbs the increments; a worker flushes the totals to the database every 10 seconds.

3 The risk: durability

Until the flush, the data lives **only in the cache**. If Redis crashes, the last 10 seconds of counts are gone. Redis persistence (AOF with `fsync` every second) and replication shrink that window, but don't close it. And anyone reading the database sees slightly stale values until the next flush.

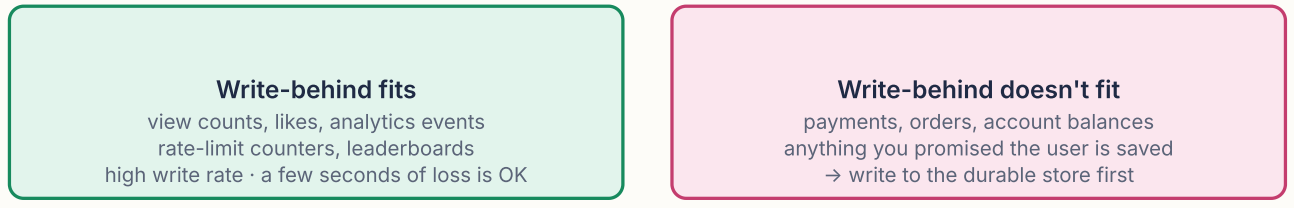


Figure 2. Use write-behind only where losing a few seconds of updates is acceptable.

4 Write-around

Write-around is the opposite. Writes go **straight to the database**, skipping the cache entirely, and any existing cache entry for that key is invalidated. The cache only gets the data later, *if* someone reads it, through cache-aside or read-through.

It suits data that's written often but read rarely, or not soon: logs, audit records, bulk imports, archived content. With write-through, those writes would flood the cache and **evict hot data**. Write-around keeps the cache full of what's actually read. The trade-off: the first read after a write is always a miss.

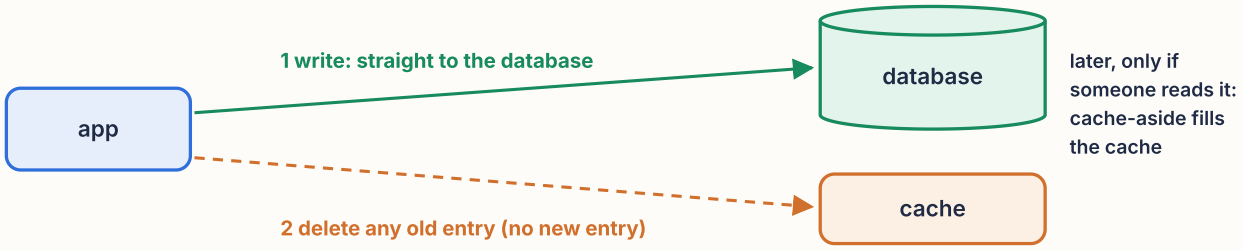


Figure 3. Write-around: the write goes only to the database; the cache gets the item later, and only if it's read.

5 The four write patterns side by side

Pattern	Write goes to	Write latency	Risk	Good for
Cache-aside (delete)	DB, then delete key	DB only	small stale window	most read-heavy data
Write-through	cache → DB, synchronous	higher (both)	caching unread data	read-soon-after-write
Write-behind	cache, DB later	lowest	losing unflushed writes	counters, analytics
Write-around	DB only	DB only	first read is a miss	logs, imports, audit

COMMON MISTAKES

- Write-behind for orders or payments, then losing them in a cache failover.
- A flush job that reads then resets in two steps, losing increments that arrive in between: use an atomic swap.
- No monitoring of flush lag, so a stuck worker silently stops data reaching the database.

6 Key terms

Write-behind / write-back: ack in cache, flush to DB later

Coalescing writes: many updates to one key become one write

Durability window: how much acknowledged data could be lost

Flush: writing buffered changes to the database

Write-around: writes skip the cache

Cache pollution: rarely read data evicting hot data

7 Interview questions

Q1. When is write-behind acceptable? BASIC

When the write volume is high and losing a few seconds of updates is acceptable, like view counters, likes and analytics, since data sits in the cache until the asynchronous flush.

Q2. What is write-around good for? BASIC

Write-heavy, read-rarely data like logs, audit records and bulk imports, because writes go only to the database and the cache isn't polluted with entries nobody reads.

Q3. Design a like counter for posts with millions of likes per minute. SCENARIO

Increment per-post counters in Redis (sharded if one key is too hot), flush totals to the database every few seconds in batches, serve counts from Redis, and accept that a crash can lose a few seconds of likes.

Q4. A nightly job imports 10 million records. How do you keep it from wrecking the cache? SCENARIO

Write-around: insert straight into the database without touching the cache (only invalidate keys that exist), so the import doesn't evict the hot working set.

Q5. How do you make write-behind safer? TRICKY

Persist the cache (AOF every second) and replicate it, flush frequently, use atomic read-and-reset, monitor flush lag, or put the writes on a durable log like Kafka and aggregate from there.

Q6. Why do 50,000 single-row updates per second hurt the database even if each is fast? TRICKY

They all lock the same row, so they serialise; each also writes the WAL and indexes. Batching collapses them into one write per interval.

Next: SD-F-005 · TTL and Expiration

TTL and Expiration

how long data may live, and why to add randomness

Watch the 4-min video

In one line: a TTL is how long a cache entry stays valid (absolute: fixed from the write; sliding: reset on every read); choose the longest TTL your staleness budget allows, invalidate explicitly for data that must change fast, and add jitter so thousands of keys don't expire at the same moment and flood the database.

1 Too long, or too short

You cache product prices for an hour. A sale starts, prices drop, and customers keep seeing the old price for 50 minutes. So you set the TTL to one second, and now the cache barely helps. How long should data live in a cache, and why do smart teams add randomness to that number?

2 Absolute and sliding TTLs

A **TTL**, time to live, is how long a cache entry stays valid. With an **absolute** TTL, the entry expires a fixed time after it was written, no matter how often it's read. With a **sliding** TTL, every read resets the clock, so popular entries stay alive while idle ones expire.

Sliding TTLs suit **sessions** (log out after 30 idle minutes). Absolute TTLs suit data that changes on its own schedule, like prices, because a popular entry with a sliding TTL could live forever, stale.

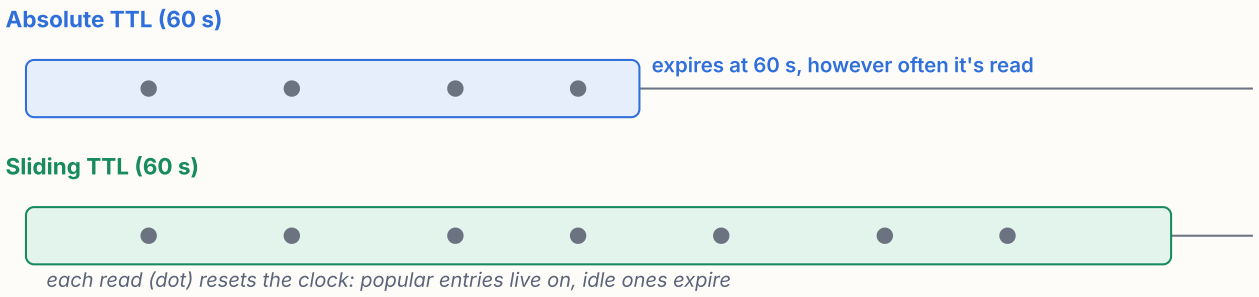


Figure 1. An absolute TTL expires on schedule; a sliding TTL is extended by every read.

3 Choosing the number: the staleness budget

Start from a **staleness budget**: how old can this data be before it hurts? An exchange rate might tolerate a minute; a product description, an hour; a user's permissions, only seconds. Then weigh the hit ratio: longer TTLs mean more hits and less database load. **Pick the longest TTL the staleness budget allows**, and combine it with **invalidation on writes** (lesson 22) for data that must update quickly. Then the TTL is only a safety net for missed invalidations.

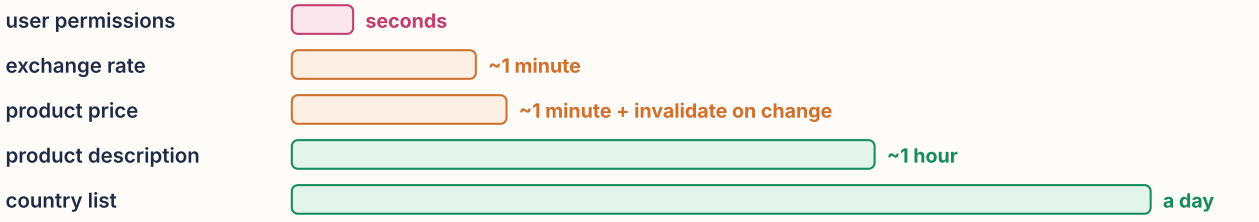


Figure 2. Staleness budgets differ by data: the TTL follows from how much staleness the business can accept.

4 Synchronised expiry, and jitter

Suppose you warm 10,000 keys at startup, all with a 60-second TTL. Sixty seconds later they all expire at the same moment, every request misses at once, and the database gets a sudden wall of queries. Then the keys are refilled together, and it happens again 60 seconds later: a heartbeat of pain.

The fix is **jitter**: add a small random amount to each TTL. Instead of exactly 60 seconds, use `60 ± 10`. Expirations spread across a 20-second window, and the database sees a smooth trickle of misses instead of a spike.

```
ttl = base + random.uniform(−spread, +spread)  e.g. 60 + uniform(−10, 10)
```

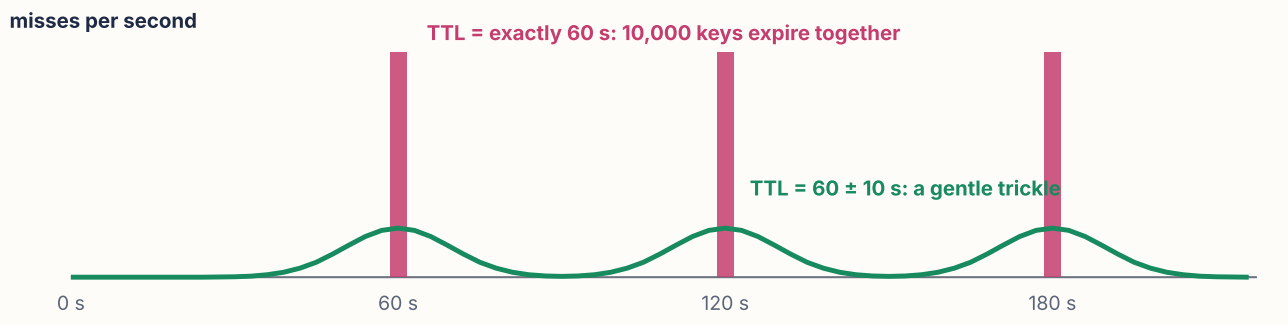


Figure 3. Without jitter, keys written together expire together and produce miss spikes; with jitter the misses spread out.

WORKED EXAMPLE: PRODUCT PRICES

Prices are cached for 60 seconds ± 10 seconds of random jitter. They're never more than about 70 seconds stale, which the business accepts for browsing. A price change for a sale also **deletes the key immediately**, so sale prices appear within milliseconds; the TTL only matters if an invalidation is lost. Because expirations are spread out, database load stays flat. (Checkout still reads the price from the database.)

5 How Redis expires keys

- **Lazy expiry**: when you access a key, Redis checks its TTL and deletes it if it has expired.
- **Active expiry**: a background job runs about 10 times a second, samples random keys that have TTLs (20 per round), and deletes the expired ones; if many of the sample were expired, it repeats straight away.

Together they keep memory tidy without scanning every key. Set TTLs with `EXPIRE key 60` or `SET key value EX 60`, and inspect them with `TTL key` (−1 means no TTL, −2 means the key doesn't exist).

COMMON MISTAKES

- No TTL at all: one missed invalidation leaves wrong data forever.
- The same fixed TTL for keys written in bulk, causing synchronised expiry.
- Sliding TTLs on data that changes, so hot entries never refresh.
- Overwriting a key with `SET` without `EX`, which silently removes its TTL.

6 Key terms

TTL: time to live: how long an entry is valid

Sliding TTL: every read extends the lifetime

Synchronised expiry: many keys expiring at the same moment

Lazy expiry: delete expired keys when accessed

Absolute TTL: expires a fixed time after the write

Staleness budget: how old data may be before it hurts

Jitter: random spread added to TTLs

Active expiry: background sampling of expired keys

7 Interview questions

Q1. How do you choose a TTL? BASIC

Start from the staleness budget, how old the data can be before it causes harm, and pick the longest TTL within it to maximise hits, with explicit invalidation for data that must change quickly.

Q2. Why add jitter to TTLs? BASIC

To spread expirations randomly, so many keys don't expire at the same moment and flood the database with misses.

Q3. Absolute or sliding TTL for a login session? BASIC

Sliding, usually with an absolute maximum: active users stay logged in, idle sessions expire, and every session ends eventually (say 30 minutes idle, 12 hours maximum).

Q4. Every 5 minutes, database CPU spikes for a few seconds. The cache uses a 300 s TTL. What's happening? SCENARIO

Keys were written together (at startup or by a batch) and expire together, so misses arrive in a burst every TTL. Add jitter, and coalesce concurrent misses for the same key.

Q5. How does Redis find expired keys without scanning everything? TRICKY

Lazily on access, plus an active cycle about 10 times a second that samples keys with TTLs and deletes expired ones, repeating while many samples are expired.

Q6. If you invalidate on every write, why still set a TTL? TRICKY

As a safety net: invalidations can be lost (a crash between the database write and the delete, a bug, a direct database fix), and the TTL bounds how long wrong data can live.

Next: SD-F-006 · LRU Cache

LRU Cache

evict the entry that was used least recently

Watch the 4-min video

In one line: LRU keeps recently used entries and evicts the one untouched for longest; it's built with a hash map from key to node plus a doubly linked list ordered by recency, so get and put are O(1); Python's OrderedDict and functools.lru_cache give it for free, and Redis uses a sampled approximation.

1 Something has to go

Your cache has room for one million entries, and the millionth-and-first arrives. Something has to go. Which one? The most common answer, in interviews and in production, is to evict the entry that was used **least recently**: **LRU**. And yes, you may be asked to build one on a whiteboard.

2 The idea

Entries used recently are likely to be used again soon (temporal locality). So keep the recent ones, and when the cache is full, evict the one that hasn't been touched for the longest time. Every read *or* write of a key makes it the most recent.

3 O(1) get and put: hash map + doubly linked list

The interview challenge is doing both operations in constant time. A **hash map** gives O(1) lookup by key but no order. A **list** gives order, but finding a key is O(n). Combine them: a hash map from key to a **node** in a **doubly linked list**, ordered from most recent at the **head** to least recent at the **tail**. Doubly linked, because unlinking a node needs its previous neighbour, and we want that in O(1) too.

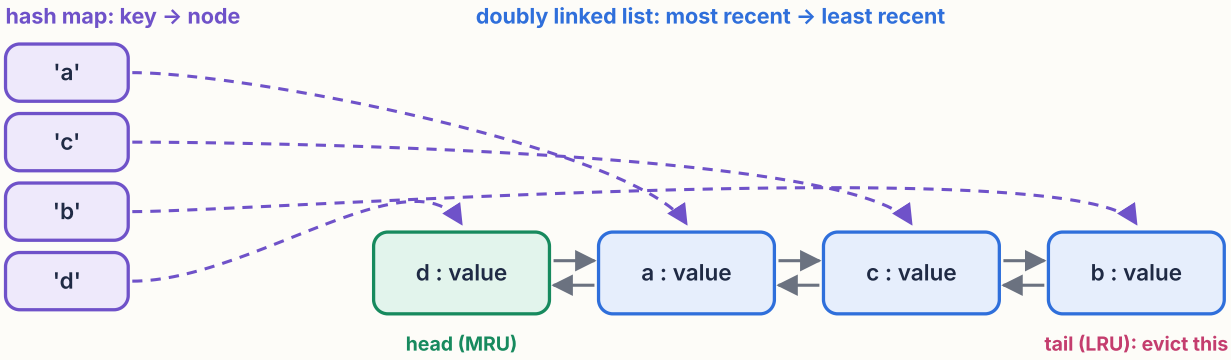


Figure 1. The map finds a node in O(1); the list keeps recency order, so the tail is always the eviction candidate.

get(key)
node = map.get(key); missing → miss
unlink node, move it to the head: O(1)
return node.value

put(key, value)
exists → update value, move to head
new → node at head, add to map
over capacity → remove tail + its map key

Figure 2. Both operations touch a constant number of pointers. Dummy head and tail nodes avoid special cases for empty lists.

4 In real code

In Python you rarely write the linked list yourself. `OrderedDict` keeps order and has `move_to_end` and `popitem(last=False)`, which is all you need. And `@functools.lru_cache(maxsize=1024)` wraps a function with a ready-made LRU cache. Java's `LinkedHashMap` with access order and `removeEldestEntry` does the same.

```
from collections import OrderedDict
class LRU:
    def __init__(self, cap): self.cap, self.d = cap, OrderedDict()
    def get(self, k):
        if k not in self.d: return None
        self.d.move_to_end(k); return self.d[k]
    def put(self, k, v):
        self.d[k] = v; self.d.move_to_end(k)
        if len(self.d) > self.cap: self.d.popitem(last=False)
```

Figure 3. A complete LRU cache with OrderedDict. In an interview, be ready to also write the hash map + linked list version.

WORKED EXAMPLE: AN API GATEWAY RESPONSE CACHE

An API gateway caches responses for frequently requested endpoints, like the product catalogue for a category. It keeps the **10,000 most recently requested** responses in an in-memory LRU cache. Popular categories stay cached because they're requested constantly; rarely viewed ones get evicted as new requests arrive. Memory stays bounded at 10,000 entries no matter how many categories exist.

5 LRU in Redis, and LRU vs LFU

Redis doesn't keep a perfect LRU list: that would cost memory and work on every key. With `maxmemory-policy allkeys-lru`, it **samples** a few keys (`maxmemory-samples`, 5 by default) and evicts the least recently used among them. This **approximated LRU** is close enough in practice, and raising the sample size to 10 makes it closer.

When would you choose LRU over LFU? When **recent access predicts future access**, which is true for most workloads: sessions, trending content, recently edited documents. LFU wins when popularity is stable over long periods (next lesson). A known weakness of LRU: one big **scan** (a report reading every product once) pushes the whole hot set out.

COMMON MISTAKES

- Forgetting to move a key to the head on *get*, not just on put.
- Removing the tail node but not its key from the map (a memory leak and wrong lookups).
- Using `lru_cache` on methods with `self`, which keeps every instance alive.
- No `maxmemory` policy in Redis, so writes fail with OOM errors instead of evicting.

6 Key terms

LRU: evict the least recently used entry

Doubly linked list: each node links to previous and next

lru_cache: Python decorator memoising a function with LRU

Scan pollution: a one-off full scan evicting the hot set

MRU end / head: where the most recent entry sits

OrderedDict: Python dict that keeps and reorders insertion order

Approximated LRU: Redis samples a few keys and evicts the oldest

7 Interview questions

Q1. Implement an LRU cache. BASIC

Use a hash map from key to node plus a doubly linked list ordered by recency: on get, move the node to the head; on put, insert at the head and evict the tail when over capacity, all in O(1).

Q2. When would you choose LRU over LFU? BASIC

When recent use predicts the next use, like sessions and trending content, while LFU fits long-term stable popularity.

Q3. Why a doubly linked list rather than a singly linked one? BASIC

To unlink a node in O(1) you need its previous node; a doubly linked list stores it, so moving any node to the head is constant time.

Q4. Redis is full and new writes fail. What do you set? SCENARIO

A maxmemory limit with an eviction policy such as allkeys-lru (or volatile-lru to evict only keys with TTLs), and alert on the eviction rate so you know when to add memory.

Q5. A nightly report reads every product once and the hit ratio collapses the next morning. Why? TRICKY

The scan made every product 'recent' and evicted the real hot set. Use a scan-resistant policy (LFU, or segmented LRU / W-TinyLFU as in Caffeine), or bypass the cache for batch reads.

Q6. How does Redis approximate LRU, and why? TRICKY

It stores a small last-access clock per key and, on eviction, samples a few keys (5 by default) and evicts the oldest of them; a true LRU list would need extra pointers and updates on every access.

Next: SD-F-007 · LFU and FIFO Eviction

LFU and FIFO Eviction

when recency alone is fooled

Watch the 4-min video

In one line: LFU evicts the entry with the lowest access count (with decay so old fame fades), which keeps long-term best-sellers and resists scans; FIFO evicts the oldest insertion and is cheap but blind; cache pollution is useless entries pushing out useful ones, and modern caches such as W-TinyLFU combine recency and frequency.

1 Recency gets fooled

A nightly batch job scans a million old products once each. With an LRU cache (lesson 20), every one of them becomes the most recently used entry, and together they push out your truly popular items. By morning the best-sellers are gone from the cache and the hit ratio collapses. Recency alone was fooled. Two other policies help: LFU and FIFO.

2 LFU: least frequently used

Each entry keeps a **counter** of how many times it has been accessed. When the cache is full, evict the entry with the **lowest count**. Items that are popular over a long period, like top products or the home page, keep high counts and stay cached, even while a burst of other keys passes through.

Pure LFU has a problem: **old popularity never fades**. Last month's viral item keeps its huge count forever, and new hot items can't displace it. The fix is **decay**: counters are periodically halved, or decrease over time, so frequency reflects *recent* popularity. Redis's LFU policies (`allkeys-lfu`, `volatile-lfu`) use an 8-bit logarithmic counter per key with a configurable decay time (`lfu-decay-time`, 1 minute by default).

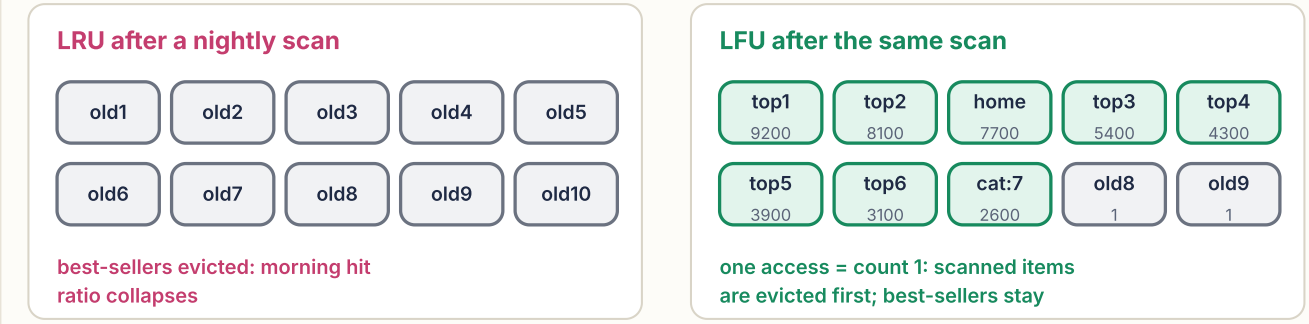


Figure 1. After a scan, LRU holds the scanned items; LFU still holds the best-sellers, because one access gives only a count of 1.

3 FIFO: first in, first out

The simplest policy: evict the entry that was **inserted** earliest, regardless of how it's been used. It needs no bookkeeping on reads, which makes it cheap and easy to scale (a read never has to update shared state). But it may evict a hot item just because it's old. FIFO works when entries have similar value, or when combined with TTLs. Recent research (S3-FIFO) shows a FIFO with a small probation queue can match LRU on real workloads at lower cost.

policy		keeps track of	weakness
LRU	least recently used	recency	fooled by scans
LFU	least frequently used	count (with decay)	slow to adapt to new hits
FIFO	first in, first out	insertion order	evicts hot items for being old
W-TinyLFU	Caffeine	admission by frequency sketch	the best all-rounder

Figure 2. What each eviction policy keeps track of, and where it goes wrong.

4 Cache pollution

Cache pollution is when entries that will never be used again fill the cache and push out useful ones. Scans, crawlers and one-off reports are common causes. LRU is vulnerable, because a single access makes an item look hot. LFU resists pollution, because one access gives a low count.

WORKED EXAMPLE: AN ONLINE STORE'S CATALOGUE

The store's top products are viewed all day, every day. With Redis's `allkeys-lfu` policy, those products build high frequency counts and stay cached, while one-off views from crawlers and long-tail searches get evicted first. The catalogue hit ratio stays around 95% even while a crawler walks through every product page.

5 The modern answer: TinyLFU

TinyLFU uses a compact frequency sketch (a count-min sketch: a few small counter arrays indexed by hashes, a few bytes per entry) to decide whether a new item is worth **admitting** at all: it only lets the item in if it's more popular than the item it would evict. **Window TinyLFU**, used by Java's Caffeine library, adds a small LRU window (about 1% of the cache) so brand-new items get a chance to build up frequency. It beats pure LRU and pure LFU on most real workloads.

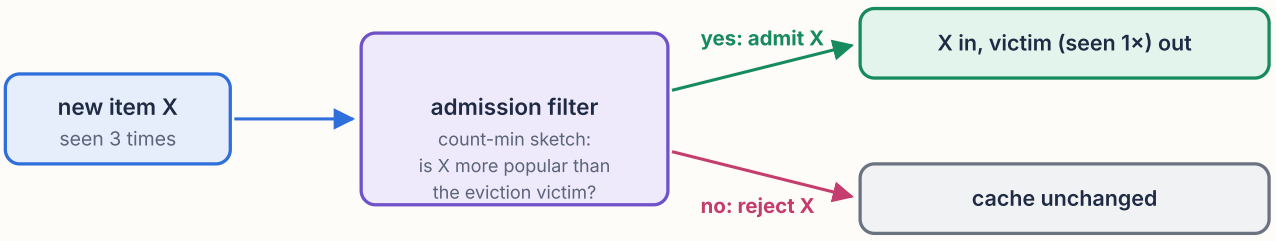


Figure 3. TinyLFU's admission filter: a new item only replaces the eviction victim if it has been seen more often.

COMMON MISTAKES

- LFU without decay: yesterday's viral key is cached forever.
- Assuming LRU everywhere, then losing the hot set to every batch job and crawler.
- Changing the eviction policy without measuring the hit ratio before and after on real traffic.

6 Key terms

LFU: evict the lowest access count

FIFO: evict the earliest inserted

Scan resistance: a one-off scan doesn't flush the hot set

TinyLFU: admit a new item only if it's more popular

Decay: counts shrink over time so old fame fades

Cache pollution: never-reused entries evicting useful ones

Count-min sketch: compact approximate frequency counter

W-TinyLFU: TinyLFU plus a small LRU window (Caffeine)

7 Interview questions

Q1. When does LFU beat LRU? BASIC

When popularity is stable over time, like top products, and when scans or one-off accesses would otherwise push hot items out, because LFU needs repeated use to keep an item.

Q2. What is cache pollution? BASIC

Filling the cache with entries that won't be reused, evicting valuable ones; LFU and TinyLFU resist it better than LRU.

Q3. Why does LFU need decay? BASIC

Without it, items that were popular long ago keep high counts forever and block new hot items; decaying counters make frequency reflect recent popularity.

Q4. Your cache hit ratio drops every night when a reporting job runs. What do you change? SCENARIO

Switch to a scan-resistant policy (allkeys-lfu in Redis, W-TinyLFU in-process), or make the job bypass the cache, and measure the hit ratio before and after.

Q5. When is FIFO a reasonable choice? TRICKY

When entries have similar value or short TTLs, when you need reads with no bookkeeping for scalability, or as part of a design like S3-FIFO that adds a probation queue to filter one-hit items.

Q6. How does TinyLFU decide admission cheaply? TRICKY

It keeps approximate access counts in a count-min sketch (with periodic halving), and compares the newcomer's estimated frequency with the eviction candidate's, admitting only if the newcomer is more popular.

Next: SD-F-008 · Cache Invalidation Strategies backendroadmap.com

Cache Invalidation Strategies

keeping copies fresh, and why it's so hard

Watch the 4-min video

In one line: caches stay fresh by TTL expiry (simple, stale up to the TTL), explicit delete on write (fast, but every write path must remember), event-driven invalidation from the database log (nothing forgets), and versioned keys or tag purges; it's hard because reads, writes and invalidation messages race, get delayed or reordered.

1 The famous hard problem

"There are only two hard things in computer science: cache invalidation and naming things" (Phil Karlton). It's an old joke, but it's true. A user changes their name, and three different caches keep showing the old one for hours. Here are the strategies for keeping caches fresh, and why it's hard to get right.



Figure 1. Four ways to invalidate. Real systems combine them: precise invalidation for speed, TTLs as the safety net.

2 TTL-based invalidation

The simplest strategy: don't invalidate at all; entries just **expire** (lesson 19). It's easy and **self-healing**: even if something goes wrong, the stale value disappears eventually. The cost: data can be stale for up to the TTL. Fine for data with a generous staleness budget.

3 Explicit delete on write

When the application updates the database, it deletes the related cache keys, as in cache-aside (lesson 16). It's fast and precise, but **every code path that writes must remember** to invalidate, including admin tools, batch jobs and migrations. And one change can affect many keys: a user's name appears in `user:42`, their posts list, a team page and search results.

4 Event-driven invalidation

Every change to the database is published as an **event**, either by the application or by **change data capture** (CDC), which reads the database's own log (Postgres WAL, MySQL binlog) with a tool like Debezium. Cache invalidators subscribe and delete the affected keys. With CDC, no write path can forget, because the database log captures every change, even a manual SQL fix.

WORKED EXAMPLE: A PROFILE UPDATE

A user updates their profile. The user service writes to Postgres and publishes a `user-updated` event to a Kafka topic (or Debezium emits it from the WAL). Several consumers listen: the profile cache service deletes `user:42`, the feed service clears cached author info, and the search indexer updates the user's document. Each team owns its own invalidation, and the user service doesn't need to know which caches exist.

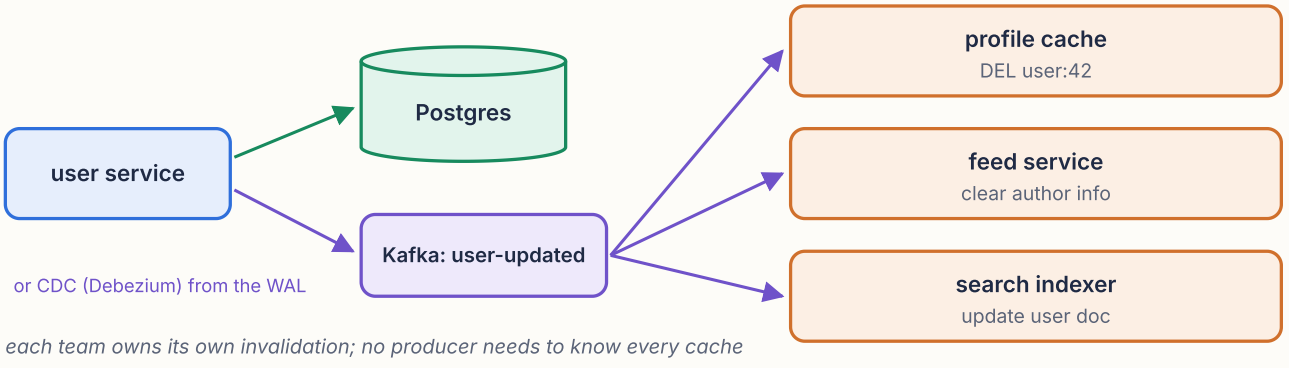


Figure 2. Event-driven invalidation: one change event fans out to every cache that depends on the data.

5 Versioned keys and tag-based purges

- **Versioned keys** put a version in the key: `product:42:v7`. When the product changes, the version increments (stored in the database row, or a tiny version key), and readers ask for the new key. Old versions simply expire. No delete, so no delete race.
- **Tag-based purges** label entries with tags, like `product-42`, and purge every entry with that tag at once. CDNs such as Fastly (surrogate keys) and Cloudflare (cache tags) support it: one product change purges its page, its category pages and its API responses.

6 Why it's hard: distributed timing

Writes and reads race, and invalidation messages can be **delayed, duplicated or reordered**. The classic race: a reader misses and reads the *old* value, then pauses. Meanwhile a writer updates the database and deletes the key. Then the reader wakes up and writes the old value into the cache. The cache now holds stale data until the TTL expires. Next lesson shows how to stop that properly.

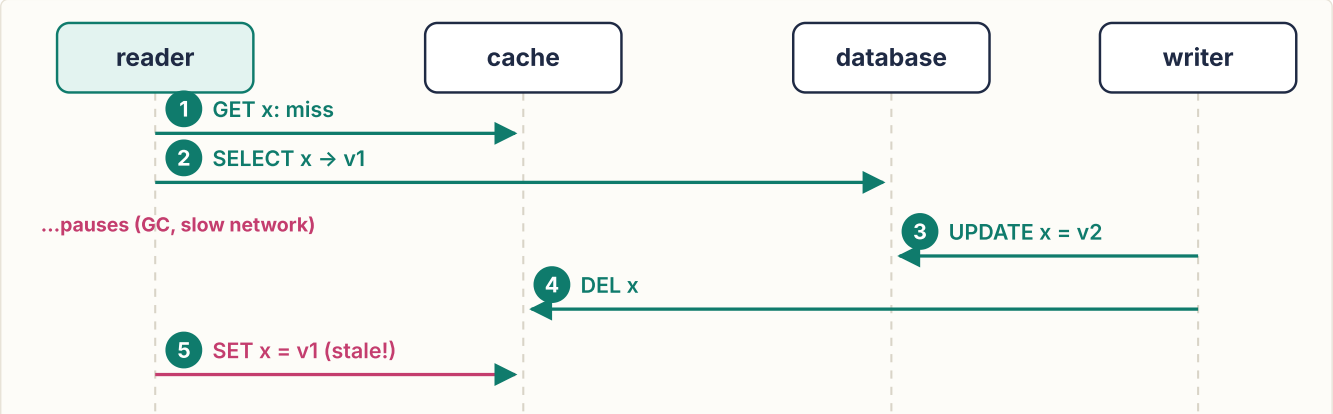


Figure 3. The stale-set race: the delete happens between the reader's database read and its cache write.

COMMON MISTAKES

- Invalidating only the obvious key and forgetting the derived ones (lists, aggregates, rendered pages).
- No TTL as a safety net, so a lost event leaves stale data forever.
- Invalidation consumers that aren't idempotent, or that crash and skip events.
- Long browser or CDN cache lifetimes on data you'll need to change quickly.

7 Key terms

Invalidation: removing or marking a cache entry stale after a change

CDC: change data capture: stream changes from the DB log

Versioned key: the version is part of the key

Stale set: an old value written into the cache after invalidation

Delete on write: the writer deletes related keys

Debezium: a common open-source CDC tool

Tag / surrogate key: label for purging many entries at once

8 Interview questions

Q1. Why is cache invalidation hard? BASIC
Because caches and databases are updated separately, so concurrent reads and writes, delayed or reordered invalidation messages, and many dependent keys can leave stale data behind.

Q2. Describe a race that leaves a stale value. BASIC
A reader misses and reads the old value, a writer updates the database and deletes the key, then the slow reader writes the old value into the cache, where it stays until the TTL.

Q3. Ten services cache user data. How do you invalidate them all when a user changes? SCENARIO
Publish a change event (or use CDC from the database log) to a topic; each service subscribes and invalidates its own keys; keep TTLs as a safety net.

Q4. A product change must update its page, category pages and API responses on the CDN. How? SCENARIO
Tag every cached response with the product's tag (surrogate key) and purge by tag on change; or version the URLs so new content gets new cache keys.

Q5. Why does CDC make invalidation more reliable than app-level deletes? TRICKY
Every committed change appears in the database log, including from batch jobs, admin tools or manual SQL, so no write path can skip invalidation; the app can't forget.

Q6. How do versioned keys avoid the delete race? TRICKY
Readers always look up the current version first, so a slow reader that caches an old value writes it under the old key, which nobody reads anymore; it simply expires.

Next: SD-F-009 · Cache Consistency

Cache Consistency

stop stale sets, and say how stale is acceptable

Watch the 4-min video

In one line: a slow reader can write an old value after a delete, and without a TTL it stays forever; defend with delete-after-write, TTLs, delayed double delete, and properly with leases (a token issued on a miss and voided by a delete), then decide explicitly which data accepts bounded staleness and which must read the source of truth.

1 Forever stale

Last lesson's race: a slow reader writes an old value into the cache after a delete. With a TTL, the damage is limited. Without one, the cache serves that value **forever**, or until someone notices. Step by step: reader A misses on key x and reads version 1 from the database. Writer B updates the database to version 2 and deletes x. Reader A, delayed by a garbage-collection pause or a slow network, now sets x to version 1. Every future read hits that stale value, and nothing ever deletes it.

2 First lines of defence

- **Delete after the database write, never before.** Deleting first lets a reader repopulate the old value before the write lands; deleting after shrinks the window to the rare race above.
- **Always set a TTL**, even a long one (an hour, a day), as a safety net, so no stale value lives forever.

3 Delayed double delete

The writer updates the database, deletes the key, waits a short time (say 500 ms, longer than a typical read takes), and deletes it **again**, usually from a background task or a delayed queue message. Any stale value set by a slow reader in that window is cleaned up. It's simple, but it's a heuristic: a reader slower than the delay can still win.

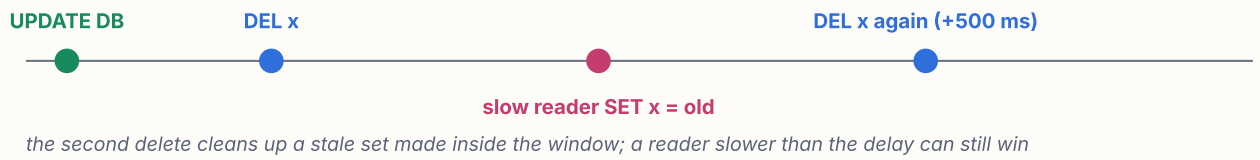


Figure 1. Delayed double delete: a second delete after the window removes stale sets from slow readers.

4 Leases: fixing the race properly

On a miss, the cache gives the reader a **lease token** for that key. When the reader sets the value, it must present the token. If a delete happened in between, the cache has invalidated the token, so the stale set is **rejected**.

Leases also prevent herds: while one client holds the lease for a key, other clients that miss are told to **wait briefly and retry** (they'll usually find the value filled), instead of all querying the database.

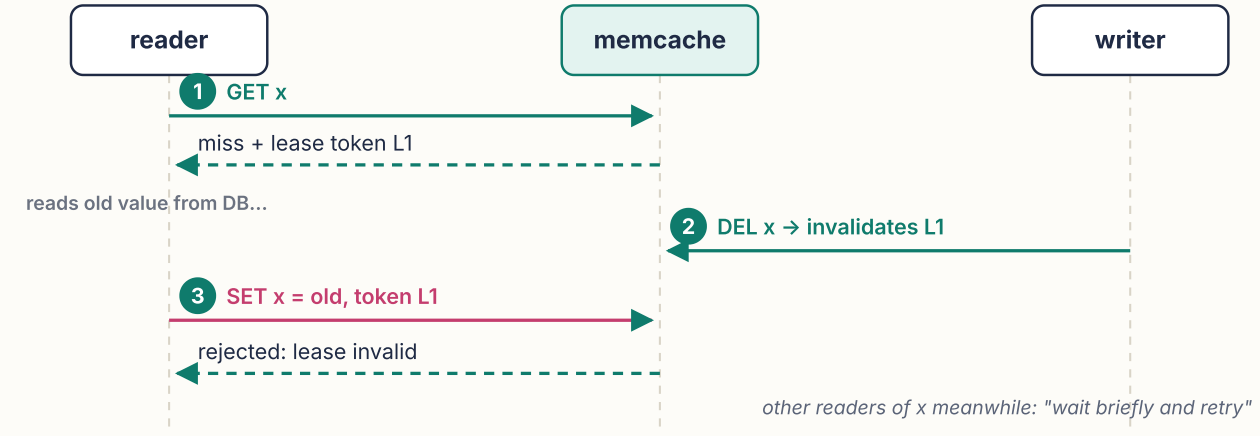


Figure 2. Leases: the delete voids the reader's token, so its late write of the old value is refused.

WORKED EXAMPLE: FACEBOOK'S MEMCACHE LEASES

In the paper *Scaling Memcache at Facebook* (NSDI 2013), a reader that missed got a 64-bit lease token, and a delete of the key invalidated outstanding leases, so stale sets failed. The cache issued **at most one lease per key every 10 seconds**, so a hot key's miss sent one query to the database instead of thousands. Facebook reported that this cut peak database query rates on hot keys from about 17,000 per second to about 1,300.

5 Decide how much consistency you need

Most data can accept **bounded staleness**: values may be a few seconds (or minutes) old, guaranteed by TTLs and invalidation. For data that must be exact, like balances or inventory at checkout, read from the database, or use write-through with strong guarantees. Being explicit about the staleness bound ("the product page may be up to 60 seconds stale; checkout always reads the database") is a great point to make in interviews.

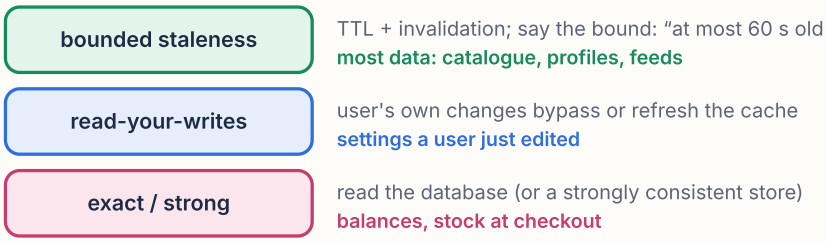


Figure 3. Three consistency levels and where each fits.

COMMON MISTAKES

- No TTL anywhere, so one lost delete or race means stale data forever.
- Promising "the cache is always consistent" in an interview: say the bound instead.
- Using cached stock levels to decide whether checkout can succeed.

6 Key terms

Stale set: writing an old value into the cache after a change

Lease token: proof a reader may fill a key; voided by deletes

Read-your-writes: a user always sees their own latest change

Delayed double delete: delete, wait, delete again

Bounded staleness: data may be old, but never older than X

Source of truth: the store that holds the correct value

7 Interview questions

Q1. How can a cache serve stale data forever? BASIC

A slow reader that missed can write an old value into the cache after a writer has updated the database and deleted the key, and without a TTL nothing ever removes it.

Q2. How do leases help? BASIC

The cache issues a token on a miss and invalidates it on delete, so a stale set with an old token is rejected; limiting leases per key also stops thundering herds.

Q3. What staleness would you accept for a product page, a cart and a payment? SCENARIO

Product page: tens of seconds, with invalidation on change. Cart: read-your-writes (the user's own changes must show). Payment: none; read and write the database transactionally.

Q4. Your cache library has no leases. How do you reduce stale sets? SCENARIO

Delete after writing, set TTLs on everything, add a delayed second delete, use versioned values (only SET if the version is newer, with a Lua script), or let CDC drive invalidation.

Q5. Why is delayed double delete only a heuristic? TRICKY

A reader slower than the delay (a long GC pause, a network stall) can still write its old value after the second delete; only a mechanism tied to the key's version or a lease closes it completely.

Q6. How do leases prevent a thundering herd as well as stale sets? TRICKY

Only one client gets a lease for a missing key per interval; the others are told to wait and retry, so one database query refills the key instead of thousands.

Next: SD-F-010 · Cache Stampede and Thundering Herd backendroadmap.com

Cache Stampede and Thundering Herd

when one hot key expires and everyone rebuilds it

Watch the 4-min video

In one line: a stampede happens when a hot key expires and thousands of concurrent misses all regenerate it; prevent it with single-flight inside a server, a short distributed lock across servers, early refresh (probabilistic or scheduled) so the key never expires under load, and stale-while-revalidate so users get the old value while one task refreshes it.

1 One key, fifty thousand readers

Your homepage data is cached under one key and read 50,000 times per second. The key expires. In the next 100 milliseconds, thousands of requests miss, and every one of them runs the same expensive query against the database. The database slows, queries pile up, the next requests time out, and the whole site goes down. That's a **cache stampede**.

2 Why it's so dangerous

A stampede, also called **dogpiling** or a **thundering herd**, happens when many requests miss on the same key at the same time and all try to regenerate it. The more popular the key and the slower the regeneration, the worse it gets: 50,000 requests per second × a 200 ms query = about **10,000 concurrent queries for one value**. And the slower the database gets, the longer each query takes, so even more requests pile in.

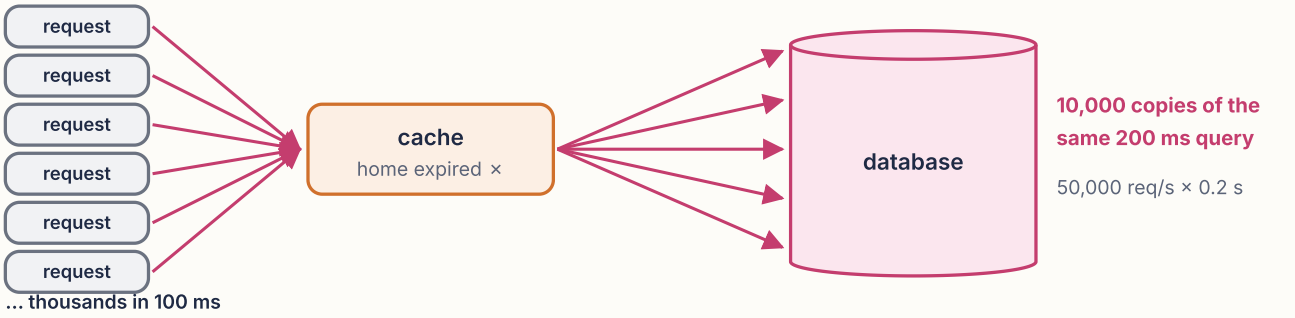


Figure 1. A stampede: every request that arrives while the key is missing runs the same query.

3 Fix 1: request coalescing (single-flight)

Within one server, only the **first** request for a missing key goes to the database. Concurrent requests for the same key wait for that result and share it. Go's `singleflight` package does exactly this; in Python, a per-key `asyncio` future does the same. It turns thousands of queries per server into one per server.

4 Fix 2: a distributed lock

Across many servers, the first request that misses takes a short lock in Redis with `SET lock:home <id> NX PX 5000` (only if not exists, expires in 5 s). It regenerates the value, stores it and releases the lock. Requests that fail to get the lock either wait briefly and re-read the cache, or serve a slightly stale value. Always give the lock an expiry, so a crashed holder doesn't block everyone, and release it only if you still own it (compare the id).

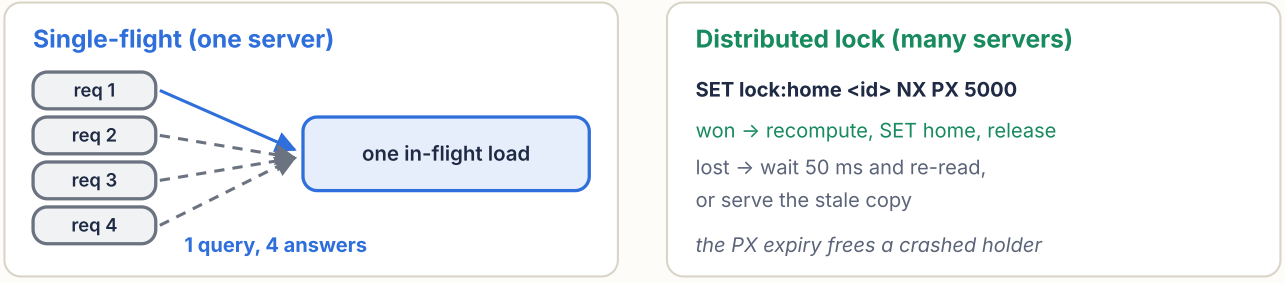


Figure 2. Single-flight collapses misses inside one server; a lock with an expiry lets only one server regenerate.

5 Fix 3: never let a hot key expire under load

Probabilistic early expiration (the **XFetch** algorithm): each request near the end of the TTL has a small, growing chance of refreshing the value early. One lucky request recomputes it before it expires, while everyone else keeps hitting:

```
refresh early if now − delta × beta × ln(random()) ≥ expiry   (delta = recompute time, beta ≈ 1)
```

Or simply run a **background job** that refreshes known hot keys on a schedule, so they never expire at all.

6 Fix 4: stale-while-revalidate

Keep serving the old value after it goes stale, while a single background task fetches the fresh one. Users get an instant response, and only one regeneration runs. Implement it with a **soft TTL** stored inside the value (refresh after it) and a longer **hard TTL** on the key (delete after it). HTTP supports the same idea with `Cache-Control: stale-while-revalidate`, and CDNs honour it.

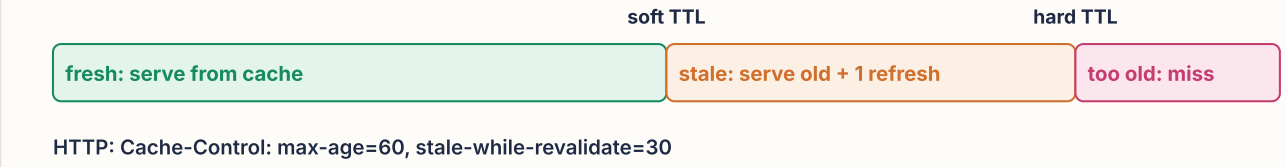


Figure 3. Soft and hard TTLs: between them the cache serves the old value while exactly one request refreshes it.

WORKED EXAMPLE: THE HOMEPAGE KEY

The homepage key is read at 50,000 requests per second, with a 60-second TTL. Without protection, each expiry sends thousands of queries to the database. With a 60-second soft TTL, a 120-second hard TTL, stale-while-revalidate and a Redis lock so only one worker refreshes, each expiry causes **exactly one query**, and users never see a slow page.

COMMON MISTAKES

- A lock without an expiry: one crashed worker blocks the key forever.
- Lock losers that retry immediately in a tight loop, hammering Redis.
- Fixing stampedes on one key but leaving thousands of keys with the same TTL (that's an avalanche, next lesson).

7 Key terms

Cache stampede: many concurrent misses regenerating one key

Single-flight: one in-flight load per key, shared by waiters

XFetch: probabilistic early refresh before expiry

Stale-while-revalidate: serve old while one task refreshes

Dogpiling / thundering herd: other names for the same thing

Distributed lock: SET NX PX so one server regenerates

Soft / hard TTL: refresh after soft, delete after hard

8 Interview questions

Q1. What is a cache stampede? BASIC

When a hot key expires or is evicted, many concurrent requests miss at once and all regenerate it, overloading the database.

Q2. How do you prevent it? BASIC

Coalesce requests with single-flight, use a distributed lock so one request regenerates, refresh early with probabilistic expiration or background jobs, and serve stale values while revalidating.

Q3. Design caching for a homepage read 50,000 times a second. SCENARIO

Cache it in Redis (and in process memory for a second or two), refresh it from a background job or with a soft TTL plus stale-while-revalidate, guard regeneration with a lock, and never let it hard-expire under load.

Q4. Your lock-based fix works, but lock losers add latency. What else? SCENARIO

Let losers serve the stale value instead of waiting, add a short in-process cache in front of Redis, and refresh early so the key is rarely missing at all.

Q5. Why is the lock's expiry important, and what's the risk if it's too short? TRICKY

Without it, a crashed holder blocks the key forever; if it's shorter than the regeneration time, a second worker can acquire it and you get two regenerations. Set it longer than the slowest recompute and check ownership on release.

Q6. How does XFetch avoid stampedes without locks? TRICKY

Each read near expiry refreshes with a probability that rises as expiry approaches (and with longer recompute times), so typically one request refreshes early and the key never expires while popular.

Next: SD-F-011 · Cache Penetration and Avalanche backendroadmap.com

Cache Penetration and Avalanche

requests for keys that don't exist, and caches that fail all at once

Watch the 4-min video

In one line: cache penetration is requests for keys that don't exist passing straight through to the database; fix it with negative caching (a short-TTL "not found"), a Bloom filter of valid keys and validation and rate limits; a cache avalanche is a mass expiry or cache failure that floods the database, prevented with TTL jitter, replicas, warming and database protection.

1 Requests for things that don't exist

An attacker sends requests for random product IDs: 9,800,123, 7,415,002 and so on. None of them exist. Every request misses the cache, goes to the database, finds nothing, and **caches nothing**. The cache is useless, and the database takes every hit. That's **cache penetration**. It can be accidental too: a buggy client looping on a deleted ID, or a scraper walking an ID range.



Figure 1. Penetration: "not found" is never cached, so every request for a missing key reaches the database.

2 Fix 1: negative caching

When the database returns nothing, **cache that fact**: store a special null marker under the key with a short TTL, maybe 30 to 60 seconds. Repeat requests for the same missing key now hit the cache. Keep the TTL short (and delete the marker when the item is created), so a new item becomes visible quickly.

3 Fix 2: a Bloom filter in front

Random IDs defeat negative caching, because each ID is new. A **Bloom filter** helps: a compact bit array plus k hash functions that answers *"definitely not in the set"* or *"possibly in it"*. To add a key, set the k bits its hashes point to. To check a key, look at those bits: if any is 0, the key was never added.

Load all valid product IDs into a Bloom filter. Before touching the cache or the database, check the filter; if it says **definitely not**, reject immediately. False positives are rare and tunable; false negatives never happen. Deleting is the catch: a plain Bloom filter can't remove items, so rebuild it periodically (or use a counting or cuckoo filter).

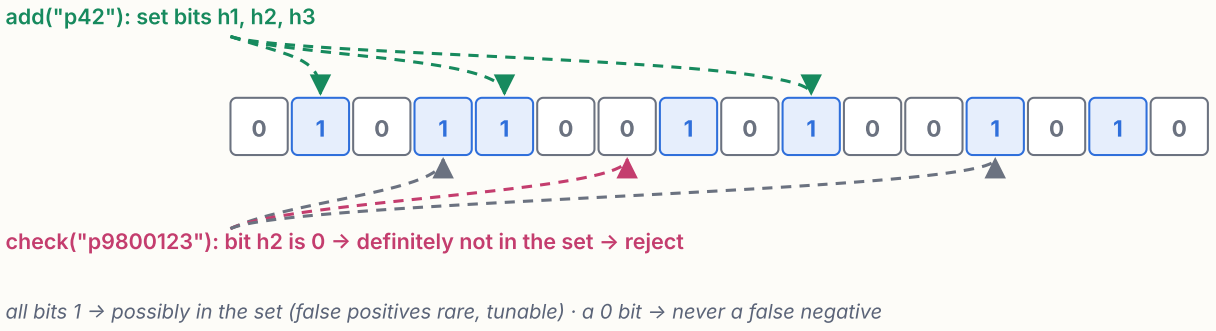


Figure 2. A Bloom filter: adding sets bits; a check that finds any 0 bit proves the key was never added.

WORKED EXAMPLE: RANDOM PRODUCT IDS

The service keeps a Bloom filter of all **10 million** valid IDs. With a 1% false-positive rate it needs about 9.6 bits per item: roughly **12 MB** of memory and 7 hash functions (RedisBloom's `BF.RESERVE products 0.01 10000000` sets this up). 99% of bogus requests are rejected in microseconds; the remaining 1% are caught by negative caching. On top: rate limiting per client, and validating the ID format at the edge.

4 Cache avalanche

An **avalanche** is when a large part of the cache fails at once and the full load falls on the database. Two common causes:

- **Mass expiry**: many keys set at the same time with the same TTL expire together (lesson 19).
- **Cache failure**: a Redis node or cluster goes down, or restarts empty.

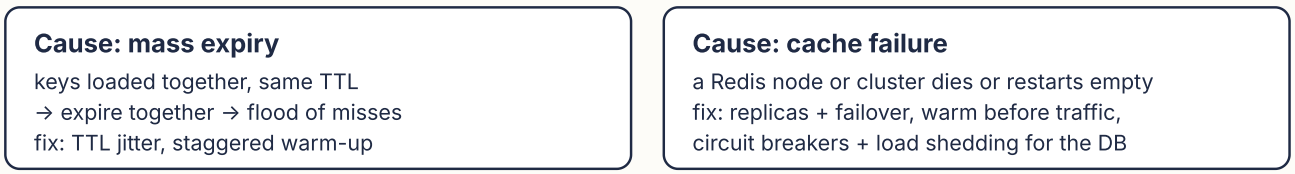


Figure 3. The two causes of an avalanche and how to prevent each.

Prevention: add **jitter** to TTLs; run a **highly available** cache with replicas and automatic failover (lesson 28); **warm** the cache before sending traffic to a new or restarted node; and protect the database with **circuit breakers, rate limits and load shedding** (lesson 10), so if the cache does fail, the database degrades instead of collapsing.

PENETRATION, BREAKDOWN, STAMPEDE, AVALANCHE: DON'T MIX THEM UP

- **Penetration**: keys that don't exist, every request misses.
- **Stampede / breakdown**: one hot key expires, many concurrent misses (lesson 24).
- **Avalanche**: many keys or the whole cache fail at once.

5 Key terms

Cache penetration: requests for non-existent keys reaching the DB

Bloom filter: bit array: definitely not, or possibly in

Cache avalanche: mass expiry or cache failure floods the DB

Negative caching: caching "not found" with a short TTL

False positive: filter says maybe, but the key doesn't exist

Cache warming: pre-loading hot keys before taking traffic

6 Interview questions

Q1. How do you stop requests for missing keys from hammering the database? BASIC
Cache negative results with a short TTL, put a Bloom filter of valid keys in front so impossible keys are rejected early, and add validation and rate limiting.

Q2. What is a cache avalanche? BASIC
A mass expiry or cache cluster failure that sends a flood of misses to the database, prevented with TTL jitter, replicas, cache warming and database protection.

Q3. Can a Bloom filter give false negatives? BASIC
No. If a key was added, all its bits are 1, so the filter always says 'possibly in'. It can give false positives, at a rate you choose by sizing the bit array and hash count.

Q4. Scrapers request user IDs from 1 to 100 million, most of which don't exist. What do you do? SCENARIO
Rate-limit per client and IP, reject IDs that fail a Bloom filter of existing users, cache 'not found' briefly, and consider non-sequential IDs so the space can't be walked.

Q5. Your Redis cluster restarts empty after maintenance, and the database falls over. How do you avoid that next time? SCENARIO
Use replicas with failover so a node restart doesn't empty the cache, warm hot keys before sending traffic, ramp traffic gradually, and guard the database with rate limits and load shedding.

Q6. How big is a Bloom filter for 10 million keys at 1% false positives? TRICKY
About 9.6 bits per key, so roughly 96 million bits ≈ 12 MB, with about 7 hash functions; halving the false-positive rate costs only about 1.4 more bits per key.

Next: SD-F-012 · Hot Keys

Hot Keys

one key, one node, and a million readers

Watch the 4-min video

In one line: sharding spreads keys, not traffic: one very popular key lives on one node and can saturate it; detect hot keys with metrics or sampling, then serve them from a short-lived local (L1) cache on each app server, copy them under several suffixes across shards, or read them from replicas, accepting a few seconds of staleness.

1 A celebrity posts

A celebrity posts, and their profile is read a million times a minute. All those reads hit one cache key, which lives on one Redis node. That node's CPU hits 100% while the rest of the cluster sits idle. Adding more nodes doesn't help, because the key can only live in one place. That's the **hot key** problem.

2 Why sharding doesn't save you

Sharding (consistent hashing in lesson 5, Redis Cluster slots in lesson 28) spreads *keys* across nodes, but each key maps to exactly **one** node. So the cluster's capacity for a single key is the capacity of one node, maybe 100,000 simple operations per second, less with big values, which also saturate the node's network. When one key gets more traffic than that, its node becomes the bottleneck, and every other key on it suffers too.

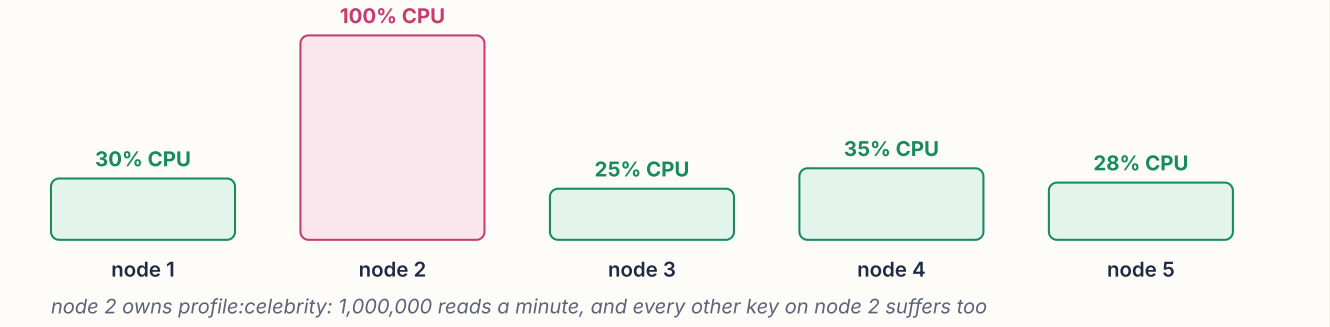


Figure 1. One hot key: its node is saturated while the others idle.

3 Detect them first

- `redis-cli --hotkeys` (needs an LFU eviction policy, so Redis tracks access frequency).
- Sampling with `MONITOR`, carefully: it's expensive on a busy server, so only for seconds.
- Counting key accesses in the client library or a proxy, and exporting the top-N as metrics.
- Alerting when one node's CPU or network is far above its peers.

4 Fix 1: a local in-process cache (L1)

The most effective fix: keep the hottest keys in memory **on each application server** for a short time, 1 to 5 seconds. With 200 app servers and a 2-second L1 TTL, the Redis node sees at most about 200 requests every 2 seconds for that key, instead of a million a minute. The trade-off is a few seconds of staleness. Lesson 32 covers multi-level caching in general.

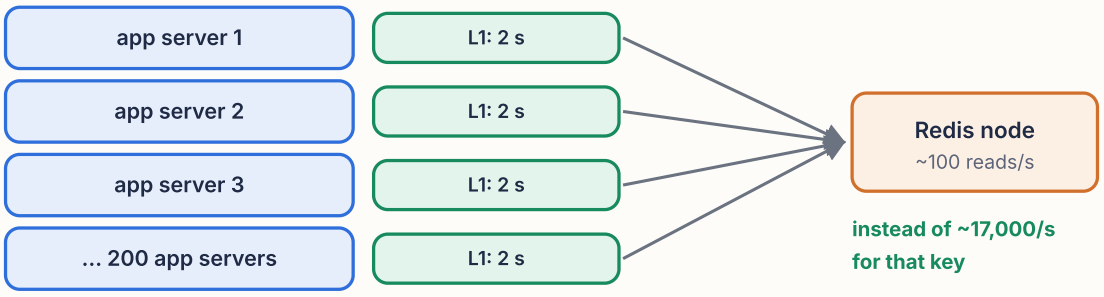


Figure 2. A 2-second in-process cache on every app server shrinks the hot key's load on Redis by about 170 times.

5 Fix 2: replicate the key under suffixes

Store copies of the hot value under several keys, like `profile:42#1` to `profile:42#10`. Each lands on a different shard. Readers pick a random suffix, spreading the load across ten nodes. Writes must update or delete **all** copies, so this suits read-mostly keys.

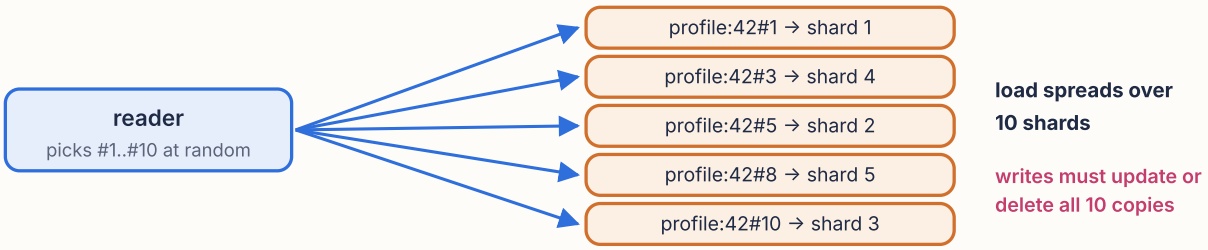


Figure 3. Suffix replication: ten copies on ten shards, chosen at random by readers.

6 Fix 3: read replicas and smart proxies

Redis replicas can serve reads (`READONLY` in cluster mode), so a hot key's reads spread across the primary and its replicas, with a little replication lag. Some managed caches and proxies detect hot keys automatically and cache them closer to the client.

WORKED EXAMPLE: THE CELEBRITY PROFILE

A million reads per minute is about **17,000 per second**. Each of 200 app servers caches the profile locally for 2 seconds, so Redis sees about **100 reads per second** for that key. The key is also replicated under ten suffixes as a backup if L1 is bypassed. When the celebrity updates their profile, all copies are deleted, and readers accept up to 2 seconds of staleness.

COMMON MISTAKES

- Adding cache nodes to fix a hot key: it still lands on one of them.
- Huge values on a hot key (a 1 MB JSON read 10,000 times a second is 10 GB/s of network).
- Forgetting that hot *writes* (a global counter) need a different fix: shard the counter and sum the parts.

7 Key terms

Hot key: one key with far more traffic than others

L1 / local cache: short-lived in-process cache on each app server

--hotkeys: redis-cli option to list hot keys (LFU needed)

Hot shard: the node that holds a hot key

Suffix replication: copies under key#1..key#n on different shards

Sharded counter: split a hot counter into parts, sum on read

8 Interview questions

Q1. How do you handle a single extremely hot cache key? BASIC
Detect it with metrics or hotkeys sampling, add a short-lived local in-process cache on app servers, replicate the key under several suffixes across shards, and use read replicas, accepting a few seconds of staleness.

Q2. Why doesn't adding nodes help? BASIC
Because one key always maps to one shard, so its load can't spread unless you copy it.

Q3. A flash sale item gets 200,000 reads per second. Design the read path. SCENARIO
Serve it from an in-process cache refreshed every second or two on every app server (and a CDN for the page), with Redis behind it; replicate the Redis key under suffixes; never let the key expire under load.

Q4. A global 'likes' counter key is hot for writes. What do you do? SCENARIO
Shard the counter: increment one of N sub-keys chosen at random, and sum them on read (cached briefly), or aggregate in memory and flush periodically (write-behind).

Q5. What's the cost of an L1 cache on every app server? TRICKY
Staleness up to its TTL, memory on every server, and inconsistency between servers during that window; for most hot read keys a 1 to 5 second bound is acceptable.

Q6. How would you find hot keys in production without slowing Redis? TRICKY
Enable an LFU policy and use redis-cli --hotkeys, or count accesses in the client or proxy and export top-N metrics; avoid long MONITOR sessions on busy servers.

Next: SD-F-013 · Redis Fundamentals

Redis Fundamentals

data structures, one fast thread, persistence and eviction

Watch the 5-min video

In one line: Redis is an in-memory data structure server (strings, hashes, lists, sets, sorted sets, streams); it's fast on one thread because data is in memory, commands are simple, there are no locks and an event loop serves many connections; every command is atomic, Lua scripts make multi-step logic atomic, RDB and AOF give persistence, and maxmemory policies decide eviction.

1 One core, hundreds of thousands of ops per second

Redis powers caches, leaderboards, rate limiters, session stores and queues, and one instance handles hundreds of thousands of operations per second on a single core. But it's single-threaded, so how is it so fast? And if it lives in memory, what happens when it restarts?

2 Data structures, not just strings

Redis is an in-memory **data structure server**. Choosing the right structure often turns a complex feature into one command.

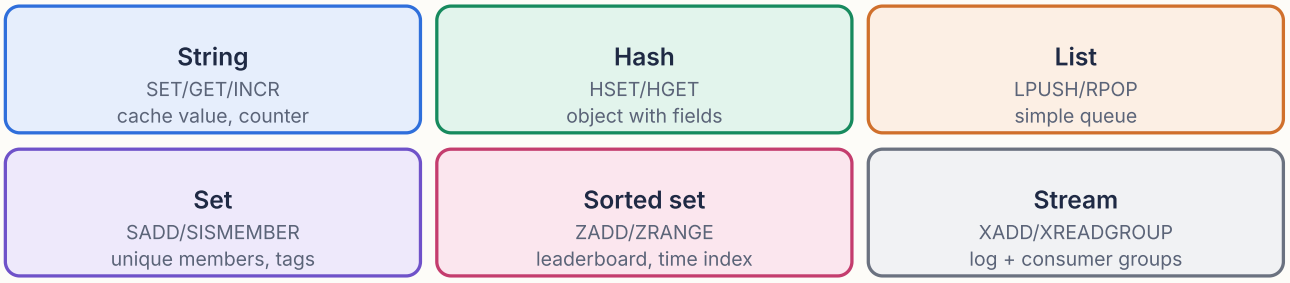


Figure 1. The core Redis data types, a typical command for each, and what they're used for.

3 Why it's fast on one thread

- **Everything is in memory**, so there's no disk wait on reads.
- **Simple commands**: most run in O(1) or O(log n).
- **One thread**: no locks, no context switches between threads.
- **I/O multiplexing**: an event loop (epoll on Linux) handles thousands of connections at once.

The network is usually the bottleneck, not the CPU. Since Redis 6, network I/O can use extra threads (`io-threads`), while commands still run on one.

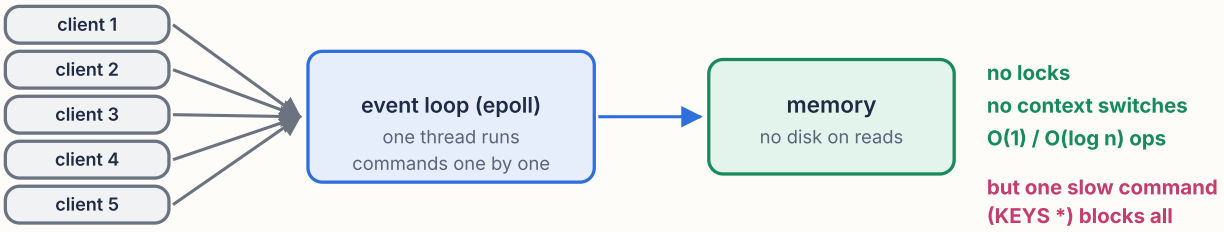


Figure 2. Many connections, one event loop, data in memory: fast, but one slow command delays everyone.

4 Atomic by design

Because commands run one at a time, **every command is atomic**: `INCR` can't be interrupted halfway. For multi-step logic, a **Lua script** (`EVAL`) runs atomically on the server: no other command runs in between. That's how a rate limiter checks and updates a counter safely. `MULTI` / `EXEC` transactions also run a batch without interleaving, but can't branch on results.

The flip side: one slow command blocks everyone. `KEYS *` on millions of keys, `HGETALL` on a huge hash or a long Lua script freezes the server; use `SCAN` instead.

5 Persistence: RDB and AOF

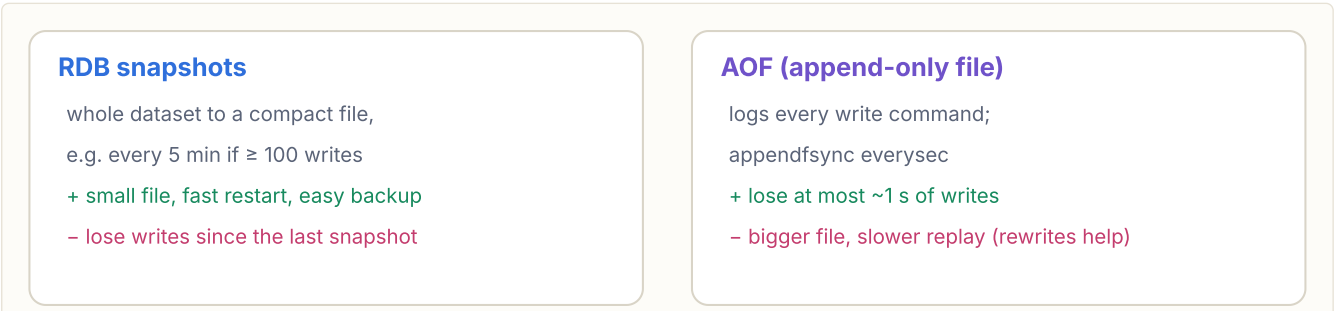


Figure 3. The two persistence options. Many deployments use both: AOF for durability, RDB for backups and fast restarts.

6 When memory fills up: eviction policies

Policy	Behaviour	Use for
`noeviction`	reject new writes with an error	Redis as a primary data store or queue
`allkeys-lru` / `allkeys-lfu`	evict from all keys (lessons 20, 21)	a pure cache: the usual choice
`volatile-lru` / `-lfu` / `-ttl`	evict only keys that have a TTL	mixed: cache keys with TTLs + keys that must stay

Set `maxmemory` and a policy explicitly, so behaviour under pressure is predictable.

WORKED EXAMPLE: A LEADERBOARD AND A RATE LIMITER

Leaderboard as a sorted set: `ZADD lb 1520 alice` sets a score, `ZREVRANGE lb 0 9 WITHSCORES` gets the top ten, `ZREVRANK lb alice` gives a player's position, each in O(log n). **Rate limiter**: `INCR rate:user42:202611061402` , then `EXPIRE … 60` on the first hit, both in one Lua script so increment and check happen atomically; over 100 → reject. And **pub/sub** can broadcast events, like cache invalidations, to many subscribers.

COMMON MISTAKES

- `KEYS *` in production code.
- No `maxmemory` , so the OS kills Redis when memory runs out.
- Treating Redis as durable by default: without AOF, a restart loses everything since the last snapshot.
- Huge values or collections (a 10 million element list) that make single commands slow.

7 Key terms

Sorted set (ZSET): members ordered by a score, O(log n) ops
Event loop: one thread serving many sockets via epoll
RDB: periodic snapshot file
maxmemory policy: what to do when memory is full
Stream: append-only log with consumer groups
Lua script: server-side script that runs atomically
AOF: log of every write command
SCAN: incremental key iteration, unlike blocking KEYS

8 Interview questions

Q1. Why is Redis fast despite being single-threaded? BASIC
Data lives in memory, commands are simple and efficient, a single thread avoids locks and context switches, and an event loop with I/O multiplexing serves many connections.

Q2. RDB versus AOF? BASIC
RDB takes periodic compact snapshots that restart fast but can lose minutes of writes, while AOF logs every write and loses at most about a second, at the cost of bigger files and slower restarts.

Q3. Which eviction policy for a cache? BASIC
allkeys-lru or allkeys-lfu with an explicit maxmemory; noeviction is for when Redis holds data you must not lose.

Q4. Build a top-10 leaderboard for a game with millions of players. SCENARIO
A sorted set keyed by game: ZINCRBY on each score change, ZREVRANGE 0 9 for the top ten and ZREVRANK for a player's rank, all O(log n); shard by game or season if one set gets too big.

Q5. Redis latency spikes every few minutes. What could it be? SCENARIO
A slow command (KEYS, big HGETALL, long Lua), RDB fork on a large dataset, AOF fsync on a slow disk, or eviction storms; check SLOWLOG, LATENCY DOCTOR and persistence settings.

Q6. How do you make 'check then update' atomic in Redis? TRICKY
Use a single command if one exists (INCR, SET NX), or a Lua script that reads and writes in one server-side step; WATCH with MULTI/EXEC gives optimistic locking as an alternative.

Next: SD-F-014 · Redis Replication, Sentinel and Cluster backendroadmap.com

Redis Replication, Sentinel and Cluster

high availability, and scaling beyond one machine

Watch the 5-min video

In one line: replicas copy the primary asynchronously and can serve reads; Sentinel watches the primary and promotes a replica when a quorum agrees it's down; Redis Cluster splits keys into 16,384 hash slots spread over several primaries, clients route by slot and follow MOVED redirects, and writes not yet replicated can be lost on failover.

1 One Redis is a single point of failure

Your Redis server holds every user session. One night it dies: everyone is logged out, and the app can't serve anyone until it's back. One node is a single point of failure, and it also caps your memory and throughput. Replication, Sentinel and Cluster solve those problems.

2 Replication

A **primary** accepts writes, and one or more **replicas** copy its data. Replication is **asynchronous**: the primary acknowledges a write to the client before replicas receive it. Replicas can serve reads and stand ready to take over. `WAIT 1 100` blocks until at least one replica confirms (or 100 ms pass), but by default Redis favours speed over durability.

3 Sentinel: automatic failover

Who promotes a replica when the primary dies? **Redis Sentinel**: a group of Sentinel processes, usually **three or more** on separate machines, monitors the primary. When a **quorum** agrees it's down, the Sentinels elect a leader that promotes the best replica (least lag), repoints the other replicas, and tells clients the new primary's address. Clients ask Sentinel where the primary is instead of hard-coding it.

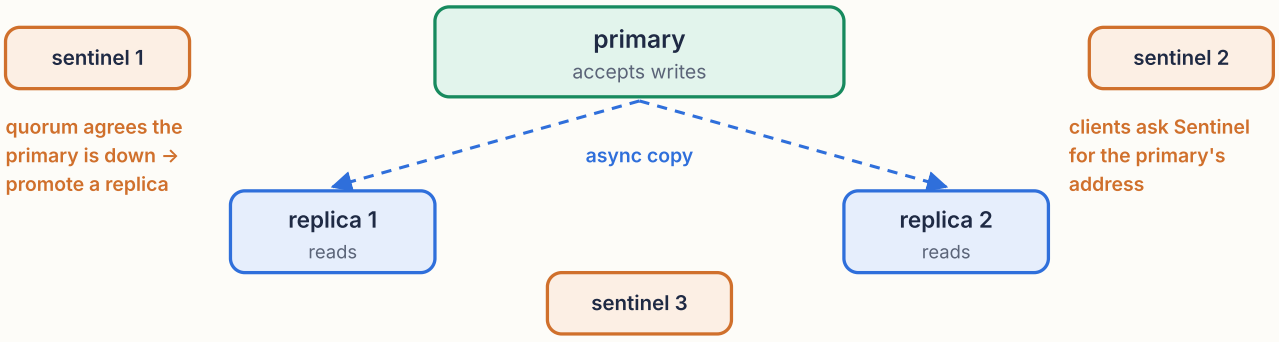


Figure 1. Replication with Sentinel: three Sentinels watch the primary; on failure a quorum promotes a replica and clients follow.

4 Redis Cluster: scaling out

Sentinel gives high availability, but all data still fits on one primary. To scale beyond one machine, use **Redis Cluster**. The key space is split into **16,384 hash slots**: a key's slot is `CRC16(key) mod 16384`. Each primary owns a range of slots, and each has its own replicas, which the cluster promotes automatically.

Clients learn the **slot map** and send each command straight to the right node. If a slot has moved, the node replies with a **MOVED** redirect (or **ASK** during a migration), and the client updates its map. **Resharding** moves slots between nodes while the cluster keeps serving; adding a node means migrating some slots to it.

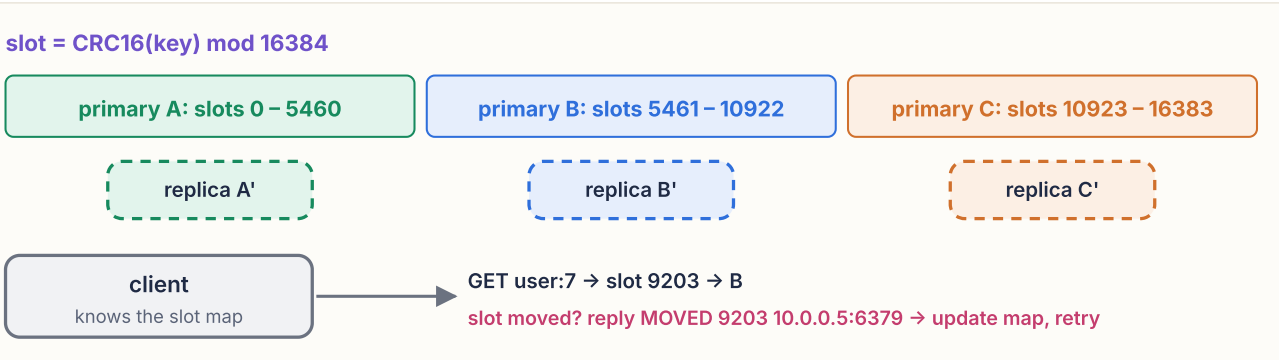
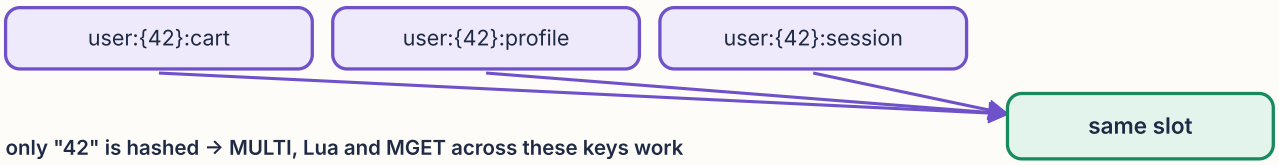


Figure 2. Redis Cluster: three primaries own slot ranges; clients route by slot and follow MOVED when slots move.

5 Multi-key operations and hash tags

Commands that touch several keys (`MGET`, `SUNION`), transactions and Lua scripts only work when **all keys are in the same slot**. **Hash tags** solve this: only the part of the key inside curly braces is hashed, so `user:{42}:cart` and `user:{42}:profile` land in the same slot. Use them deliberately: put everything under one tag and you create a hot slot.



only "42" is hashed → MULTI, Lua and MGET across these keys work

over-use one tag and you build a hot slot

Figure 3. Hash tags: the braces decide the slot, so related keys stay together.

WORKED EXAMPLE: A SESSION CLUSTER

A six-node Redis Cluster stores sessions: three primaries, each owning about **5,461 slots**, and one replica per primary in another availability zone. If a primary fails, its replica is promoted automatically within seconds (after `cluster-node-timeout`, e.g. 15 s). But because replication is asynchronous, writes acknowledged by the failed primary but not yet copied are **lost**: a few users may lose a recent session update, or need to log in again.

COMMON MISTAKES

- Two Sentinels (no majority possible when one fails), or all Sentinels on the same machine as the primary.
- Clients hard-coding the primary's address instead of asking Sentinel, or not handling MOVED.
- Multi-key Lua scripts without hash tags: `CROSSSLOT` errors in production.
- Assuming failover loses nothing: asynchronous replication always has a window.

6 Key terms

Replica: a copy of the primary, updated asynchronously

Sentinel: processes that monitor and fail over a primary

Hash slot: one of 16,384 buckets of the key space

Resharding: moving slots between nodes while online

WAIT: block until N replicas confirm a write

Quorum: how many Sentinels must agree it's down

MOVED / ASK: redirects to the node that owns a slot

Hash tag: {part} of a key that alone decides the slot

7 Interview questions

Q1. How does Redis Cluster distribute keys? BASIC

Keys are hashed with CRC16 into 16,384 hash slots, slots are spread across primaries, clients route by slot and follow MOVED redirects, and hash tags keep related keys in one slot.

Q2. What data can be lost on failover? BASIC

Writes the primary acknowledged but hadn't yet replicated, because replication is asynchronous; WAIT and AOF reduce but don't eliminate that risk.

Q3. Sentinel or Cluster? BASIC

Sentinel when the data fits on one machine and you need high availability; Cluster when you need more memory or throughput than one primary, with failover built in.

Q4. Sessions must survive a node failure. How do you set up Redis? SCENARIO

Replicas in another zone with automatic failover (Sentinel or Cluster), AOF everysec on primaries, WAIT 1 for critical writes if the latency is acceptable, and clients that reconnect to the new primary.

Q5. Why does a Lua script that touches user:42:cart and user:42:profile fail in Cluster? TRICKY

The keys hash to different slots, possibly on different nodes, so the script can't run atomically; rename them with a hash tag like user:{42}:cart so they share a slot.

Q6. What is split brain in Redis, and how is it limited? TRICKY

During a partition the old primary may keep accepting writes while a replica is promoted elsewhere; those writes are lost when it rejoins. min-replicas-to-write makes an isolated primary stop accepting writes, limiting the damage.

Next: SD-F-015 · Memcached vs Redis

Memcached vs Redis

a simple fast cache, or a data structure server

Watch the 4-min video

In one line: Memcached is a deliberately simple, multithreaded, volatile key-value cache with slab memory and client-side sharding; Redis is a data structure server with atomic operations, Lua, persistence, replication and pub/sub; pick Memcached for a pure blob cache on big multi-core boxes, Redis for everything else, which is most new systems.

1 Is Memcached just the old choice?

Everyone says Redis, and Redis is great. But Facebook, Wikipedia and many large sites have run Memcached for years, at enormous scale. Is it just the old choice, or sometimes the better one?

2 Memcached: deliberately simple

Memcached is an in-memory key-value store: you `get`, `set`, `delete` and `incr` **opaque values** by key, with a TTL. No data structures, no persistence, no replication, no scripting. Clients spread keys across servers with consistent hashing (ketama, lesson 5), so the servers don't even know about each other. Values are limited to 1 MB by default.

3 Threads and memory

Memcached is **multithreaded**: worker threads handle connections in parallel, so a single instance uses a big machine fully. Redis runs commands on one thread (lesson 27), so to use many cores you usually run several Redis instances or a cluster.

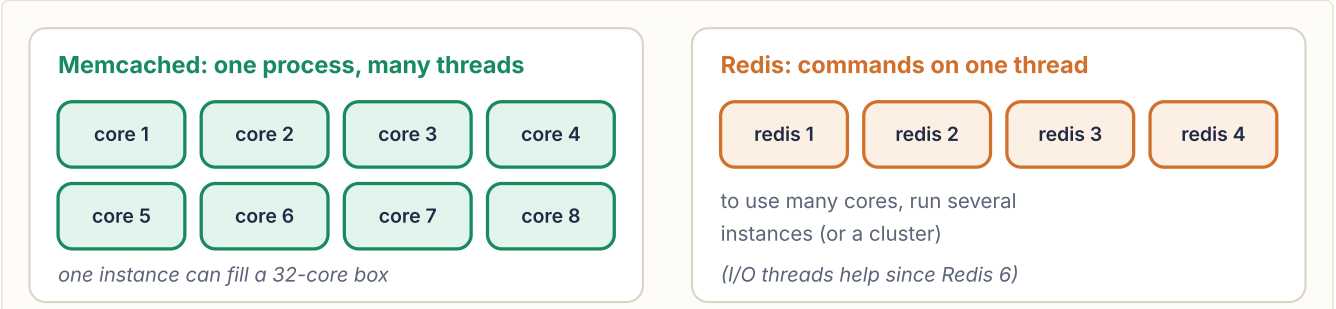


Figure 1. Memcached scales up with threads inside one process; Redis scales out with more processes.

Memory management differs too. Memcached uses a **slab allocator**: memory is divided into slab classes of fixed-size chunks (64 B, 128 B, 256 B, ... growing by a factor of 1.25 by default). Each item goes into the smallest chunk that fits. There's no fragmentation over time, and allocation is very fast. The cost is wasted space inside chunks, and if your item sizes shift, memory may be stuck in the wrong slab class.

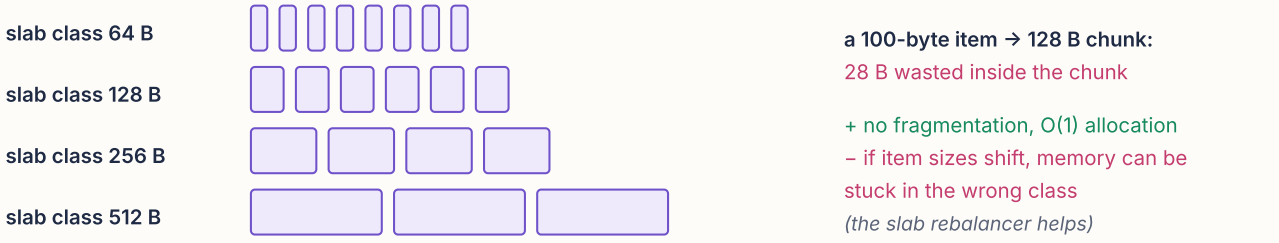


Figure 2. Slab classes of fixed-size chunks (simplified): each item takes the smallest chunk that fits.

4 Redis: a data structure server

Hashes, lists, sets, sorted sets, streams, pub/sub, Lua scripts, transactions, persistence with RDB and AOF, replication, Sentinel and Cluster (lessons 27 and 28). Redis can be a cache, a rate limiter, a queue, a leaderboard, a lock service, or a primary store for some use cases.

	Memcached	Redis
Data model	opaque strings / blobs	strings, hashes, lists, sets, sorted sets, streams
Threads	multithreaded	single-threaded commands (I/O threads optional)
Persistence	none	RDB snapshots, AOF
Replication / HA	none built in (client-side)	replicas, Sentinel, Cluster
Scaling out	client-side consistent hashing	Redis Cluster hash slots, or client/proxy sharding
Atomic logic	incr, CAS	every command, Lua scripts, transactions
Memory	slab allocator	jemalloc, can fragment (active defrag helps)

WORKED EXAMPLE: HTML FRAGMENT CACHE

A content site renders HTML fragments (a product card, a sidebar) and caches each one as a string for 5 minutes. That's pure key-value: get or set a blob, nothing else. Memcached fits perfectly: multithreading uses all the cores on a large cache box, slab allocation handles millions of similar-sized fragments, and losing the cache on restart is fine, because fragments can be rebuilt.

5 So which one?

- **Memcached**: a simple, volatile key-value cache of strings or serialised objects, at very high throughput, using many cores on big nodes, with minimal operational features.
- **Redis**: when you need data structures, atomic operations, persistence, replication and failover, pub/sub, or one tool that covers caching and more.

For most new systems Redis (or a compatible fork such as Valkey) is the default, and that's fine. Saying *why* is what earns points in an interview.

COMMON MISTAKES

- Storing data you can't rebuild in Memcached: it has no persistence.
- Running one Redis instance on a 64-core box and calling it saturated at one core.
- Picking Redis then using it only for get/set, and paying for persistence you don't need (turn it off for a pure cache).

6 Key terms

Opaque value: the cache doesn't look inside it

Slab class: one chunk size (64 B, 128 B, ...)

CAS: check-and-set: update only if unchanged

Slab allocator: fixed-size chunk classes, no fragmentation

Client-side sharding: clients pick the server by hashing

Valkey: open-source Redis fork

7 Interview questions

Q1. When would you pick Memcached over Redis? BASIC

For a pure, volatile key-value cache of strings or blobs, like HTML fragments or serialised objects, where you want multithreaded performance on large machines, predictable slab memory, and no need for data structures, persistence or replication.

Q2. What does Redis add? BASIC

Rich data structures, atomic operations and Lua, persistence, replication with failover, and pub/sub.

Q3. You need a leaderboard, a rate limiter and a page cache. One tool or two? SCENARIO

Redis for all three: sorted sets for the leaderboard, INCR plus EXPIRE in Lua for the limiter, strings with TTLs for pages. Memcached can't do the first two.

Q4. Memcached evicts items even though stats show free memory. Why? SCENARIO

Slab calcification: memory is assigned to slab classes for old item sizes, and the class your new items need is full. Enable the slab rebalancer/automove, or restart with tuned growth factors.

Q5. How does Memcached scale across servers without a cluster protocol? TRICKY

Clients hash keys onto servers with consistent hashing, so each client independently finds the right server; all clients must share the same server list and hash function.

Q6. Is Redis single-threaded a weakness? TRICKY

Rarely for caching: it's usually network-bound, commands are fast, and you scale out with more instances. It matters for CPU-heavy commands and for using a big multi-core box with one instance.

Next: SD-F-016 · Distributed Caching

Distributed Caching

spread the cache over many nodes, and grow it safely

Watch the 4-min video

In one line: a distributed cache partitions keys across nodes so memory and throughput scale; use consistent hashing with virtual nodes (or hash slots) so adding a node moves only about 1/N of keys, route in the client, a proxy or the servers, give each shard a replica, grow gradually with warm-up, and keep the database protected while hit ratios recover.

1 From three nodes to thirty

Your three cache nodes are full and the hit ratio is dropping. You need thirty. Add them the wrong way and almost every key moves, the cache goes cold, and the database gets crushed.

A **distributed cache** splits the key space across many nodes, so total memory and throughput grow with the node count. Each key lives on one node, chosen by a **partitioning function**. The questions are: which function, and who runs it?

2 Which function: modulo vs consistent hashing

Naive `hash(key) mod N` works until N changes: going from 3 nodes to 4 remaps about **three quarters** of all keys. Consistent hashing (lesson 5) fixes it: nodes and keys sit on a ring, each key belongs to the next node clockwise, and adding a node moves only about **1/N** of the keys. Virtual nodes keep the load even.

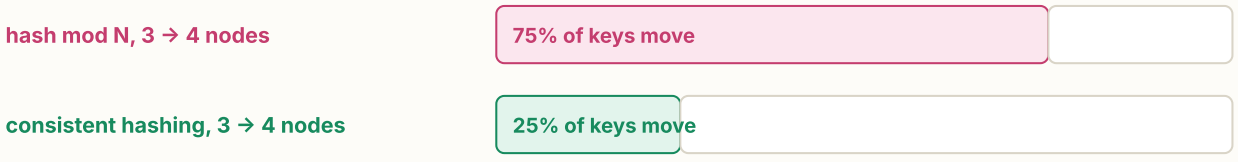


Figure 1. Growing from 3 to 4 nodes: modulo hashing moves about 75% of keys; consistent hashing about 25%, the new node's fair share.

3 Who routes: client, proxy or server

Client-side sharding: the cache client library knows the node list and hashes keys itself, like Memcached clients with ketama. Fast, no extra hop, but every client must share the same node list and algorithm. **Proxy-based**: clients talk to a proxy (twemproxy, Envoy's Redis proxy, mcrouter) that routes keys; clients stay simple at the cost of a hop. **Server-side**: Redis Cluster puts routing in the servers, with hash slots and MOVED redirects (lesson 28).

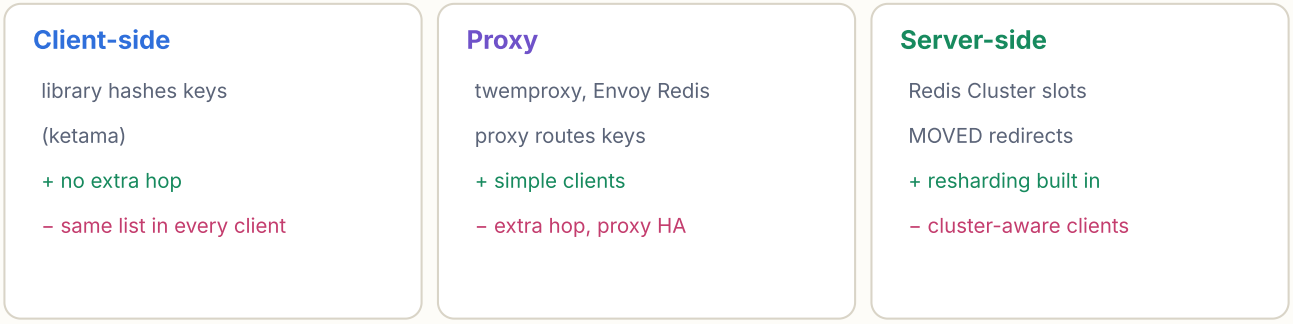


Figure 2. Three places to put the routing logic, with their trade-offs.

4 When a node fails

When a cache node dies, its share of keys becomes misses: with 10 nodes, about **10%** of keys. The database sees a sudden jump in load, so it needs headroom, or protection (rate limits, load shedding, request coalescing). **Replication** helps most: each shard has a replica that takes over with its data, so the hit ratio barely dips.

WORKED EXAMPLE: SCALING FROM 3 TO 30 NODES

Don't jump at once. With consistent hashing and virtual nodes, add nodes **gradually**, a few at a time: 3 → 6 → 10 → 15 → 20 → 25 → 30. Each step moves only a fraction of keys (3 → 6 moves half, but later steps move less: 25 → 30 moves about 17%), and the database absorbs the extra misses before the next step. Optionally, **pre-warm** new nodes by copying hot keys before shifting traffic to them. With Redis Cluster, migrate hash slots in small batches.

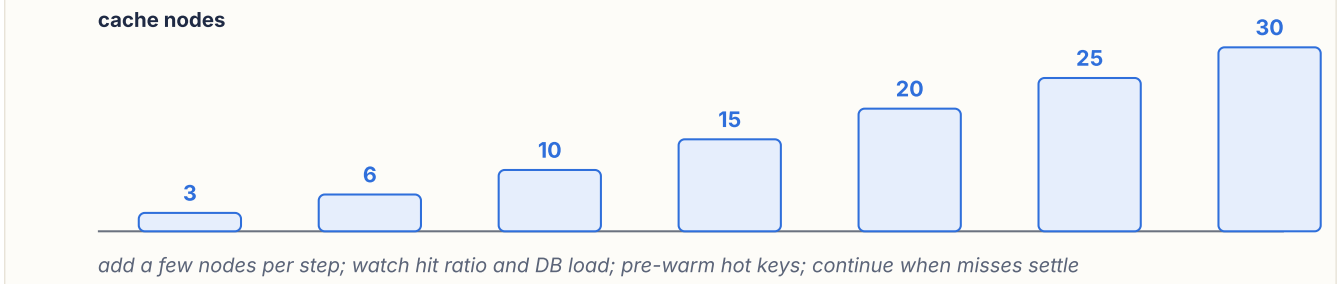


Figure 3. Grow in steps, letting the hit ratio recover between them.

5 Operating tips

- Watch the **hit ratio** and **database load** during every step.
- Keep TTLs **jittered**, so warmed keys don't all expire together (lesson 19).
- A bigger cluster means more connections from every app server: 200 pods × 30 nodes × a pool of 10 = 60,000 connections. Use pooling, multiplexing clients, or a proxy.
- Make sure every client agrees on the node list; a client with an old list reads and writes the wrong nodes.

COMMON MISTAKES

- Modulo sharding "for now", then a cold cache at the first resize.
- Doubling the cluster in one step during peak traffic.
- No replicas, so every node failure is a burst of database load.

6 Key terms

Partitioning function: maps a key to a node

Proxy sharding: a proxy picks the node (twemproxy, mcrouter)

Pre-warming: copying hot keys to new nodes before traffic

Client-side sharding: the library picks the node

Hash slots: Redis Cluster's 16,384 buckets

Shard replica: a copy of one node's data for failover

7 Interview questions

Q1. How do you add a cache node without a mass of misses? BASIC
Use consistent hashing with virtual nodes, or slot-based sharding, so only about 1/N of the keys move, add nodes gradually, pre-warm hot keys, and keep the database protected while hit ratios recover.

Q2. What happens when a node fails? BASIC
Its keys turn into misses until rebuilt, so use replicas for failover and give the database headroom.

Q3. Client-side sharding or a proxy for a polyglot company with 15 services? SCENARIO
A proxy (or Redis Cluster) avoids keeping 15 client libraries in sync on node lists and hashing; client-side is fine for one language and gives the lowest latency.

Q4. How many keys move when you go from 10 to 11 nodes with consistent hashing? SCENARIO
About 1/11, roughly 9%: the new node's share. With modulo, about 10/11, roughly 91%.

Q5. Two app versions use different node lists during a deploy. What breaks? TRICKY
They hash the same key to different nodes: one writes or invalidates on a node the other never reads, so stale data and extra misses appear. Roll out node-list changes atomically or via a proxy.

Q6. Why is connection count a scaling concern for distributed caches? TRICKY
Every app instance keeps connections to every node; it multiplies with pods and nodes and can exhaust file descriptors or memory on the cache servers. Pool, multiplex or use a proxy.

Next: SD-F-017 · CDN and HTTP Caching

CDN and HTTP Caching

the headers that decide who may cache what, for how long

Watch the 4-min video

In one line: Cache-Control decides caching: max-age for everyone, s-maxage for shared caches like CDNs, public or private (only the user's browser), no-store for sensitive data; ETags with If-None-Match turn re-downloads into cheap 304s, Vary and the CDN cache key decide what counts as the same response, and surrogate keys purge related responses at once.

1 Two problems, one cause

Your API's product list is the same for every visitor, yet every request travels to your origin servers across the world. Meanwhile, a user's private account page got cached at the CDN and shown to someone else. Both problems come down to one thing: **HTTP caching headers**. (Phase 2, lessons 19 and 28 introduced status codes, ETags and CDNs; here we go deeper.)

2 Cache-Control

- **max-age=N**: any cache may reuse the response for N seconds.
- **s-maxage=N**: overrides max-age for **shared** caches (CDNs, proxies), so the CDN can keep it longer than browsers.
- **public**: shared caches may store it (even with an Authorization header).
- **private**: only the user's own browser may cache it, never a shared cache.
- **no-cache**: may be stored, but must be revalidated with the server before each use.
- **no-store**: don't store it anywhere.

`public, max-age=60`	browser and CDN may keep it 60 s
`public, max-age=10, s-maxage=300`	browser 10 s, CDN / proxies 300 s
`private, max-age=60`	only the user's browser; never a CDN
`no-cache`	may store, but must revalidate before every use
`no-store`	don't store anywhere: banking, health pages

Figure 1. Common Cache-Control values and what each one allows.

What does private mean? It's for responses specific to one user, like an account page or a cart. The browser may cache it, but a CDN or corporate proxy must not, because it could serve one user's data to another. For anything sensitive, like banking pages, use **no-store** instead. And never let a CDN cache a response that depends on a cookie unless the cookie is part of the cache key.

3 Validation: ETags and 304

When a cached response expires, it doesn't have to be downloaded again. The server sends an **ETag**, a fingerprint of the content (a hash or a version). Later the client sends `If-None-Match` with that ETag. If nothing changed, the server replies **304 Not Modified** with no body, and the client keeps its copy. `Last-Modified` and `If-Modified-Since` work the same way with timestamps.

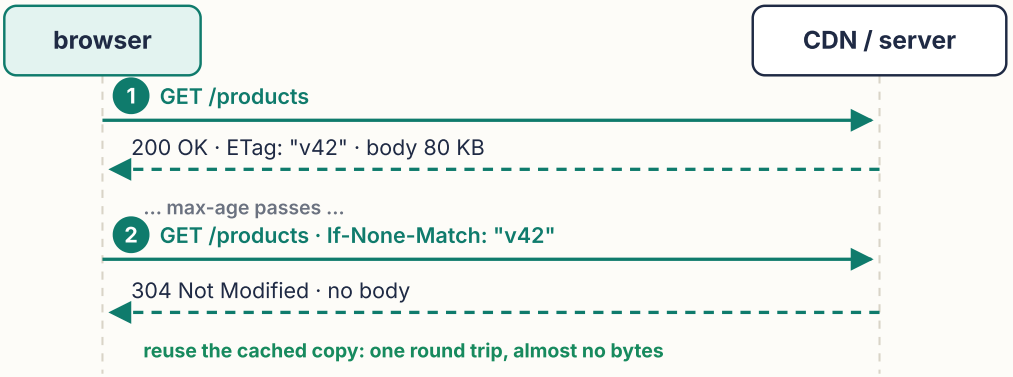


Figure 2. Revalidation with an ETag: an unchanged resource costs one small round trip instead of a full download.

4 Vary and stale-while-revalidate

Vary tells caches which request headers change the response, like `Accept-Encoding` or `Accept-Language`, so a gzip version isn't served to a client that can't read it. Use it carefully: `Vary: Cookie` or `Vary: User-Agent` explodes the number of cache variants and the hit ratio collapses. **stale-while-revalidate=N** lets a cache serve a slightly stale copy for N seconds while it fetches a fresh one in the background (lesson 24).

5 At the CDN: cache keys and purges

The CDN's **cache key** decides what counts as the same response: usually host, path and query string. Strip tracking parameters (`utm_*`, `gclid`) and sort the rest, or every marketing link becomes its own miss. **Surrogate keys** (cache tags) let you purge every response related to one product with a single API call, instead of guessing URLs (lesson 22).

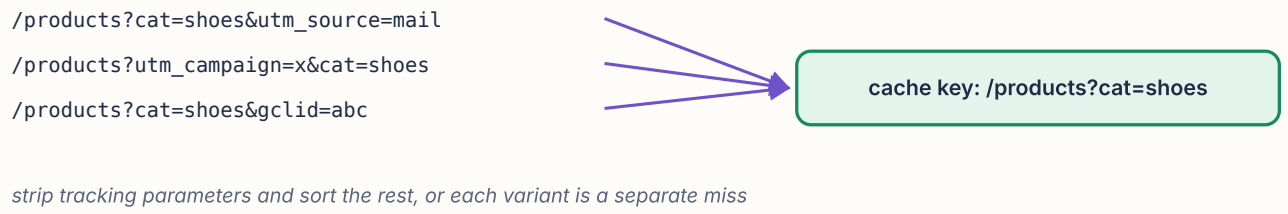


Figure 3. Normalising the cache key: three URLs, one cached response.

WORKED EXAMPLE: GET /PRODUCTS

A public endpoint returns the same list for everyone. It sends `Cache-Control: public, max-age=10, s-maxage=30` plus an ETag. The CDN serves it for 30 seconds from the edge, so the origin sees about **one request per edge location every 30 seconds**, instead of thousands. Browsers revalidate after 10 seconds and usually get a cheap 304.

COMMON MISTAKES

- `public` (or no header, with a CDN that caches by default) on user-specific pages: data leaks between users.
- Long max-age on HTML or API responses you'll need to change: you can't purge browsers. Use long max-age only for versioned assets (`app.3f9a.js`).
- `Vary: Cookie` on a CDN: almost zero hit ratio.
- Confusing no-cache (revalidate) with no-store (never store).

6 Key terms

max-age / s-maxage: freshness for all caches / for shared caches

no-store: never store the response

ETag: content fingerprint for revalidation

Vary: request headers that change the response

public / private: shared caches allowed / only the browser

no-cache: store, but revalidate before use

304 Not Modified: unchanged: reuse your copy

Cache key: what the CDN treats as the same response

7 Interview questions

Q1. What does Cache-Control: private mean? BASIC

Only the end user's browser may cache the response; shared caches like CDNs and proxies must not, because it's user-specific. Use no-store for sensitive data.

Q2. How do ETags work? BASIC

The server sends a content fingerprint; the client later sends If-None-Match with it, and if nothing changed the server replies 304 Not Modified without a body, so the cached copy is reused.

Q3. max-age vs s-maxage? BASIC

max-age applies to every cache; s-maxage overrides it for shared caches, so you can let a CDN keep a response longer than browsers, where you can't purge.

Q4. Cache a public product list at the CDN without stale prices for minutes. SCENARIO

Short s-maxage (e.g. 30 s) plus stale-while-revalidate, an ETag, surrogate-key purges on price changes, and a short browser max-age.

Q5. A user saw another user's account page. What happened and how do you fix it? SCENARIO

The response was cached by a shared cache: it lacked private/no-store, or the CDN ignored the session cookie. Send Cache-Control: private or no-store on personal pages, and configure the CDN to bypass cache when a session cookie is present.

Q6. Why can Vary destroy your CDN hit ratio? TRICKY

Each distinct value of a varied header becomes a separate cached variant; varying on Cookie or User-Agent creates nearly one variant per user or browser. Vary only on low-cardinality headers like Accept-Encoding.

Next: SD-F-018 · Multi-Level Caching

Multi-Level Caching

an in-process L1 in front of a shared L2

Watch the 4-min video

In one line: multi-level caching stacks a tiny, microsecond in-process L1 (near cache) in each pod in front of a shared L2 like Redis and the database; reads check L1, then L2, then the database and fill back; the hard part is invalidating hundreds of L1 copies, handled with short L1 TTLs and pub/sub broadcasts, with an explicit staleness bound.

1 Half a million Redis calls a second

Your Redis cache is fast: one millisecond per lookup. But your service does 50 lookups per request and runs at 10,000 requests per second. That's **500,000 Redis calls per second**, plus network time on every one: 50 ms of waiting per request if done one by one. What if the hottest data lived inside each process?

2 The levels

Multi-level caching stacks caches from fastest and smallest to slowest and largest. **L1** is an in-process cache inside each application instance: a dictionary in memory, lookups in microseconds, no network. **L2** is a shared distributed cache like Redis: milliseconds, bigger, shared by all instances. Behind them, the database; in front of everything, the CDN. Like CPU caches, each level catches what the faster one misses.

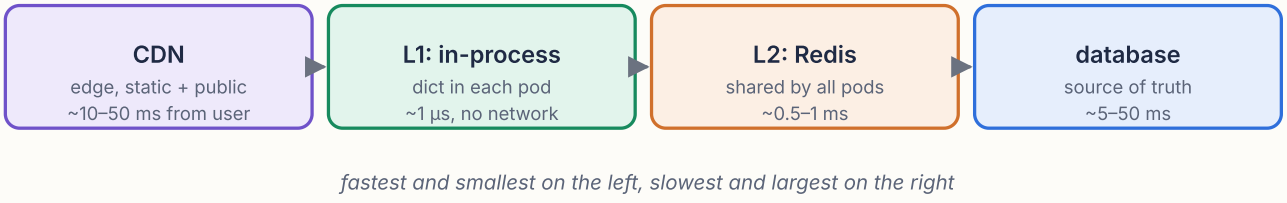


Figure 1. The cache levels, from the edge to the source of truth.

3 The read path

A read checks L1 first. On a miss, it checks L2. On another miss, it reads the database, then fills L2 and L1 on the way back. Most hot reads stop at L1. L1 is often called a **near cache**, because it sits right next to the code. In Python, `cachetools.TTLCache` gives you one; in Java, Caffeine.



Figure 2. A two-level read: L1, then L2, then the database, filling back on the way.

4 The hard part: invalidating every L1

When data changes you can delete it from Redis, but you can't easily reach the L1 caches inside 200 pods. Common answers:

- **Very short L1 TTLs**, a few seconds, so L1 copies expire on their own.
- **Broadcast invalidations** with Redis pub/sub or a message bus: every pod subscribes and deletes the key from its local cache.
- **Redis client-side caching** (`CLIENT TRACKING`, Redis 6+): the server remembers which keys each client read and pushes invalidations automatically.

WORKED EXAMPLE: FEATURE FLAGS

Flags are read on almost every request. Each pod keeps them in an in-process TTL cache for **5 seconds**, with Redis as L2 holding them for **5 minutes**. When someone flips a flag, the admin service updates the database, deletes the Redis key and publishes a `flag-changed` event. Pods drop their L1 copy within milliseconds; a pod that misses the event (pub/sub is fire-and-forget) still picks up the change within 5 seconds, because of the TTL.

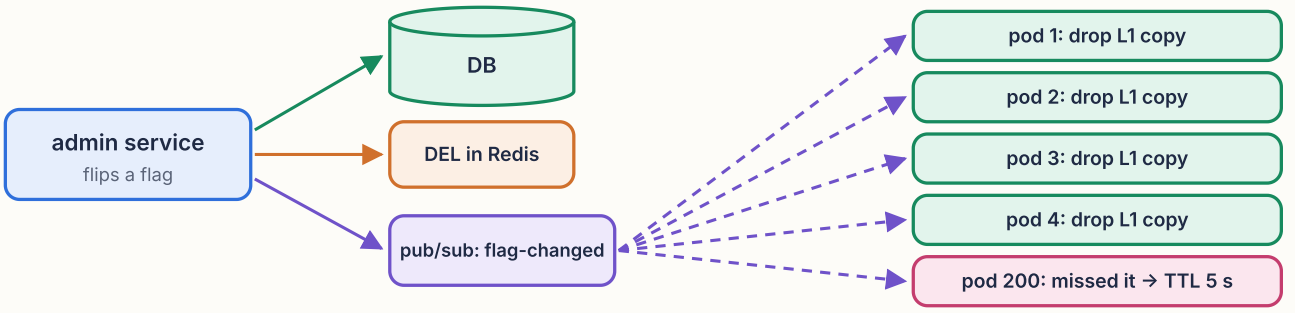


Figure 3. Invalidating L1 across pods: delete in Redis, broadcast an event, and rely on the short L1 TTL for any pod that missed it.

5 Risks of in-process caches across many pods

- **Inconsistency**: different pods can hold different versions, so two requests from the same user might see different values.
- **Memory multiplies**: every pod stores its own copy; 200 pods × 500 MB = 100 GB.
- **Cold starts**: new or restarted pods begin empty and add load on L2.
- **Invalidation fan-out** to hundreds of pods adds moving parts.

So use L1 for **small, hot, read-mostly** data that tolerates a few seconds of staleness: configuration, feature flags, reference data, hot keys (lesson 26). Keep user-specific or rapidly changing data in L2 only, and always state the staleness bound.

COMMON MISTAKES

- An L1 with no TTL and no size limit: stale forever and a memory leak.
- Caching per-user data in L1, so memory explodes and users see inconsistent results across pods.
- Relying only on pub/sub for invalidation: messages can be missed.

6 Key terms

L1 / near cache: in-process cache in each instance

Fill back: populate faster levels on the way out

Client-side caching: Redis tracks keys and pushes invalidations

L2: shared distributed cache (Redis)

Invalidation broadcast: pub/sub message telling every pod to drop a key

Staleness bound: the maximum age you promise

7 Interview questions

Q1. What are the risks of in-process caches across many pods? BASIC
Pods can serve different, inconsistent versions, memory is duplicated per pod, new pods start cold, and invalidating every pod's copy is hard.

Q2. How do you keep levels in sync? BASIC
Short L1 TTLs, invalidation broadcasts through pub/sub, and deleting from L2 on writes, accepting a small, explicit staleness window.

Q3. Each request reads 50 config values from Redis. How do you cut that? SCENARIO
Put an L1 TTL cache (a few seconds) in each process for config, or load the whole config periodically into memory; also batch remaining Redis reads with MGET or pipelining.

Q4. Which data belongs in L1? SCENARIO
Small, hot, read-mostly data that tolerates seconds of staleness: feature flags, configuration, reference tables, and the hottest keys. Not carts, balances or per-user data.

Q5. Why isn't pub/sub alone enough to invalidate L1 caches? TRICKY
Redis pub/sub is fire-and-forget: a pod that is disconnected or restarting misses the message and keeps the stale copy. A short TTL bounds that; streams or versioned keys make it more reliable.

Q6. How does Redis client-side caching work? TRICKY
With CLIENT TRACKING the server remembers which keys a client has read and sends it an invalidation message when one of them changes, so the client's local copy can be dropped.

Next: SD-F-019 · Caching in Python Applications

Caching in Python Applications

lru_cache, cachetools, Redis from async code, and good keys

Watch the 4-min video

In one line: functools.lru_cache suits pure functions in one process but has no TTL and is per worker; cachetools adds TTL and size-bounded caches; with many workers and pods, shared or changing data belongs in Redis via redis.asyncio; choose serialization deliberately, and build keys from a namespace, a version and every input that changes the result.

1 Fast in the test, broken in production

You add `@functools.lru_cache` to a slow function and it gets a hundred times faster in your test. Then you deploy with eight Gunicorn workers on four pods, and the hit ratio is terrible, memory grows, and users see stale data. Python caching has a few traps.

2 functools.lru_cache

It wraps a function and remembers results by arguments, with a `maxsize` (and `@functools.cache` is the unbounded version). Perfect for **pure functions** with hashable arguments: parsing a config, computing a small lookup. But it has **no TTL**: cached values never expire on their own. Use `f.cache_clear()` to reset, and `f.cache_info()` to see hits and misses. On methods it keeps `self` alive, so prefer module-level functions.

3 cachetools, and async code

The **cachetools** library adds control: `TTLCache(maxsize=10_000, ttl=60)` evicts entries after a time, `LRUCache` and `LFUCache` bound size, and the `@cached(cache)` decorator wraps functions with any of them (pass a lock if threads share it).

For **async** code, remember that a plain `lru_cache` on an `async def` function caches the **coroutine object**, not the result: the second await fails. Use an async-aware cache (for example `async-lru` or `aiocache`), or cache the awaited result yourself.

4 The multi-worker trap

Each worker **process** has its own separate cache. With 8 workers on 4 pods that's **32 independent caches**. Each one warms up separately, so hit ratios are low; memory is duplicated 32 times; and invalidation only clears one worker's cache, so the other 31 keep serving stale values. For shared or changing data, use Redis.



8 Gunicorn workers × 4 pods = 32 separate caches: each warms alone, memory ×32, and cache_clear() in one worker leaves 31 still serving the old value

Figure 1. In-process caches are per worker process, not per service.

5 Redis from async code, and serialization

From FastAPI or other async code, use **redis.asyncio** (part of redis-py), so cache calls don't block the event loop, and reuse one client with its connection pool. Then think about **serialization**: values must become bytes. For big objects, serialization can cost more time than the Redis call itself.

format	speed / size	portable?	watch out
JSON	slower, larger	any language	no datetimes/bytes natively
msgpack	fast, compact	many languages	a schema is still on you
pickle	fast, any object	Python only	unsafe with untrusted data

Figure 2. Three common serializations and their trade-offs.

6 What goes into a cache key

Everything that changes the result, and nothing else.

- A **namespace** for the service and data type: `search:`.
- A **schema version**: `v2:`. Bump it when the format changes, and old entries are simply ignored until they expire.
- The **identifying inputs**: `tenant42:` plus a hash of the normalised query.
- The **user or tenant** whenever results differ per user, or you'll leak data between them.
- Nothing irrelevant, like a timestamp or a request ID, which makes every key unique and the hit ratio zero.

Example: `search:v2:tenant42:9f86d081…`.

WORKED EXAMPLE: AN EMBEDDING CACHE

A FastAPI service computes text embeddings with a model call that takes 200 ms. Before calling the model it normalises and hashes the text and looks up `embed:v1:<model>:<sha256>` in Redis with redis.asyncio. On a hit it deserialises the vector from msgpack in microseconds; on a miss it calls the model and stores the result with a one-day TTL. Repeat texts now cost about a millisecond instead of 200, and the model name in the key means switching models never returns old vectors.

```
import hashlib, msgpack, redis.asyncio as redis
r = redis.Redis()

async def embed(text: str, model='text-emb-3'):
    norm = ' '.join(text.lower().split())
    key = f'embed:v1:{model}:' + hashlib.sha256(norm.encode()).hexdigest()
    if (hit := await r.get(key)) is not None:
        return msgpack.unpackb(hit)          # ~1 ms
    vec = await call_model(norm, model)      # ~200 ms
    await r.set(key, msgpack.packb(vec), ex=86_400)
    return vec
```

Figure 3. The embedding cache in a few lines: namespace, version, model and a content hash in the key.

COMMON MISTAKES

- `lru_cache` for data that changes, with no way to expire it across workers.
- `lru_cache` on an async function.
- Unpickling data from a cache other systems can write to: remote code execution.
- Keys without the tenant, so one customer sees another's results.

7 Key terms

lru_cache: per-process memoisation with LRU, no TTL

redis.asyncio: non-blocking Redis client in redis-py

Key namespace: prefix naming the service and data type

cachetools: TTLCache, LRUCache, LFUCache + @cached

Serialization: turning objects into bytes

Key version: bump to ignore old formats

8 Interview questions

Q1. What should a cache key contain? BASIC

A namespace, a version, and every input that changes the result, including the tenant or user when results differ, but nothing irrelevant like timestamps.

Q2. What are the pitfalls of lru_cache in multi-worker deployments? BASIC

Every process has its own cache, so hit ratios drop, memory is duplicated, there's no TTL, and invalidation only reaches one worker, so shared or changing data belongs in Redis.

Q3. Why not put lru_cache on an async function? BASIC

It caches the coroutine object; awaiting it a second time raises an error. Use an async-aware cache or cache the awaited result.

Q4. Design caching for an LLM-backed endpoint with repeated prompts. SCENARIO

Normalise the prompt, hash it with the model and parameters into a versioned key scoped to the tenant, store the response in Redis with a TTL, and coalesce concurrent identical requests.

Q5. When is pickle acceptable for cache values? TRICKY

Only when the cache is private to trusted Python services, versions are compatible, and nobody else can write to it; otherwise use JSON or msgpack.

Q6. Your cache hit ratio is 0% even though the same data is requested. What do you check? TRICKY

The keys: something varying (a timestamp, request ID, unsorted parameters, different whitespace) is in the key, or each worker has its own cold in-process cache.

Next: SD-J-001 · Stateless Service Design

Stateless Service Design

any instance, any request, nothing lost when it dies

Watch the 4-min video

In one line: a stateless service keeps no client or request state in its own memory or disk between requests; sessions, files, caches, data and background work live in external stores (Redis, S3, databases, queues), so any instance can serve any request and instances are disposable, which makes scaling, deploys and failures simple and safe.

1 Gone with the pod

A user uploads a profile photo, and it's saved to the pod's local disk. Ten minutes later Kubernetes reschedules that pod, and the photo is gone. Another user's session lived in memory on a pod that just crashed, and they're logged out. The fix for both is the same: make the service **stateless**.

2 What stateless means

A stateless service keeps **no client or request state** in its own memory or disk **between requests**. Every request carries, or can look up, everything needed to handle it (a token, an ID). So any instance can serve any request, and instances can be added, removed or replaced at any time without losing anything.

Statelessness doesn't mean there's no state. It means the state lives **somewhere else**: sessions in Redis or a signed token, uploaded files in object storage like S3, caches in a shared cache, durable data in a database, background work in a queue. The service becomes a disposable process that reads and writes external state.

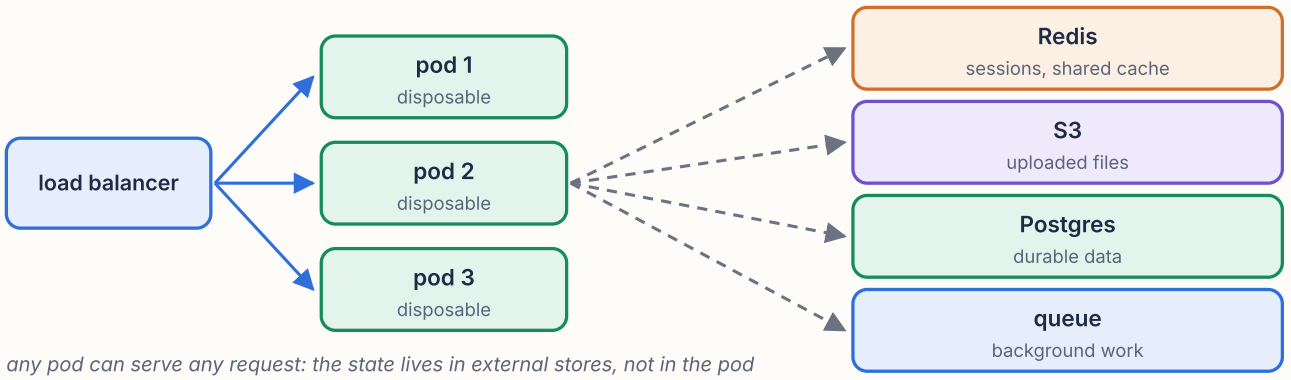


Figure 1. A stateless tier: identical, disposable pods in front of external stores.

3 The twelve-factor app

The **twelve-factor app** methodology captures this: processes are stateless and share nothing; configuration comes from the environment; backing services (databases, caches, queues) are attached resources; and processes are disposable: they start fast and shut down gracefully, finishing in-flight requests when they receive SIGTERM (lesson 7).



Figure 2. Four twelve-factor ideas that make a service stateless and disposable.

4 Idempotent handlers

Stateless handlers should be **idempotent** where possible. A client might retry after a timeout, and the retry might land on a different instance. With an **idempotency key** stored in the database (Phase 2, API lesson 39), the second attempt returns the original result instead of charging a card twice. Retries become safe no matter which instance handles them.

WORKED EXAMPLE: A FASTAPI APP ON MANY PODS

Originally, uploads were written to `/tmp` on the pod. Now the API generates a **presigned S3 URL**, the client uploads directly to S3, and the pod only stores the object key in Postgres. Sessions move to Redis, and local caches become short-lived optimisations only. The pods can now scale from 3 to 30, and be killed at any moment, with nothing lost. Bonus: large uploads no longer pass through the API at all.

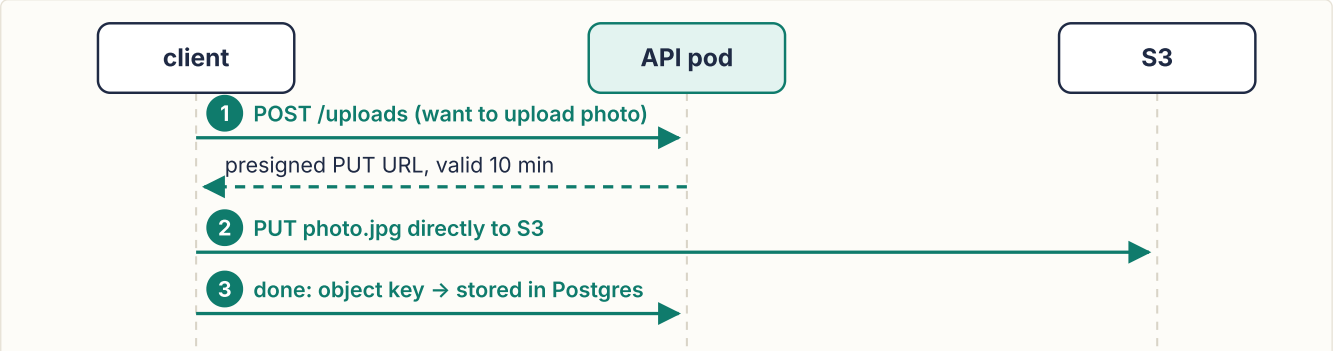


Figure 3. Presigned uploads: the API hands out a short-lived URL, the client sends the file straight to S3, the API stores only the key.

5 The payoff

- **Horizontal scaling is trivial**: a new instance is identical to the others (Phase 1, lesson 7).
- **Autoscaling, rolling deploys and spot instances** are safe (lessons 13 and 14).
- **Any load-balancing algorithm** works, with no sticky sessions (lesson 6).
- **Failures are cheap**: a dead instance loses nothing that matters.

COMMON MISTAKES

- Writing uploads, generated reports or SQLite files to the container's disk.
- In-memory sessions or rate-limit counters, so behaviour changes with which pod you hit.
- Scheduled jobs (cron) inside every web pod, so they run N times.
- Ignoring SIGTERM, so every deploy drops in-flight requests.

6 Key terms

Stateless service: no client state kept between requests

Disposable process: starts fast, stops gracefully

Presigned URL: short-lived URL to upload or download directly

External state: Redis, S3, databases, queues

Twelve-factor app: methodology for cloud-friendly services

Idempotency key: makes a retried write return the first result

7 Interview questions

Q1. What makes a service stateless? BASIC
It keeps no client state locally between requests: sessions, files, caches and data live in external stores like Redis, S3 and databases, so any instance can handle any request and instances are disposable.

Q2. Why does it matter? BASIC
It makes horizontal scaling, autoscaling, rolling deploys and failure recovery simple and safe.

Q3. Users lose uploaded files after deploys. What do you change? SCENARIO
Stop writing to local disk: upload to object storage (ideally directly with presigned URLs), store only the object key in the database, and serve files from S3 or a CDN.

Q4. How do you make a POST /payments handler safe to retry on any instance? SCENARIO
Require an idempotency key, store it with the result in the database inside the same transaction as the payment, and return the stored result on repeats.

Q5. Is a service with a local in-memory cache still stateless? TRICKY
Yes, if the cache is only an optimisation: losing it costs a few misses, never correctness, and every instance can rebuild it from the shared stores.

Q6. What does 'disposable' require in practice? TRICKY
Fast startup, readiness checks, handling SIGTERM by failing readiness and draining in-flight work, and no state that would be lost if the process is killed at any moment.

Next: SD-J-002 · Scaling Stateful Services

Scaling Stateful Services

partition the state, give each piece one owner, route to it

Watch the 4-min video

In one line: to scale a service that must keep state in memory, partition the state by key, give each partition a single owner (consistent hashing or a coordinator), route requests to the owner, and handle rebalancing carefully by rebuilding, transferring or reconnecting, while keeping the state recoverable from a durable log.

1 Some state must stay in memory

Not everything can be stateless. A chat server holds a million open WebSocket connections. A game server keeps the live state of each match in memory. A stream processor keeps running counts per user. That state must live in memory, close to the code, for speed. How do you scale it across many machines?

2 Ownership

Split the state into **partitions** by a key: user ID, room ID, match ID. Each partition is **owned by exactly one node** at a time. Requests for a key must reach the node that owns it. With single ownership there are no conflicting copies, and the owner can keep everything in memory and process its keys one at a time, without locks.

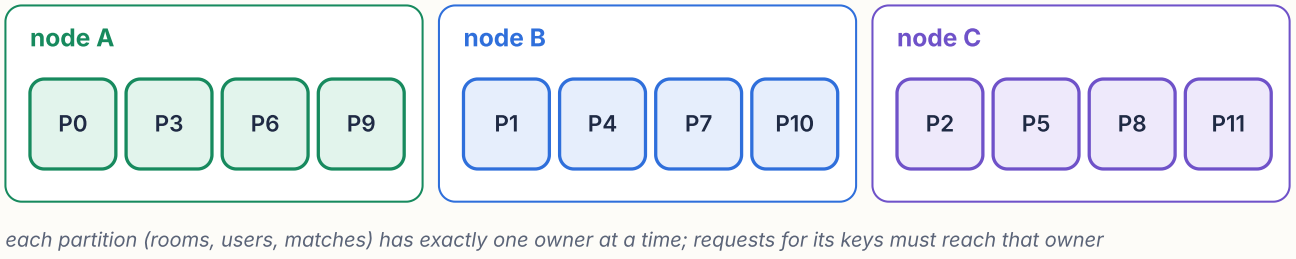


Figure 1. Twelve partitions spread over three nodes, each with exactly one owner.

3 Deciding who owns what, and routing

Consistent hashing (lesson 5) maps keys to nodes, so adding a node moves only a small share. Or a **coordinator** (ZooKeeper, etcd, a Kafka consumer group) assigns partitions to nodes and keeps a shared map. Either way, every router must agree on the current owner.

Then requests must reach the owner: a hash-aware load balancer (Envoy ring hash, Maglev), a routing layer that looks up the partition map, or any node that accepts and forwards.

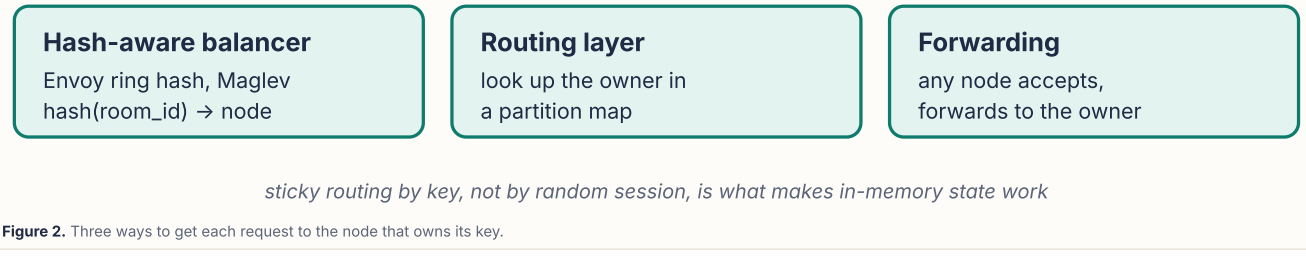


Figure 2. Three ways to get each request to the node that owns its key.

4 The hard part: rebalancing

When a node joins, leaves or crashes, partitions move, and the new owner needs the state. Options:

- **Rebuild** it from a durable source: a database, or by replaying a log (Kafka Streams restores state from changelog topics).
- **Transfer** it from the old owner before switching (graceful moves).
- **Reconnect**: clients re-establish it (WebSocket clients reconnect and resubscribe).

During the move, **pause or buffer** requests for that partition, so two nodes never both think they own it. Leases or **fencing tokens** (an epoch number that increases with each new owner, checked by storage) stop an old owner that doesn't know it was replaced.

WORKED EXAMPLE: A WEBSOCKET CHAT GATEWAY

Each gateway node owns the connections of the users connected to it. A registry in Redis maps `user:bob → gateway 2` (with a TTL refreshed by heartbeats). When a message arrives for Bob, the chat service looks up the owning node and sends the message there, which pushes it down the open socket. If a node dies, its users reconnect, land on other nodes, and the registry updates; messages sent meanwhile are kept in the chat store and delivered on reconnect.

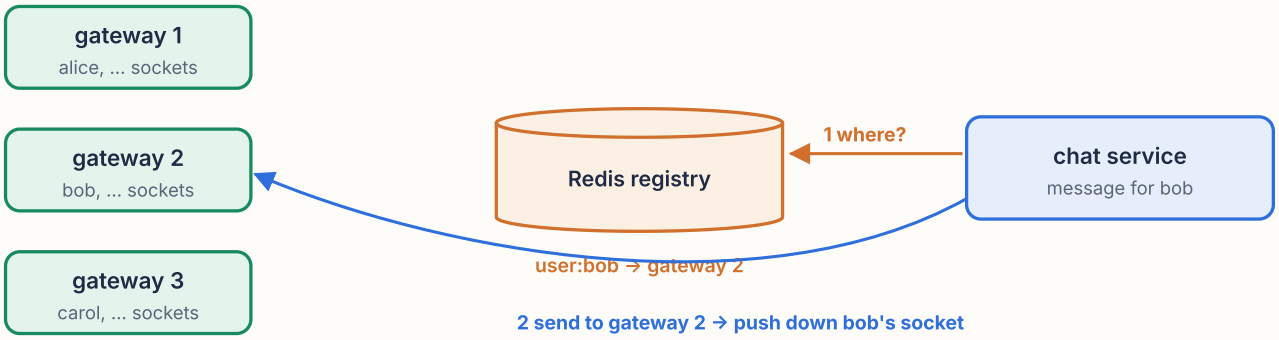


Figure 3. Routing to the owner: a registry says which gateway holds Bob's socket.

5 Keep the state recoverable

- Write important changes to a **durable log or database**, and treat in-memory state as a fast cache of it.
- **Snapshot** periodically, so recovery doesn't replay everything.
- Keep partitions **small and numerous** (hundreds or thousands), so rebalancing moves many small pieces quickly instead of a few giant ones.

COMMON MISTAKES

- Two owners after a network partition, both writing (no fencing).
- As many partitions as nodes, so every scale event moves huge chunks.
- State that exists only in memory, so a crash loses it.

6 Key terms

Partition: a slice of the state, chosen by key

Partition map: which node owns which partition

Fencing token: increasing epoch that rejects old owners

Single owner: only one node handles a partition at a time

Rebalancing: moving partitions when nodes change

Changelog: durable log used to rebuild state

7 Interview questions

Q1. How would you scale a service that must keep in-memory state? BASIC
Partition the state by key, give each partition a single owner using consistent hashing or a coordinator, route requests to the owner, and handle rebalancing by rebuilding, transferring or reconnecting, while keeping the state recoverable from a durable log.

Q2. What's the hardest part? BASIC
Moving ownership safely when nodes join or fail, without two owners and without losing state.

Q3. Design scaling for multiplayer game matches held in memory. SCENARIO
Partition by match ID; a matchmaker assigns each match to one game server and records it; clients connect to that server; state changes are checkpointed so a crashed match can resume elsewhere.

Q4. How do you deliver a chat message to a user connected to one of 50 gateways? SCENARIO
Keep a registry of user → gateway (refreshed by heartbeats), look it up and forward to that gateway; or publish to a per-gateway channel; store messages durably for offline or reconnecting users.

Q5. How do you stop a paused old owner from corrupting state after failover? TRICKY
Use a fencing token: each new owner gets a higher epoch from the coordinator, and storage rejects writes carrying an older epoch.

Q6. Why many small partitions rather than one per node? TRICKY
Moving a partition is the unit of rebalancing; small partitions spread evenly, move quickly and in parallel, and let you add nodes without re-hashing everything.

Next: SD-J-003 · Database Bottlenecks

Database Bottlenecks

diagnose what's saturated, then climb the scaling ladder

Watch the 4-min video

In one line: when the database is the bottleneck, first find what is saturated (connections, CPU, disk IOPS or locks), watch for hot rows, then climb the ladder in order: optimise queries, add indexes, pool connections, cache hot reads, add read replicas, partition tables, and only then shard.

1 Adding servers stops helping

Your app servers scale out easily: add more, and throughput goes up. Then one day adding servers makes nothing faster. Latency climbs, and every graph points to the same place: the database. It's the most common bottleneck in system design.

2 What runs out



Figure 1. The four things a database runs out of.

- **Connections**: each Postgres connection is a process using several MB; thousands of them slow everything (Phase 2, lesson 82 covers pooling).
- **CPU**: expensive queries, sorts, hash joins, sequential scans.
- **Disk IOPS**: random reads that miss the buffer cache, and writes to the WAL.
- **Lock contention**: transactions waiting on the same rows.

Look at `pg_stat_activity` (what's running and waiting), `pg_stat_statements` (which queries cost the most total time), CPU and IOPS graphs, and lock waits.

3 The sneaky one: the hot row

Every request updates the same row, like a global counter. Each update must lock the row, so updates run **one after another**, no matter how big the database is. Throughput is capped by how fast one row can be locked, changed and committed: maybe a few hundred to a few thousand per second.

WORKED EXAMPLE: THE PAGE-VIEWS COUNTER

A `page_views` row is updated on every request. At 5,000 requests per second, transactions queue behind its lock and latency explodes. Fixes:

- **Sharded counters**: 64 rows (`shard = random 0..63`), increment one at random, `SUM` on read. Contention drops 64 times.
- **Buffer in Redis**: `INCR`, and flush to the database every few seconds (write-behind, lesson 18).
- **Events**: send view events to a queue and aggregate them in batches.

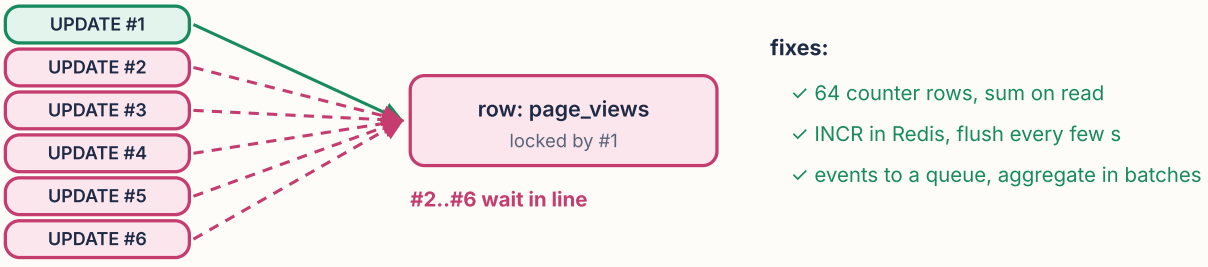


Figure 2. A hot row serialises every writer; spreading or batching the writes removes the queue.

4 The scaling ladder

Start with the cheapest steps:

1. **Optimise queries**: `EXPLAIN ANALYZE`, fix N+1 queries, select only needed columns, paginate with cursors.
2. **Add the right indexes** (composite, covering, partial: Phase 2, database lessons).
3. **Connection pooler** such as PgBouncer, so thousands of app connections share a few dozen database connections.
4. **Cache hot reads** in Redis to cut read load (lessons 15 to 33).
5. **Read replicas**: route read-only queries to them, accepting a little replication lag.
6. **Partition big tables** by time or key, so queries and maintenance touch less data.
7. **Shard** across several databases: scales writes, but costs cross-shard queries, transactions and operational complexity.

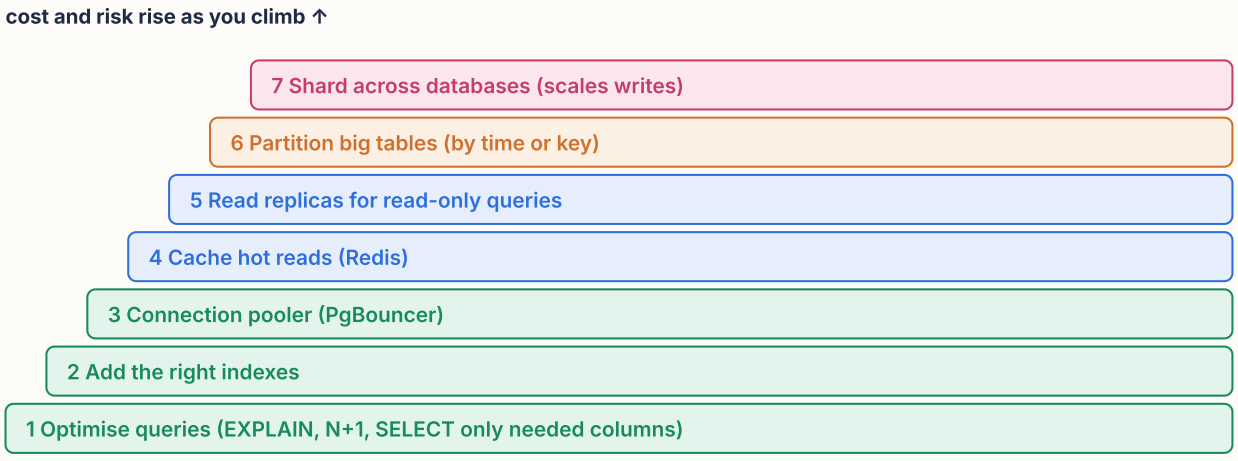


Figure 3. The database scaling ladder: climb from the bottom, measuring after each step.

Climb in order. Each step is cheaper and less risky than the next. Many teams jump to sharding when a missing index or a connection pooler would have solved the problem. And measure after each step: the bottleneck may move somewhere else.

COMMON MISTAKES

- Scaling the database up before looking at the top queries in pg_stat_statements.
- Adding read replicas when the problem is writes or locks.
- Autoscaling app servers that each open 20 connections, until the database hits max_connections.
- Sharding first, and spending a year on cross-shard queries.

5 Key terms

Saturation: a resource at its limit, so work queues

Sharded counter: a counter split over many rows

Connection pooler: many app connections → few DB connections

Hot row: one row updated by most requests

pg_stat_statements: Postgres query cost statistics

Read replica: a copy that serves read-only queries

6 Interview questions

Q1. What is your step-by-step approach when the database is the bottleneck? BASIC
First find what's saturated: connections, CPU, IOPS or locks. Then climb the ladder: optimise queries, add indexes, pool connections, cache hot reads, add read replicas, partition tables, and only then shard.

Q2. How do you fix a hot row? BASIC
Split it into sharded counters, buffer increments in Redis, or aggregate events in batches.

Q3. CPU is low but queries are slow and p99 is terrible. What do you suspect? SCENARIO
Lock contention (hot rows, long transactions) or I/O waits; check pg_stat_activity wait events and lock waits, and look for long-running transactions holding locks.

Q4. Reads are 95% of traffic and the primary is at 90% CPU. Next steps? SCENARIO
Find and optimise the top read queries and indexes, then cache hot reads in Redis, then route remaining read-only queries to replicas, keeping read-your-writes paths on the primary.

Q5. Why can adding app servers make the database slower? TRICKY
More servers mean more connections and more concurrent queries; past a point, context switching, memory per connection and lock contention reduce database throughput. A pooler caps concurrency.

Q6. When is sharding actually the right step? TRICKY
When writes or data size exceed what one well-tuned primary (with partitioning) can handle, and you have a shard key that keeps most queries and transactions on one shard.

Next: SD-J-004 · Asynchronous Processing backendroadmap.com

Asynchronous Processing

move slow, unreliable work off the request path

Watch the 4-min video

In one line: the request handler does only what the user needs now and puts everything else on a queue for background workers; make slow, unreliable, retryable or batchable work asynchronous, report long tasks with 202 Accepted plus a status URL, and make tasks idempotent with retries, backoff, a dead-letter queue and monitoring.

1 Eight seconds to sign up

A user clicks sign up. Your API creates the account, sends a welcome email, resizes their profile photo into five sizes and syncs them to the CRM. Eight seconds later the page finally responds, if the email provider didn't time out first. Most of that work didn't need to happen while the user waited.

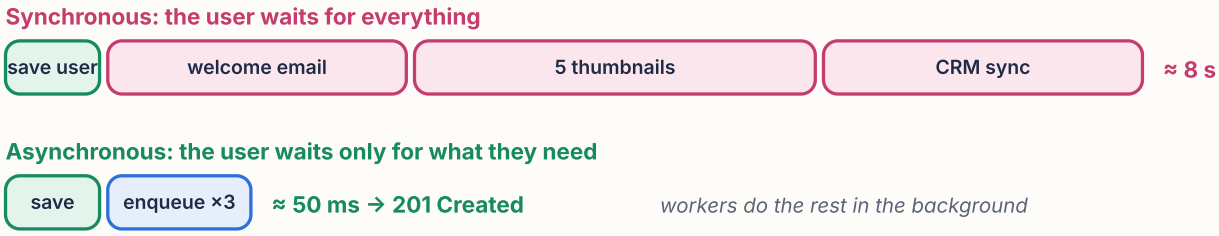


Figure 1. The same sign-up done synchronously and asynchronously.

2 The pattern

The request handler does only what the user needs **right now**: validate input, save the account. Everything else becomes a **task**, a message put onto a **queue**. Separate **worker** processes pull tasks from the queue and do the slow work in the background. The API responds in milliseconds, and a slow or failing email provider no longer breaks sign-up.

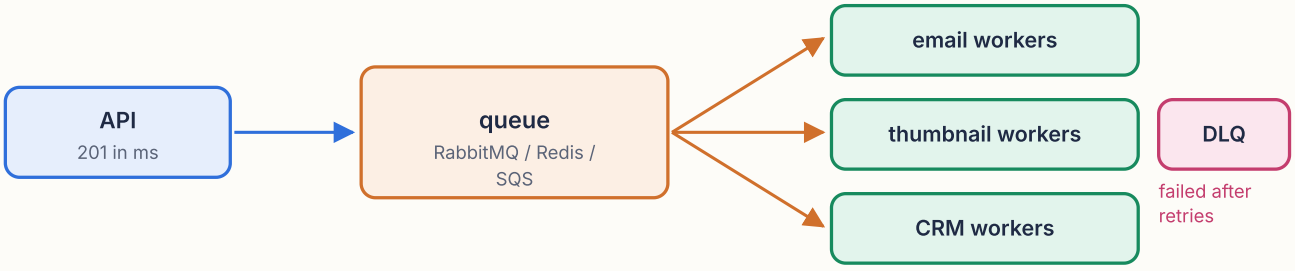


Figure 2. API, queue, workers: the queue absorbs bursts, workers scale independently, and repeatedly failing tasks go to a dead-letter queue.

3 What should be asynchronous?

- **Slow** work: video encoding, reports, AI model calls.
- **Unreliable** work: third-party APIs that can fail or time out.
- **Retryable** work: emails, webhooks, notifications.
- **Batchable** work: analytics, search indexing, bulk updates.

Keep work synchronous when the user needs the result immediately to continue: the payment authorisation at checkout, the login check.

4 Reporting status: 202 Accepted

For long tasks the user is waiting on, the API returns **202 Accepted** with a task ID and a status URL (Phase 2, API lesson 51). The client polls that URL, or is notified when the task finishes through a WebSocket, server-sent events, a push notification or an email. Task status lives in a database or Redis: *pending*, *running*, *succeeded* with a result link, or *failed* with a reason.

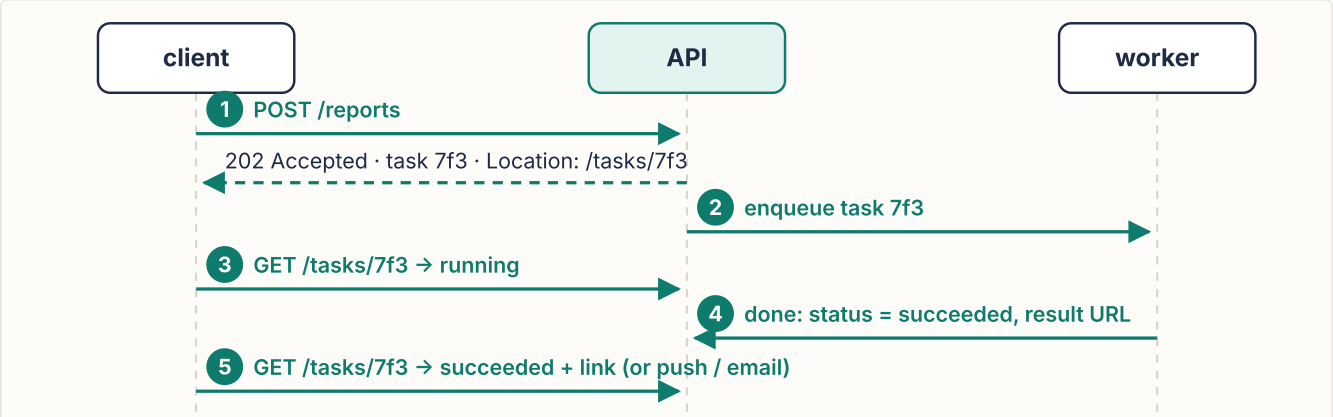


Figure 3. The 202 pattern: accept, return a task ID at once, and let the client poll or be notified.

5 Tools

In Python, **Celery** is the classic task queue (brokers RabbitMQ or Redis, retries, scheduling with beat, result backends). **RQ** is a simpler option built on Redis; **Arq** is designed for asyncio. In the cloud, managed queues like **SQS** or **Azure Service Bus** with serverless or container workers do the same job.

WORKED EXAMPLE: SIGN-UP, ASYNCHRONOUSLY

On sign-up the API saves the user, enqueues `send_welcome_email` and `generate_thumbnails`, and returns **201** in about 50 ms. Email workers call the provider with retries and exponential backoff (1 s, 2 s, 4 s ... up to an hour). Thumbnail workers resize the uploaded image into five sizes and store them in S3. The user sees their new account instantly, and thumbnails appear a few seconds later.

6 Background work needs care

- **Idempotent tasks**: a task may run more than once after a crash or retry (at-least-once delivery), so a second run must do no harm, e.g. check "email already sent for user 42".
- **Retries with backoff** and jitter, and a **dead-letter queue** for tasks that keep failing, so they don't block the queue and someone can inspect them.
- **Monitor** queue depth and the age of the oldest task, so you notice when workers fall behind.
- **Autoscale workers** on queue length (KEDA, lesson 14).
- Enqueue only **after** the database commit (or use a transactional outbox), or a worker may look for a user that doesn't exist yet.

COMMON MISTAKES

- Putting large payloads (the image itself) in the message instead of a reference (the S3 key).
- Non-idempotent tasks that send two welcome emails after a retry.
- No visibility into the queue, so a backlog of 2 million tasks is found by a customer.

7 Key terms

Task / job: a unit of background work, as a message

Worker: process that executes tasks

Dead-letter queue: where repeatedly failing tasks go

Transactional outbox: write the event in the same DB transaction, publish after

Queue / broker: holds tasks until a worker takes them

202 Accepted: request accepted, processing later

At-least-once: a task may run more than once

8 Interview questions

Q1. What work should be done asynchronously? BASIC

Anything slow, unreliable, retryable or batchable that the user doesn't need immediately, like emails, thumbnails, reports and third-party calls.

Q2. How do you report task status? BASIC

Return 202 Accepted with a task ID and a status URL, store status in a database or Redis, and let the client poll it or notify them through WebSockets, server-sent events, push or email.

Q3. Design PDF report generation that takes 2 minutes. SCENARIO

POST returns 202 with a task ID; a worker builds the PDF, stores it in S3 and updates the task to succeeded with a presigned link; the UI polls or gets an SSE event; tasks are idempotent and retried with backoff.

Q4. Workers fall behind every Monday morning. What do you do? SCENARIO

Autoscale workers on queue depth and oldest-task age, separate queues by priority so urgent tasks aren't stuck behind bulk ones, and batch where possible.

Q5. Why must background tasks be idempotent? TRICKY

Queues deliver at least once: a worker can crash after doing the work but before acknowledging, so the task runs again. Idempotency (checks, keys, upserts) makes repeats harmless.

Q6. A worker can't find the user it was asked to email. Why? TRICKY

The task was enqueued before the database transaction committed. Enqueue after commit, or use a transactional outbox so the event is published only once the data exists.

Next: SD-J-005 · Python Concurrency Models

Python Concurrency Models

threads, processes, asyncio and the GIL

Watch the 4-min video

In one line: match the tool to the work: asyncio for many I/O waits, threads for blocking I/O libraries, processes for CPU-bound Python (the GIL lets one thread run Python bytecode at a time per process), and task queues for heavy jobs; one blocking call freezes an event loop, so offload CPU work to a process pool, and run several worker processes per server.

1 One endpoint stalls them all

Your FastAPI service handles a thousand requests per second for database lookups. Then you add PDF parsing to one endpoint, and the whole service grinds to a halt, even the fast endpoints. Same code, same server. The problem is the wrong concurrency model.

2 First: what kind of work?

I/O-bound work spends most of its time **waiting**: for the network, the database, a disk. **CPU-bound** work spends its time **computing**: parsing, compressing, resizing images, running models. The right tool depends entirely on which one you have.

work	use	why
I/O-bound, many connections	asyncio	thousands of waits on one thread
I/O-bound, blocking library	threads (asyncio.to_thread)	GIL released while waiting
CPU-bound Python	processes (ProcessPoolExecutor)	one GIL per process → real parallelism
long / heavy jobs	task queue workers	off the request path entirely

Figure 1. Which concurrency tool fits which kind of work.

3 The GIL

The **global interpreter lock** (GIL) lets only one thread execute Python bytecode at a time in a process. Threads still help with I/O, because the GIL is **released while waiting** on the network or disk (and inside many C extensions like NumPy). But for CPU-bound Python code, threads don't run in parallel; they take turns. To use several cores for Python computation, you need **several processes**, each with its own interpreter and GIL.

4 asyncio, and the blocking trap

asyncio runs many tasks on one thread with an **event loop**. When a task `await`s I/O, the loop switches to another task. That's great for thousands of concurrent connections with very little memory. But one blocking call (a slow CPU loop, `time.sleep`, a synchronous library like `requests`) freezes **every** task on that loop. That's what happened with the PDF parsing.

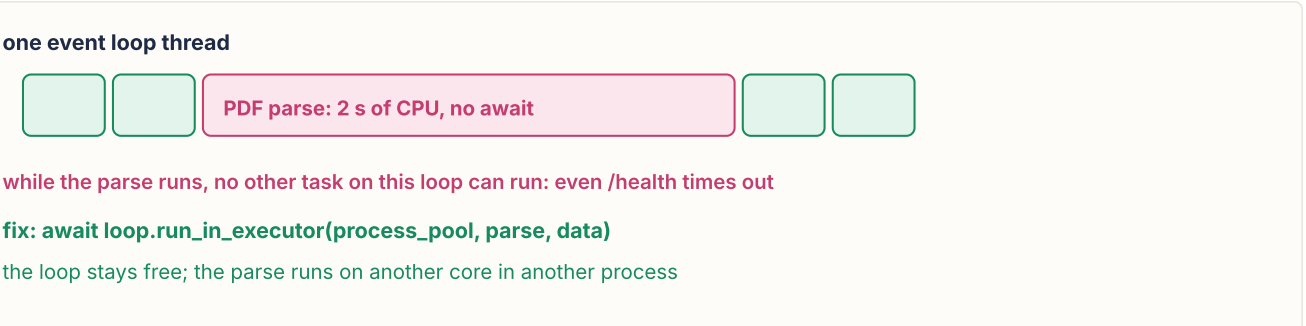


Figure 2. A CPU-heavy task without awaits blocks the event loop; every other request on that worker waits.

WORKED EXAMPLE: OFFLOADING PDF PARSING

The async API offloads CPU-heavy parsing to a process pool: `await loop.run_in_executor(pool, parse_pdf, data)` with a `ProcessPoolExecutor(max_workers=os.cpu_count())` created at startup. Each parse runs in its own process, on its own core, in parallel, while the event loop stays free to serve other requests. For blocking *I/O* libraries, `await asyncio.to_thread(func, …)` runs them in a thread pool instead.

5 In production: processes × event loops

Servers combine both. **Uvicorn** runs an asyncio event loop per worker; **Gunicorn** manages several worker processes, often with the Uvicorn worker class, typically one or two per CPU core. So processes give parallelism across cores, and asyncio gives high concurrency inside each process. For heavy jobs, use a task queue with separate workers (lesson 37).

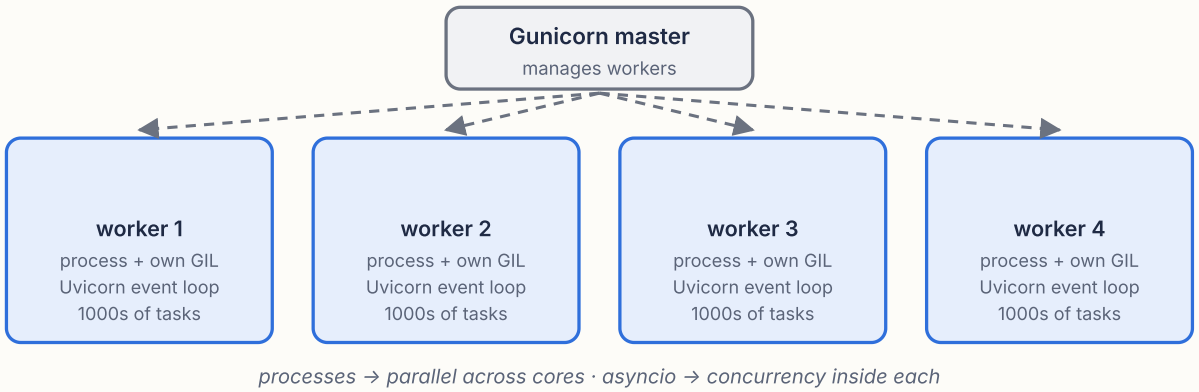


Figure 3. Gunicorn with Uvicorn workers: one process per core, each with its own event loop.

6 The future: free-threaded Python

Python 3.13 introduced an optional **free-threaded build** without the GIL (PEP 703), where threads can run Python code on several cores at once. It's experimental, and many libraries are still adapting. For now, design with the GIL in mind: processes for CPU, async or threads for I/O.

COMMON MISTAKES

- Calling `requests` or a sync DB driver inside `async def` routes.
- Using threads to speed up pure-Python number crunching.
- One Uvicorn worker on an 8-core machine.
- Creating a new process pool per request (huge startup cost).

7 Key terms

I/O-bound: time spent waiting on network or disk

GIL: one thread runs Python bytecode at a time per process

run_in_executor: run a function in a thread or process pool

Free-threaded build: Python without the GIL (3.13+, experimental)

CPU-bound: time spent computing

Event loop: runs many tasks on one thread, switching at awaits

asyncio.to_thread: run blocking code in a thread

8 Interview questions

Q1. When does asyncio help and when not? BASIC
It helps for I/O-bound work with many concurrent connections, like API calls and database queries; it doesn't help CPU-bound work, and any blocking call freezes the whole loop, so offload CPU work to a process pool.

Q2. How does the GIL affect scaling? BASIC
Only one thread runs Python bytecode at a time per process, so threads don't parallelise CPU-bound Python, and you scale across cores with multiple processes or workers.

Q3. A FastAPI endpoint resizes images and slows the whole service. Fix it. SCENARIO
Move resizing to a ProcessPoolExecutor via run_in_executor, or to a task queue with dedicated workers; keep the event loop for I/O only.

Q4. How many Gunicorn workers for a 4-core box running an async API? SCENARIO
Start with about one to two Uvicorn workers per core (4 to 8), then load test: more workers help only if the loop is busy; memory per worker is the other limit.

Q5. Do threads help with NumPy-heavy code? TRICKY
Often yes: NumPy releases the GIL inside its C loops, so threads can run those parts in parallel. Pure-Python loops don't benefit.

Q6. Why is a sync database driver in an async route so harmful? TRICKY
Each query blocks the event loop thread for its whole duration, so the worker serves one request at a time instead of thousands; use an async driver or to_thread.

Next: SD-J-006 · Batching

Batching

pay the fixed cost once for many items

Watch the 4-min video

In one line: batching groups many items into one operation so a fixed overhead (round trip, commit, syscall, GPU launch) is paid once and amortised; it raises throughput but makes items wait, so cap both batch size and wait time; micro-batching processes small batches continuously, and buffered items need a plan for crashes and partial failures.

1 Two thousand round trips a second

Your service writes every click event to Postgres, one `INSERT` at a time. At 2,000 events per second, the database spends most of its time on round trips and commits, not on data. Group those writes, and the same database handles ten times the load.

2 Why it works: amortisation

Every operation has a **fixed overhead**: a network round trip, a transaction commit (an fsync), a system call, a GPU kernel launch. If each item pays that overhead alone, the overhead dominates. Batching groups many items into one operation, so the fixed cost is paid once and spread across the batch: **amortisation**. One commit with 1,000 rows costs about the same fsync as a commit with one row.

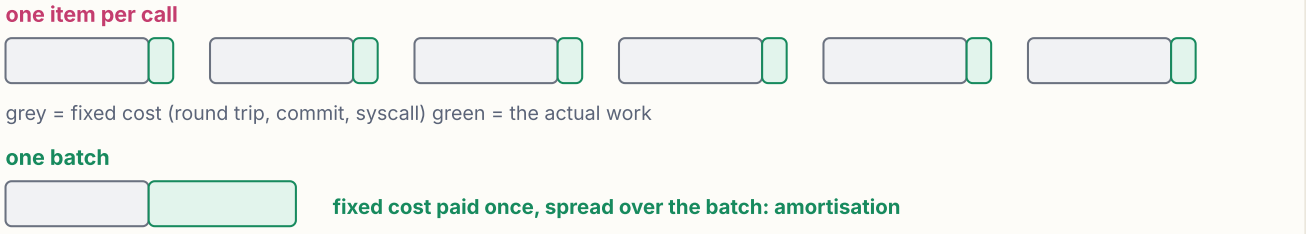


Figure 1. Per-item calls pay the fixed cost every time; a batch pays it once.

3 Where batching shows up

- **Databases**: multi-row `INSERT`, `COPY`, `executemany`, bulk upserts.
- **Redis**: `MGET` / `MSET` and pipelining (many commands, one round trip).
- **Kafka producers**: `linger.ms` and `batch.size` group messages per partition, and compress whole batches.
- **Batch APIs** that accept a list of items.
- **ML inference**: servers batch requests so the GPU processes many inputs in one pass.

4 The cost: waiting

To build a batch, items have to wait. Bigger batches give better throughput, but each item waits longer before it's processed. So set **two limits**: a maximum **batch size** and a maximum **wait time**, and flush when either is reached. Under heavy load batches fill quickly; under light load the timer bounds the delay.

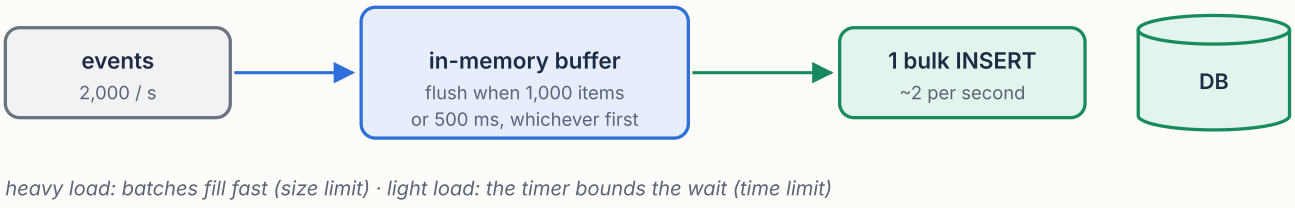


Figure 2. A buffer that flushes on size or time, whichever comes first.

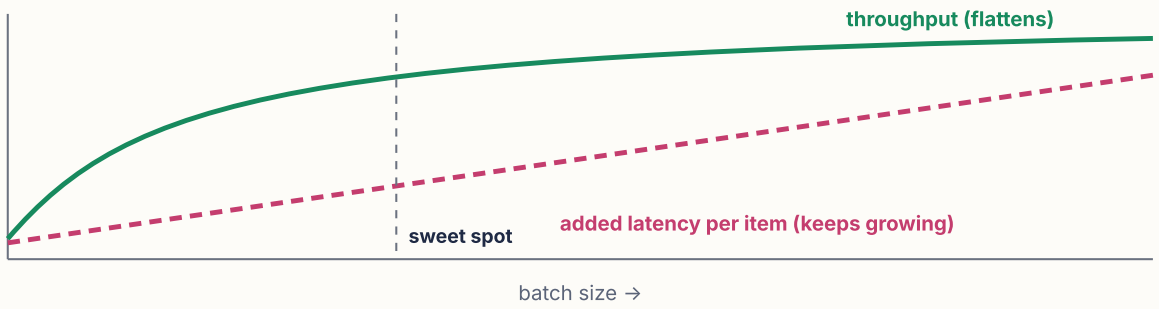


Figure 3. Throughput gains flatten as batches grow, while added latency keeps rising: pick a size inside your latency budget.

5 Micro-batching

Between one item at a time and one huge nightly batch is **micro-batching**: small batches processed continuously, every few hundred milliseconds. Spark Streaming made it famous, and **dynamic batching** in model servers such as Triton and vLLM (continuous batching of tokens) uses the same idea for inference.

WORKED EXAMPLE: CLICK EVENTS

The API appends click events to an in-memory buffer. A background task flushes it every **500 ms**, or when it reaches **1,000 events**, with one bulk insert (`COPY` or a multi-row INSERT). Database round trips drop from 2,000 per second to about **2 per second**. Each event lands at most half a second later, which is fine for analytics.

6 Risks

- **Crash loss**: items in an in-memory buffer are lost if the process dies. Put durable data on a queue first, or accept the loss for analytics.
- **Partial failure**: one bad item can fail the whole batch. Validate first, or retry the rest and send the bad item to a dead-letter queue.
- **Big batches** cause memory spikes, long locks and replication lag. Keep them moderate (hundreds to a few thousand rows).
- **Measure**: tune size and wait time against your latency budget.

COMMON MISTAKES

- A size limit with no time limit: at night, items wait forever.
- Batching user-facing writes the user must see immediately.
- Not flushing the buffer on graceful shutdown.

7 Key terms

Batching: many items in one operation

Pipelining: send many commands without waiting for each reply

Micro-batching: small batches, processed continuously

Amortisation: spreading a fixed cost over many items

linger.ms: Kafka producer wait time to fill a batch

Dynamic batching: an inference server grouping requests on the fly

8 Interview questions

Q1. What are the latency trade-offs of batching? BASIC

Batching raises throughput by amortising fixed overhead, but each item waits for the batch to fill, so you cap both the batch size and the wait time to bound the added latency.

Q2. What is micro-batching? BASIC

Processing small batches continuously, every few hundred milliseconds, to balance throughput and freshness.

Q3. Your service makes 50 Redis GETs per request. How do you speed it up? SCENARIO

Batch them with MGET or a pipeline so they share one round trip; at 0.5 ms per round trip that saves about 25 ms per request.

Q4. Design ingestion of 50,000 events per second into a database. SCENARIO

Put events on a durable queue (Kafka), have consumers read in batches, and write with bulk inserts or COPY every few hundred milliseconds or few thousand rows, with idempotent upserts for retries.

Q5. Why does batching help GPU inference so much? TRICKY

A GPU pass has large fixed costs (kernel launches, moving weights through memory); processing many inputs at once uses its parallel cores fully, so throughput rises many times while per-request latency rises a little.

Q6. One row in a 1,000-row batch violates a constraint. What happens and what do you do? TRICKY

The whole transaction fails. Validate beforehand, use ON CONFLICT handling, or split and retry (binary split) to isolate the bad row and send it to a dead-letter queue.

Next: SD-J-007 · Compression

Compression

trade CPU for bytes, when it pays

Watch the 4-min video

In one line: compression spends CPU to send and store fewer bytes; it wins on repetitive text when the network or disk is the bottleneck; use Brotli or gzip for HTTP (low levels for dynamic responses, max level precompressed for static files) and zstd for storage and internal traffic, and skip it for already-compressed or tiny data, CPU-bound servers, or secrets mixed with user input.

1 Two megabytes on a slow phone

Your API returns a 2 MB JSON product list. Mobile users on slow networks wait seconds for it. Turn on compression and it becomes about 200 KB. But compression costs CPU, and sometimes it isn't worth it.

2 CPU for bytes

The server spends CPU to shrink the data, the network carries fewer bytes, and the client spends CPU to decompress. It wins when the **network is the bottleneck** and the data is **repetitive**: JSON, HTML, logs, CSV. The **compression ratio** measures how much smaller it gets; text often shrinks to a fifth or a tenth.



Figure 1. Compression moves cost from the network to the CPUs at both ends.

3 Three algorithms

algorithm	ratio	speed	best for
gzip	good	medium	universal HTTP; every client
Brotli (br)	~15–20% smaller than gzip on text	slow at max level	HTTP text, precompressed static files
zstd	excellent	very fast	storage, logs, Kafka, internal traffic

Figure 2. gzip, Brotli and zstd compared.

Each has **levels**: higher levels squeeze more but cost much more CPU. For **dynamic** responses use a low or medium level (gzip 6 is a common default; Brotli 4–5), so compression doesn't add latency. For **static** assets, compress once at build time with the highest level (Brotli 11) and serve the precompressed file. In HTTP, the client sends `Accept-Encoding: br, gzip`, and the server replies with `Content-Encoding: br` (and `Vary: Accept-Encoding`, lesson 31).

4 Compression in storage

Columnar formats like **Parquet** store each column together, and similar values compress extremely well, often ten times or more. **Kafka** compresses whole batches of messages (lesson 39). Databases compress pages and backups. Less data on disk means less I/O and cheaper storage, and compressed data is often **faster** to read, because there are fewer bytes to read.

WORKED EXAMPLE: BROTLI AT THE CDN

A public JSON API sits behind a CDN. Enable Brotli at the CDN edge: browsers that send `Accept-Encoding: br` get Brotli, older clients get gzip. The origin spends no CPU on it, response sizes drop by around **80%**, and mobile load times improve noticeably.

5 When it's not worth it



Figure 3. Four cases where compression costs more than it saves.

Already-compressed data (JPEG, PNG, video, zip) won't get smaller. Tiny responses, under about 1 KB, can even grow. CPU-bound servers may get slower. And over HTTPS, compressing secrets together with attacker-controlled input can leak data through response sizes, as in the **BREACH** attack: don't compress

responses that reflect user input alongside tokens, or mask the tokens. So measure, rather than turning it on everywhere blindly.

COMMON MISTAKES

- Gzip on images and video at the proxy.
- Brotli level 11 on dynamic responses: huge CPU cost per request.
- Forgetting `Vary: Accept-Encoding`, so a CDN serves compressed bytes to a client that can't read them.

6 Key terms

Compression ratio: original size ÷ compressed size

Brotli: better ratio for text, all modern browsers

Accept / Content-Encoding: client offers, server declares the encoding

BREACH: attack that infers secrets from compressed sizes

gzip: universal, decent ratio

zstd: very fast, excellent ratio, storage and internal

Parquet: columnar file format that compresses well

7 Interview questions

Q1. When is compression not worth it? BASIC

When data is already compressed, like images and video, when payloads are tiny, when the server is CPU-bound rather than network-bound, or when secrets mix with user input over HTTPS.

Q2. How do you choose an algorithm? BASIC

Brotli or gzip for HTTP text, precompressed at the highest level for static files, and zstd for storage and internal traffic.

Q3. Mobile clients complain about a slow 3 MB JSON response. What do you do? SCENARIO

Enable Brotli/gzip at the CDN or proxy, and also shrink the payload: paginate, return only needed fields, maybe switch the internal path to a compact format.

Q4. Your logs cost too much to store. Options? SCENARIO

Compress with zstd (or store as Parquet with compression), tier old logs to cheaper storage, and drop or sample noisy log lines.

Q5. Why can compressed data be faster to read from disk? TRICKY

Reading is often I/O-bound; fewer bytes to read means less I/O time, and fast decompressors like zstd or LZ4 decode faster than the disk delivers raw bytes.

Q6. How does BREACH work, roughly? TRICKY

If a response contains a secret and attacker-controlled text, the attacker guesses characters: a correct guess compresses better, so the response gets slightly smaller, which they can observe even over HTTPS.

Next: SD-J-008 · Serialization Formats backendroadmap.com

Serialization Formats

JSON, MessagePack, Protobuf and Avro, and evolving schemas safely

Watch the 4-min video

In one line: JSON is readable and universal but verbose and slow; MessagePack is compact schemaless binary JSON; Protobuf and Avro use schemas, so messages are small and fast and schemas evolve under clear rules; use JSON for public APIs, Protobuf for gRPC, Avro with a schema registry for event streams, and never pickle across services.

1 JSON at a hundred thousand events a second

Your services exchange events as JSON. It's readable and easy. But at 100,000 events per second, JSON parsing burns CPU, every message repeats the field names, and one team renames a field and breaks three consumers. The serialization format matters more than it looks.

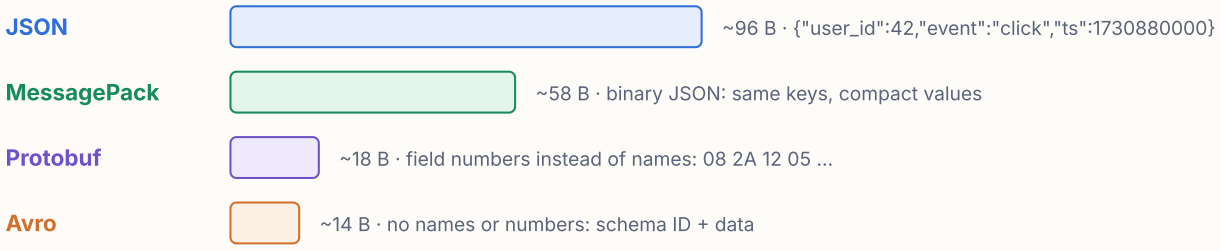


Figure 1. One small event in four formats (approximate sizes): schemas let binary formats drop the field names.

2 JSON

The default: human-readable, supported everywhere, no schema needed. But it's verbose (field names repeat in every message), numbers are text, binary data needs base64, and parsing is relatively slow. In Python, **orjson** is several times faster than the standard `json` module. JSON is still right when humans or browsers read the data.

3 MessagePack

Binary JSON: the same data model, no schema, but more compact and faster to parse. A good fit for caches (lesson 33) and internal messages when you want speed without defining schemas. Libraries exist for almost every language.

4 Protocol Buffers

A schema in a `.proto` file. Fields are identified by **numbers**, not names, so messages are small and parsing is fast. Code is generated for each language, and Protobuf powers gRPC. The evolution rules are simple: add new fields with new numbers, never reuse a number (mark removed ones `reserved`), and old and new code keep working together, because readers skip fields they don't know (Phase 2, API lesson 33).

```
syntax = "proto3";
message ClickEvent {
  int64  user_id = 1;
  string event   = 2;
  int64  ts       = 3;
  string page    = 4;   // added later: old readers skip it
  reserved 5;          // removed field: number never reused
}
```

Figure 2. A Protobuf schema that evolved safely: a new field with a new number, and a removed number reserved.

5 Avro and a schema registry

Avro also uses a schema, but the schema is stored **separately**, often in a **schema registry**. Messages carry no field names or numbers at all, just the data. The reader resolves the **writer's** schema against its own **reader** schema, which makes evolution well defined; **default values** let you add a field without breaking old readers. Avro is the classic choice for Kafka events and data lakes.

WORKED EXAMPLE: EVENTS FROM JSON TO AVRO

Producers register the event schema in a schema registry (Confluent, Apicurio). Each message carries a small schema ID plus binary data. Messages shrink by more than half. The registry enforces a compatibility mode (for example BACKWARD), so a breaking change, like removing a field without a default, is **rejected before it reaches production**.

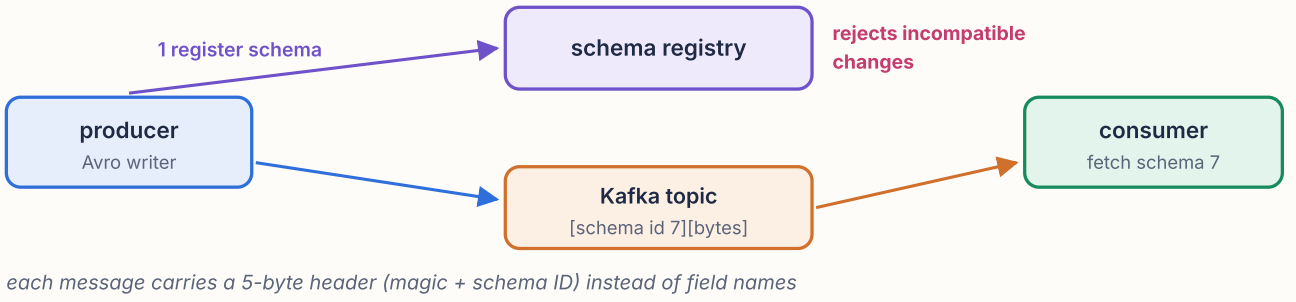


Figure 3. A schema registry: producers register schemas, messages carry only a schema ID, consumers fetch the schema to decode.

6 How to choose

Use case	Format
Public APIs, browsers, debugging	**JSON** (orjson in Python)
gRPC and service-to-service calls	**Protobuf**
Event streams, data pipelines	**Avro** (or Protobuf) with a schema registry
Compact, schemaless internal data, caches	**MessagePack**
Across services	never **pickle**: Python-only and unsafe with untrusted input

Whatever you pick, version your schemas and test compatibility in CI.

COMMON MISTAKES

- Reusing a Protobuf field number for a different field.
- Renaming JSON fields in an event without versioning, breaking consumers silently.
- Adding an Avro field without a default, breaking old readers.
- Large integers in JSON losing precision in JavaScript (above 2^53).

7 Key terms

Serialization: turning objects into bytes, and back

Field number: Protobuf's on-the-wire identity for a field

Writer / reader schema: Avro resolves the two when decoding

Schema: a formal description of the fields and types

Schema registry: stores schemas and checks compatibility

Compatibility mode: rules such as BACKWARD or FULL

8 Interview questions

Q1. Which serialization format would you use for events and why? BASIC
Avro or Protobuf with a schema registry, because they're compact and fast, and they enforce compatible schema evolution between producers and consumers.

Q2. Why not JSON everywhere? BASIC
It's verbose and slower to parse, and without a schema a renamed field can silently break consumers.

Q3. How do you add a field to a Protobuf message used by 20 services? SCENARIO
Add it with a new field number (optional), deploy readers and writers in any order since unknown fields are skipped, and never reuse or renumber existing fields.

Q4. A team wants to remove a field from an Avro event. How? SCENARIO
Under backward compatibility, readers must cope: give the field a default in the reader schema first, then stop writing it; the registry blocks removals that would break existing consumers.

Q5. Why does Avro need the writer's schema to decode? TRICKY
Messages contain only values in field order, no names or numbers, so the reader must know exactly which schema wrote them, which the schema ID in the message points to.

Q6. Why is pickle dangerous between services? TRICKY
Unpickling can execute arbitrary code embedded in the data, and it ties every consumer to Python and compatible class definitions.

Next: SD-J-009 · Resource Utilization (USE Method)

Resource Utilization (USE Method)

a checklist that finds the bottleneck fast

Watch the 4-min video

In one line: the USE method (Brendan Gregg): for every resource, hardware and software, check Utilization (how busy), Saturation (work waiting in a queue) and Errors; saturation is the key signal, because queues cause latency and averages hide bursts and single hot cores.

1 Where do you even start?

Latency is up and the dashboards are full of graphs. CPU looks fine. Memory looks fine. Guessing wastes hours. The **USE method** gives you a checklist that finds the bottleneck fast.

2 Utilization, saturation, errors

Created by Brendan Gregg, it says: **for every resource, check all three**.

- **Utilization**: how busy the resource is, as a share of time or capacity.
- **Saturation**: extra work that can't be served yet, usually waiting in a queue.
- **Errors**: failures, like dropped packets or disk errors. Often the quickest to check, so start there.

It works for hardware and for software resources: thread pools, connection pools and file descriptors all have a utilization, a queue and errors.

resource	Utilization	Saturation	Errors
CPU	% busy per core	run queue (vmstat r)	machine-check errors
Memory	% used	swapping, OOM kills	allocation failures
Disk	% busy (iostat %util)	queue length, await ms	I/O errors (dmesg)
Network	bandwidth used	drops, retransmits	interface errors
Pools (software)	active / max	waiters for a slot	timeouts acquiring

Figure 1. The USE checklist: what to look at for each resource.

3 Why saturation matters most

A resource at 70% *average* utilization can still be saturated in bursts, with requests queueing. Averages also hide individual cores: one core at 100% can bottleneck a single-threaded process while the machine average looks low. A disk at 40% busy can still have requests waiting behind a burst of writes. So look at **queues**, not just percentages (lesson 45 explains why queues explode).

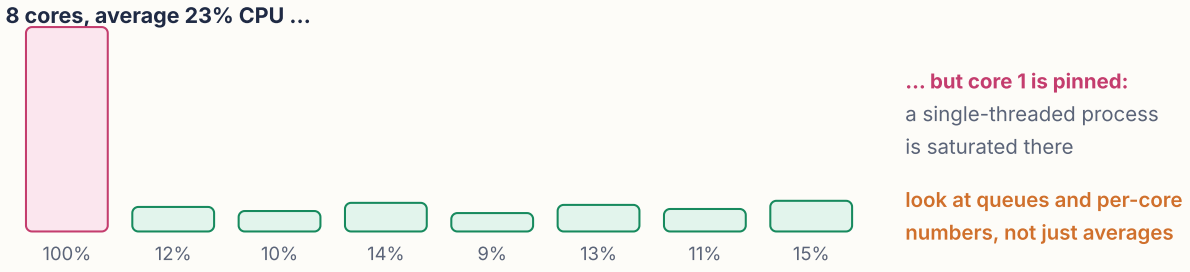


Figure 2. A healthy-looking average with one saturated core.

4 Tools

- `vmstat 1` and `top` (press 1 for per-core): CPU and the run queue (`r`).
- `free -m` and `vmstat` si/so: memory and swapping; `dmesg` for OOM kills.
- `iostat -x 1`: disk %util, queue size and await time.
- `sar -n DEV`, `ss -s`, `netstat -s`: network throughput, drops, retransmits.
- In the cloud, the same metrics come from CloudWatch or Azure Monitor (watch burst credits and IOPS limits on cloud disks).

WORKED EXAMPLE: LATENCY JUMPS, CPU IS FINE

API latency jumps. CPU is at 30% and memory is fine. Check the disk: `iostat` shows utilization near 100%, a long queue and wait times of 50 ms. Disk I/O is saturated. The cause: a log job writing to the same volume as the database. Move it to its own disk, and latency returns to normal.

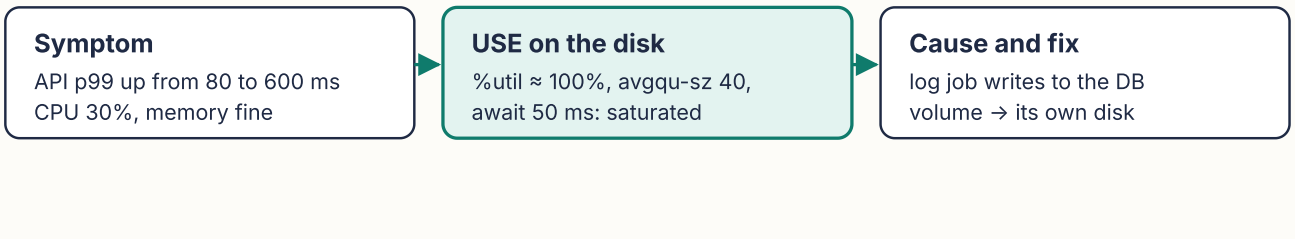


Figure 3. USE in action: walking the resources finds the saturated disk in minutes.

5 Use it systematically

The power of USE is that it's a checklist: you walk the list of resources and ask the three questions, so you won't miss the saturated one. Start with resources closest to the symptom, then work outward. It finds **resource** bottlenecks quickly; for problems inside the code itself you need profiling (next lesson).

COMMON MISTAKES

- Looking only at average CPU and memory.
- Forgetting software resources: a full connection pool looks like a slow database.
- Ignoring errors (retransmits, disk errors) that explain the latency directly.

6 Key terms

USE method: Utilization, Saturation, Errors for every resource

Saturation: queued work waiting for the resource

await (iostat): average time an I/O request takes, including queueing

Utilization: how busy a resource is

Run queue: threads waiting for a CPU

Software resource: pools, locks, file descriptors

7 Interview questions

Q1. What is the USE method? BASIC
For every resource, like CPU, memory, disk and network, check utilization, saturation and errors, to find the bottleneck systematically.

Q2. Why focus on saturation? BASIC
Because queueing causes latency, and averages can look healthy while a resource is saturated in bursts or on a single core.

Q3. Latency is high, CPU is 25%, memory fine. What next? SCENARIO
Walk USE: check per-core CPU, disk %util and await, network drops and retransmits, and software pools (DB connections, thread pools) for waiters and timeouts.

Q4. How would you apply USE to a database connection pool? SCENARIO
Utilization: connections in use / max. Saturation: requests waiting for a connection and wait time. Errors: timeouts acquiring a connection.

Q5. A disk shows 40% utilization but long waits. How? TRICKY
Utilization is averaged over the interval; bursts of writes can fill the queue briefly, and fast SSDs serve requests in parallel so %util understates load. Look at queue size and await.

Q6. When doesn't USE find the problem? TRICKY
When the bottleneck is in the code (an inefficient algorithm, lock contention inside the app) or in a downstream dependency; then use profiling and distributed tracing.

Next: SD-J-010 · Performance Profiling

Performance Profiling

measure where the time goes, don't guess

Watch the 4-min video

In one line: profile before optimising: sampling profilers like py-spy attach to a running process with low overhead and produce flame graphs where width is time; fix the widest frame, measure again; cProfile is exact but heavy, Scalene and memray add memory views, and CPU profiles don't show waiting, which needs tracing.

1 A day spent optimising the wrong thing

Your Python service runs at 100% CPU and p99 latency is climbing. You think it's the database code, so you spend a day optimising it. Nothing changes. **Never guess where time goes; measure it.** That's profiling.

2 Two kinds of profiler

Deterministic profilers, like `cProfile`, hook every function call and record exact counts and times, but add a lot of overhead and distort fast functions. **Sampling** profilers, like `py-spy`, look at the stack many times per second and count where the program is. Low overhead, and safe for production.



Figure 1. Exact but heavy, or statistical and light.

3 Reading a flame graph

Each box is a function, stacked on top of its caller. The **width** of a box is its share of samples, so wide boxes are where the time goes. Read it bottom up and look for the widest plateaus. Left to right is *not* time; it's just sorted alphabetically. A narrow tower is deep, but not expensive.

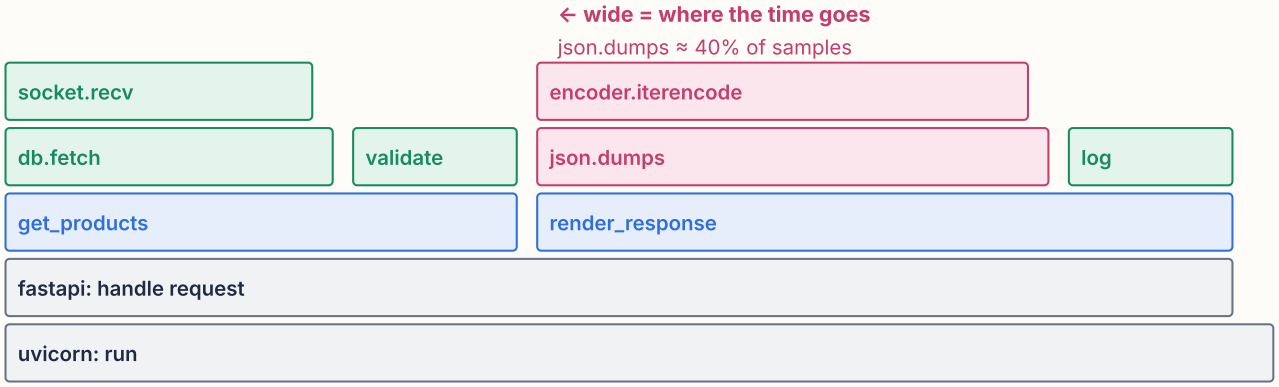


Figure 2. A flame graph (simplified): json.dumps under render_response is the widest leaf, about 40% of CPU.

4 Python tools

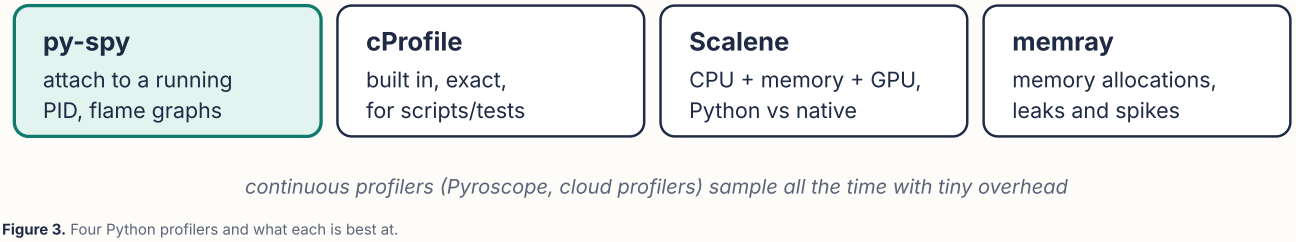


Figure 3. Four Python profilers and what each is best at.

`py-spy record -o flame.svg --pid 1234 --duration 60` attaches without code changes or restarts; `py-spy top --pid 1234` shows a live view. **Scalene** separates Python time from native time (is NumPy doing the work?), and **memray** tracks allocations to find leaks and spikes.

5 Async code and production

In asyncio, a profiler mostly shows the event loop, so look for coroutines that **block** the loop (lesson 38). py-spy's `--idle` flag includes idle time when you need it. In production, profile **one instance** for 30 to 60 seconds under real traffic. Continuous profilers (Pyroscope, Datadog, Google Cloud Profiler) sample all the time with tiny overhead.

WORKED EXAMPLE: A CPU-BOUND FASTAPI SERVICE

Run `py-spy record` on one worker for 60 seconds. The flame graph shows **40%** of the width under `json.dumps`, serialising large responses with the standard library. Switch to **orjson** (and FastAPI's `ORJSONResponse`), and CPU drops by a third while p99 improves. One measurement, one targeted fix.

6 Make it a habit

1. Reproduce with realistic load (next lesson).
2. Profile; find the widest frames.
3. Fix the biggest one.
4. Measure again: the profile changes.

And remember: CPU profiles don't show **waiting**. If latency is high but CPU is low, look at I/O, locks and downstream calls with distributed tracing instead. Profiles tell you what is busy; tracing tells you what is waiting.

COMMON MISTAKES

- Optimising code that isn't on the hot path.
- Running cProfile in production and blaming the slowdown on the app.
- Profiling with toy data, where the hot path is different.
- Reading flame graph x-position as time.

7 Key terms

Deterministic profiler: instruments every call (cProfile)

Flame graph: stacked boxes, width = share of samples

Continuous profiling: always-on low-overhead sampling

Sampling profiler: periodically samples stacks (py-spy)

Hot path: code where most time is spent

Tracing: timing requests across services, shows waits

8 Interview questions

Q1. How do you find where a Python service spends CPU time? BASIC
Attach a sampling profiler like py-spy to a running process under real load, generate a flame graph, and look for the widest frames.

Q2. Why sampling instead of cProfile in production? BASIC
Sampling has very low overhead and needs no code changes, while cProfile slows every call and distorts the results.

Q3. How do you read a flame graph? BASIC
Each box is a function on top of its caller; width is the share of samples, so the widest boxes, especially wide leaves, are where time goes; horizontal order is not time.

Q4. Latency is high but CPU is 20%. Will a profiler help? SCENARIO
Not much: the service is waiting. Use distributed tracing and metrics for I/O, locks, connection pools and downstream calls (and USE for resources).

Q5. Memory grows until pods are OOM-killed every day. What do you use? SCENARIO
A memory profiler such as memray or tracemalloc snapshots over time, to see which allocations grow; check caches without limits and objects kept alive by references.

Q6. Why can profiling an asyncio app be confusing? TRICKY
Most samples show the event loop waiting; the real issues are coroutines doing CPU work or blocking calls without awaits, which you find by looking for long synchronous frames inside tasks.

Next: SD-J-011 · Load Testing and Stress Testing backendroadmap.com

Load Testing and Stress Testing

prove capacity before launch day does it for you

Watch the 4-min video

In one line: load tests confirm targets at expected traffic, stress tests find the breaking point, soak tests catch slow leaks, and spike tests check autoscaling; the traffic model and environment must be realistic, pass criteria (p95/p99, errors) are set beforehand, every resource is watched, and the tool must avoid coordinated omission.

1 "It can handle 5,000 requests per second"

How do you know? Launch day is the worst time to find out. **Load testing** proves capacity before real users do.

2 Four kinds of test

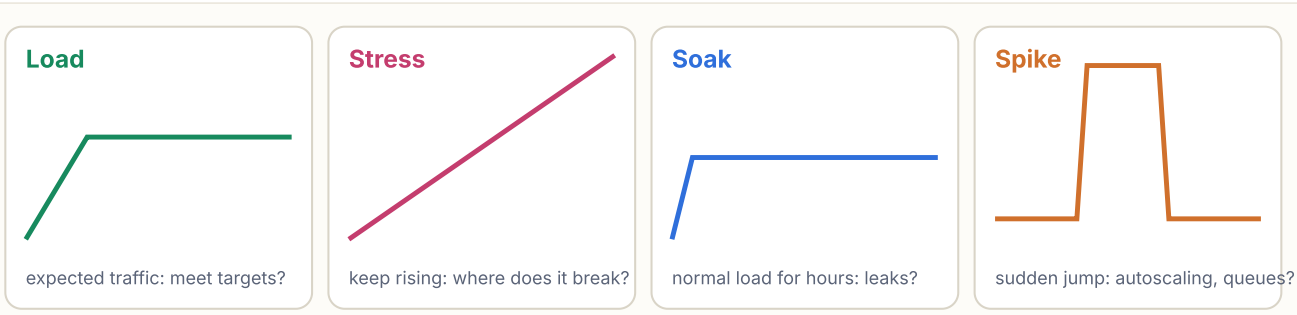


Figure 1. Four test shapes, four questions. A good plan uses more than one.

- **Load test**: expected traffic, to confirm latency and error targets.
- **Stress test**: keep increasing load beyond normal, to find the breaking point and see *how* the system fails (gracefully with 503s, or by collapsing).
- **Soak test**: normal load for hours, to catch memory leaks, connection leaks and slow degradation.
- **Spike test**: jump traffic suddenly, to check autoscaling, queues and load shedding (lessons 10 and 14).

3 The traffic model matters most

- A realistic **mix** of endpoints, e.g. 80% reads and 20% writes, weighted like production.
- Realistic **data**: many different users and products, so caches aren't unrealistically hot.
- **Think time** between user actions.
- A **production-like environment**: same instance sizes, same database size and indexes, same limits.

4 Tools

Locust (Python: simulated users are Python classes, a natural fit for Python teams), **k6** (JavaScript scripts, very efficient, good thresholds), and **JMeter** (older Java tool with a GUI). Run load generators from separate machines, and make sure *they* aren't the bottleneck (watch their CPU).

5 Measure the right things

Throughput (requests per second), latency **percentiles** (p95, p99, not averages), error rate, and server-side resources (CPU, memory, connection pools, the database and caches), so you know which saturates first. Set **pass criteria before the test**, not after.

WORKED EXAMPLE: PROVING 5,000 REQUESTS PER SECOND

A Locust test ramps to 5,000 requests per second over 10 minutes with a realistic mix of search, product views and checkout. Targets: p99 < 300 ms, errors < 0.1%. Result: p99 is **240 ms** at 5,000 per second. Pushing further, the database connection pool saturates at about **7,000**. Now you know the capacity, the headroom, and the next bottleneck.

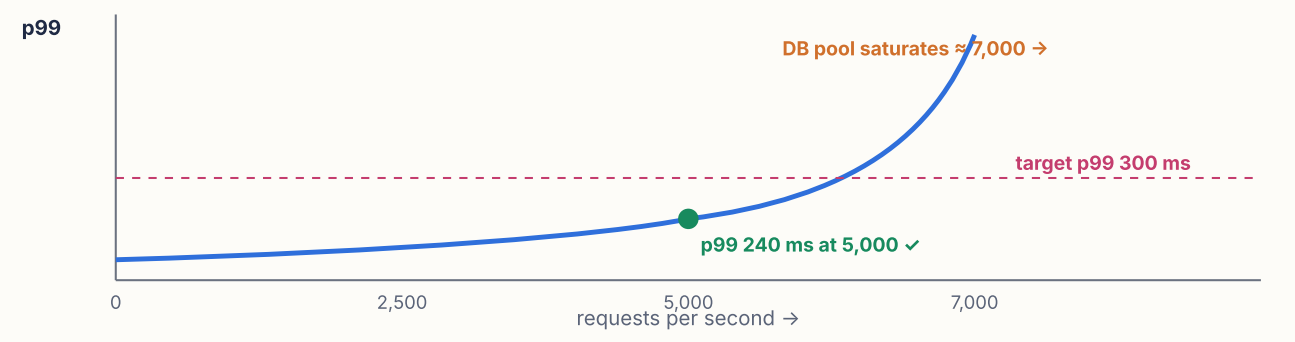


Figure 2. Latency against load: under target at 5,000 per second, with the knee near 7,000 where the connection pool saturates.

6 Make it routine, and avoid coordinated omission

Run a short load test in CI for critical paths to catch regressions, and soak tests before major releases. Beware **coordinated omission**: if the load tool waits for slow responses before sending the next request (a closed loop), it sends fewer requests exactly when the system is slow, and under-reports latency. Use tools that keep sending at a fixed rate (k6 constant-arrival-rate, wrk2).



Figure 3. Coordinated omission: a closed-loop tool hides stalls; an open-loop tool measures them.

COMMON MISTAKES

- Testing one endpoint with one user ID: everything is cached.
- A tiny staging database, so queries look fast.
- Reporting average latency.
- Load generators maxed out and reported as server capacity.

7 Key terms

Load test: expected traffic vs targets

Soak test: normal load for hours

Think time: pause between simulated user actions

Stress test: beyond normal, to the breaking point

Spike test: sudden jump in traffic

Coordinated omission: a closed-loop tool hiding slow periods

8 Interview questions

Q1. How do you design a load test? BASIC
Define the goal and pass criteria, model realistic traffic and data, ramp load in a production-like environment, and measure throughput, latency percentiles, errors and resource saturation.

Q2. What is a soak test for? BASIC
Running normal load for hours, to find memory leaks, resource exhaustion and slow degradation.

Q3. Your load test shows 20 ms p99 but production is 400 ms. Why might that be? SCENARIO
Unrealistic test: hot caches from repeated IDs, a small database, missing downstream calls, a different traffic mix, or coordinated omission in the tool.

Q4. How do you find the capacity of a service? SCENARIO
A stress test that ramps load in steps while watching p99, errors and resources; capacity is the highest rate that still meets the targets, and the first saturated resource is the next bottleneck.

Q5. What is coordinated omission? TRICKY
When a load tool waits for each response before sending the next, a slow period reduces the number of requests sent, so the slow period is under-represented in the results; fixed-rate (open-loop) generation fixes it.

Q6. Why set pass criteria before the test? TRICKY
So results can't be rationalised afterwards, and the test can fail a CI pipeline automatically when p99 or error rates regress.

Next: SD-J-012 · Queueing Theory Fundamentals backendroadmap.com

Queueing Theory Fundamentals

why latency explodes near full utilization

Watch the 4-min video

In one line: Little's Law says items in a system = arrival rate × time in system (L = λW); with random arrivals, time in system grows like service time ÷ (1 − utilization), so latency doubles at 50%, is 10× at 90% and 20× at 95%; run interactive services around 60–80%, alert on queues, and prefer shared queues over per-server queues.

1 From 70% to 90%, latency triples

Your servers run at 70% CPU and latency is fine. Traffic grows, CPU reaches 90%, and latency doesn't grow a little: it triples. Nothing is broken; this is simply how queues behave.

2 Little's Law

The average number of items in a system equals the arrival rate times the average time each spends inside: **L = λ × W**. At 100 requests per second, each taking 200 ms, about **20 requests are in flight** on average. It holds for any stable system, whatever the distributions (Phase 1, lesson 9). Turned around, it lets you estimate latency from a queue length you can observe: W = L / λ.

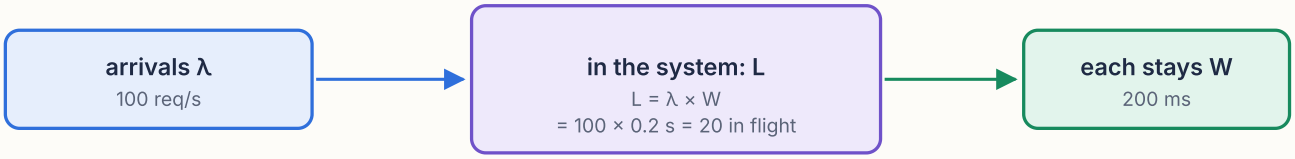


Figure 1. Little's Law with numbers: 100 requests per second × 0.2 s = 20 in flight.

3 Utilization and random arrivals

Utilization ρ is the arrival rate divided by the service rate: a server that can handle 100 requests per second and receives 80 is 80% utilised. But requests arrive **randomly**, not evenly spaced, so sometimes several arrive together and queue. The closer utilization gets to 100%, the less spare time the server has to clear those bursts.

4 The M/M/1 curve

The simplest model is **M/M/1**: random (Poisson) arrivals, random (exponential) service times, one server. Its key result:

```
time in system W = S / (1 − ρ)  (S = service time, ρ = utilization)
```

At 50%, time doubles. At 80%, it's 5×. At 90%, 10×. At 95%, 20×. The curve shoots up as utilization approaches 100%. Real systems with several servers or less variable service times do better, but the shape, a **knee** followed by a wall, is the same.

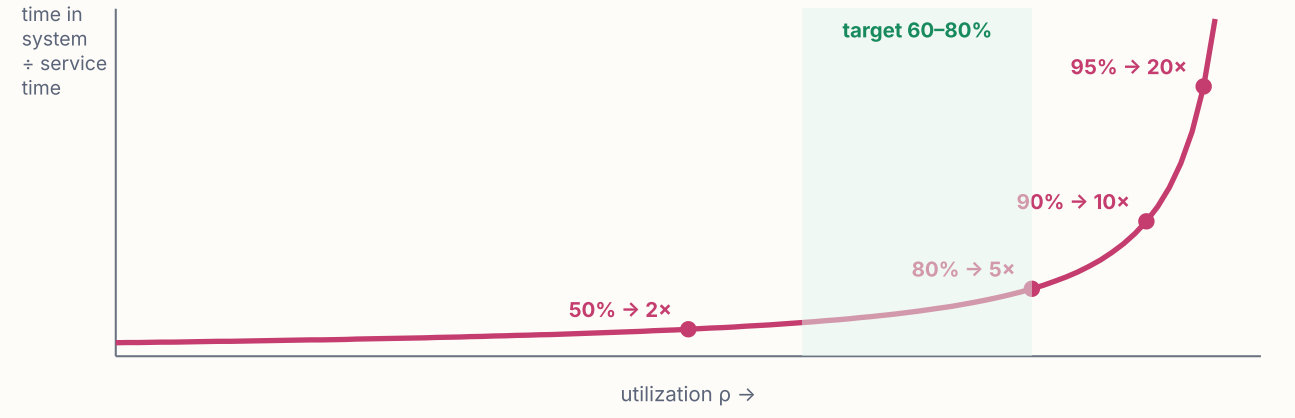


Figure 2. M/M/1: time in system as a multiple of service time. Below the knee latency barely moves; above it, it explodes.

5 Headroom isn't waste

That's why teams target around **60 to 80%** utilization for interactive services. Below the knee, small increases in traffic barely change latency. Above it, a small burst creates a long queue, latency explodes, and timeouts and retries add even more load (lesson 10). Batch jobs can run hotter, because nobody is waiting on them interactively.

WORKED EXAMPLE: WHY LATENCY TRIPLES

With a 10 ms service time: at 70% CPU, time in system ≈ 10 / 0.3 ≈ **33 ms**. At 90%, ≈ 10 / 0.1 = **100 ms**. Same code, same server, three times the latency, all from queueing.

6 Using it in practice

- **Size pools** with Little's Law: 500 requests per second × 40 ms of DB time each ≈ 20 connections busy on average; add headroom for bursts.
- **Alert on queue length and saturation**, not only CPU (lesson 42).
- **Autoscale before the knee** (lesson 14).
- **Share queues**: several servers pulling from one queue behave much better than separate queues per server, one reason load balancing (and least-connections) helps.



Figure 3. Separate queues let one line jam while a server idles; a shared queue keeps every server busy.

COMMON MISTAKES

- Planning capacity at 90%+ utilization for user-facing services.
- Assuming latency grows linearly with load.
- Ignoring variability: bursty arrivals or very uneven request costs make the knee come earlier.

7 Key terms

Little's Law: L = λ × W

Service rate μ: requests per second a server can finish

M/M/1: random arrivals, random service, one server

Arrival rate λ: requests per second coming in

Utilization ρ: λ / μ

Knee: where latency starts rising steeply

8 Interview questions

Q1. Why should you not run servers at 95% utilization? BASIC
Because waiting time grows like 1 / (1 − utilization), so near full load latency is about 20 times the service time, and any burst causes huge queues and timeouts.

Q2. What does Little's Law say? BASIC
Items in the system equal arrival rate times time in the system: L = λ × W.

Q3. A service gets 2,000 req/s and each request holds a DB connection for 15 ms. How big a pool? SCENARIO
L = 2,000 × 0.015 = 30 connections busy on average; size for bursts and the utilization target, e.g. 40 to 50, and check the database can sustain that concurrency.

Q4. Service time is 20 ms and you need p50 under 60 ms. Rough max utilization? SCENARIO
W = 20 / (1 − ρ) ≤ 60 → 1 − ρ ≥ 1/3 → ρ ≤ about 67%.

Q5. Why does one shared queue beat a queue per server? TRICKY
With separate queues, one server can be idle while another's queue is long; a shared queue lets any free server take the next item, cutting waiting time sharply at the same utilization.

Q6. Does Little's Law need Poisson arrivals? TRICKY
No. It holds for any stable system in the long run, regardless of arrival or service distributions; the 1 / (1 − ρ) formula is the part that assumes M/M/1.

Next: SD-J-013 · Tail Latency

Tail Latency

why the slowest 1% dominates at scale, and how to fight it

Watch the 4-min video

In one line: a request that waits for many parallel calls is as slow as the slowest one, so with 100 calls that are each slow 1% of the time, about 63% of requests are slow; fight the tail with hedged and tied requests, partial results at a deadline, less variance at the source, and measure p99/p99.9 at the user-facing level with merged histograms.

1 A fine p99 that isn't fine

Your search service has a p99 of 100 ms: only one request in a hundred is slow. That sounds fine. But each user search fans out to 100 shards. Now most user searches are slow.

2 Fan-out turns the tail into the common case

If a single call is slow 1% of the time and a request must wait for 100 of them, the chance that **all** are fast is $0.99^{100} \approx$ **37%**. So about **63%** of user requests hit at least one slow call. The more services and shards a request touches, the more the tail matters.

$$P(\text{request slow}) = 1 - (1 - p)^n \quad p = 1\%,\ n = 100 \rightarrow 63\%$$

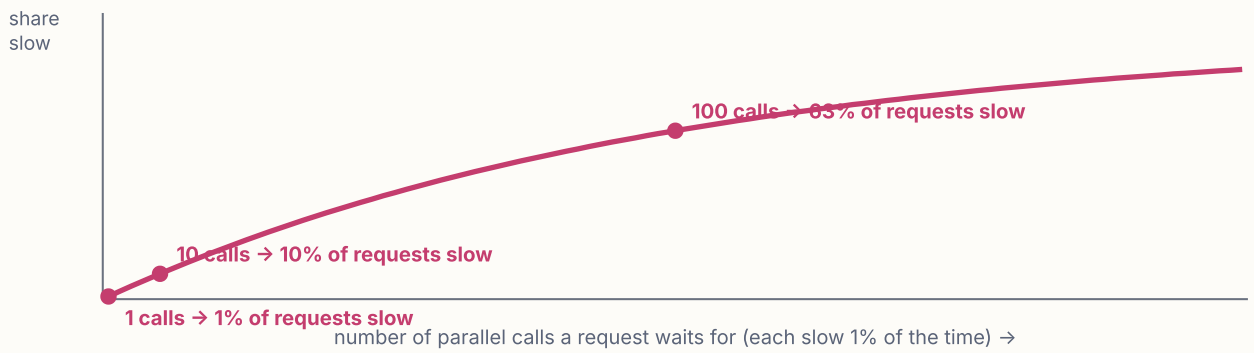


Figure 1. The share of requests that hit at least one slow call grows quickly with fan-out.

3 Where slow calls come from

Garbage-collection pauses. Noisy neighbours on shared hardware. Queueing behind a burst (lesson 45). Background work such as compaction or log rotation. Cache misses and cold starts. Network retransmissions. Most are random and short, which is exactly why they're hard to remove: at scale, something somewhere is always having a bad moment.

4 Hedged and tied requests

Hedged requests: send the request to one replica; if it hasn't answered by about the **p95** latency, send a second copy to another replica and use whichever answers first (cancel the other). Only about 5% of requests are duplicated, but the tail shrinks dramatically. Do this only for **idempotent reads**. Google described these techniques in *The Tail at Scale* (Dean and Barroso, 2013).

Tied requests go further: send to two replicas at once, each told about the other; when one starts processing, it cancels its twin, so duplicates rarely do real work.

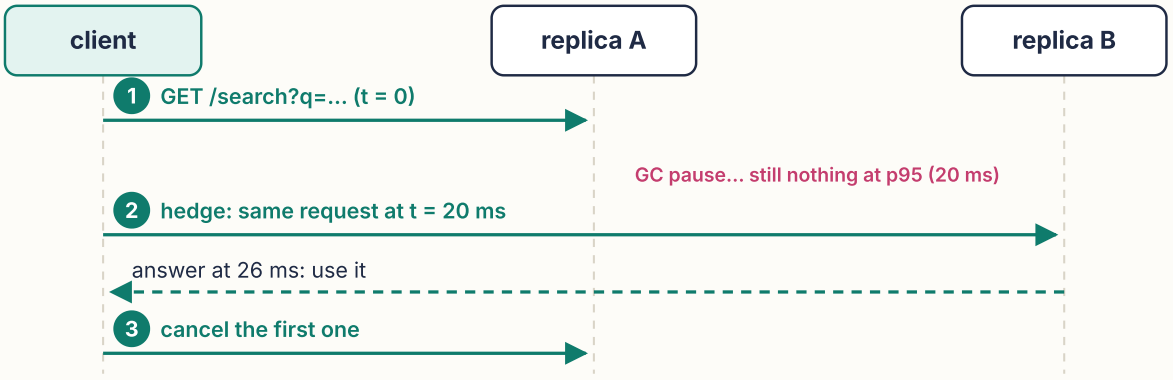


Figure 2. A hedged request: the first replica stalls, the hedge to a second replica answers, and the first is cancelled.

5 More tools

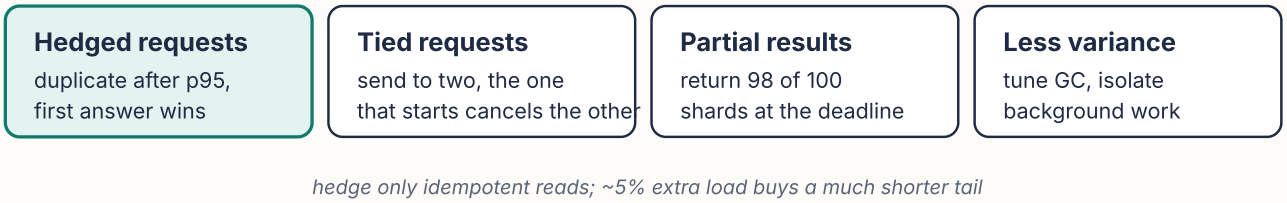


Figure 3. Four ways to cut the tail.

Tight timeouts with retries (lesson 9), returning **partial results** when a few shards are late, and reducing variability at the source: tuning garbage collection, isolating background work on separate machines or time windows, and keeping queues short.

WORKED EXAMPLE: SEARCH ACROSS 100 SHARDS

A query fans out to 100 shards, and the slowest one decides the response time. With hedging after the p95 latency, a slow shard is quietly replaced by its replica. With a deadline, the service returns results from 98 shards rather than wait for the last two (and says so). The user-facing p99 drops from seconds to a few hundred milliseconds.

6 Measure the tail properly

- Track **p99 and p99.9**, not averages.
- Measure at the **user-facing** level, not only per service.
- **Never average percentiles** across servers: the average of ten servers' p99s is not the fleet's p99. Merge histograms (Prometheus histograms, HDRHistogram) instead.

COMMON MISTAKES

- Judging a fan-out system by per-backend p99.
- Hedging writes, causing duplicates.
- Hedging immediately (not after a delay), doubling load for nothing.
- Averaging percentiles on dashboards.

7 Key terms

Tail latency: the slowest requests: p99, p99.9

Hedged request: a delayed duplicate to another replica

Partial results: answer without the slowest shards at a deadline

Fan-out: one request calling many backends in parallel

Tied request: two copies that cancel each other on start

Histogram merge: the right way to combine percentiles

8 Interview questions

Q1. Why does fan-out amplify tail latency? BASIC
A request waits for its slowest call, so with 100 calls each slow 1% of the time, about 63% of requests hit at least one slow call.

Q2. What are hedged requests? BASIC
After a short delay, like the p95 latency, send a duplicate to another replica and use the first response, cutting the tail for a small extra load.

Q3. Your API calls 5 services in parallel, each with p99 = 200 ms. What's the user-facing p99 roughly? SCENARIO
The chance all five are under their p99 is 0.99^5 ≈ 95%, so about 5% of requests exceed 200 ms: the user-facing p99 is well above 200 ms, closer to the services' p99.8.

Q4. How would you cut the p99 of a search over 50 shards? SCENARIO
Hedge slow shard calls after the p95, set a deadline and return partial results, reduce variance on the shards (GC, background compaction), and keep shard queues short.

Q5. Why can't you average p99 across servers? TRICKY
Percentiles don't add or average; one server with terrible latency can dominate the real fleet p99 while the average looks fine. Merge the histograms and compute the percentile from the combined data.

Q6. When is hedging a bad idea? TRICKY
For non-idempotent operations, when the backend is overloaded (duplicates add load at the worst time; use a budget), or when slowness is caused by the request itself, so the hedge is just as slow.

Next: SD-J-014 · Performance Optimization Strategy

Performance Optimization Strategy

set a target, measure, fix the critical path, verify

Watch the 4-min video

In one line: to make something faster, set a measurable target, trace a real request to see where time goes, optimise the biggest part of the critical path first (Amdahl's Law caps the gain from small parts), favour the usual big wins (fewer round trips, parallel independent calls, caching, precomputation, indexes, less data), then verify under realistic load and keep the measurement.

1 "The dashboard is slow, make it faster"

Where do you start? Rewrite it in a faster language? Add a cache everywhere? Without a strategy you'll optimise the wrong thing. The answer is almost never to start rewriting code.

2 Measure first

Define the **target**: "page load under 500 ms at p95". Then **trace** a real request end to end and see where the time goes: distributed tracing (OpenTelemetry) shows each span, the database queries, downstream calls and rendering. The answer is often surprising. Profiling (lesson 43) then shows what's hot inside each service.

3 The critical path and Amdahl's Law

Optimise the **critical path**, the longest chain of dependent work. Speeding up anything off that path changes nothing. **Amdahl's Law** sets the limit: if a part takes 20% of the time, making it infinitely fast saves only 20%. Optimising a step that's 5% of the request is wasted effort; go after the biggest span first.



make part X infinitely fast → the request is at most 20% faster

speedup ≤ 1 / (1 − share) = 1 / 0.8 = 1.25× · so go after the biggest span first

Figure 1. Amdahl's Law: the share a part takes is the most you can gain by speeding it up.

4 The usual big wins: round trips

- Replace **N+1 queries** with one query or a join (Phase 2, lesson 60).
- **Batch** calls together (lesson 39).
- Run **independent calls in parallel**: with asyncio, `gather` lets three 200 ms calls finish in 200 ms, not 600.

Round trips add up fast when each one costs milliseconds.

5 The rest of the toolbox

- **Cache** expensive results (lessons 15 to 33).
- **Precompute** aggregates ahead of time instead of on every request.
- **Paginate**, and return only the fields you need.
- Add the right **index**.
- Use the right **data structure**: a set, not a list, for membership checks.
- Move non-urgent work to **background jobs** (lesson 37).

Each of these is cheaper than rewriting the service.

WORKED EXAMPLE: A 2-SECOND PRODUCT PAGE

Tracing shows four downstream calls made one after another: user, product, recommendations and reviews, around 400 ms each. They're independent, so call them in parallel with `asyncio.gather`. The page now waits only for the slowest call. Add a cache for recommendations, and the page loads in about **400 ms**.

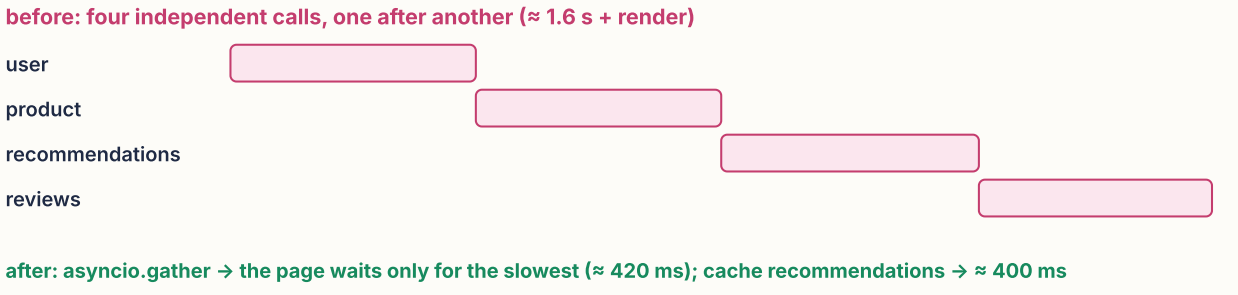


Figure 2. The trace that explains the slow page: four sequential spans that don't depend on each other.

```
# before: ~1,600 ms
user = await get_user(uid); product = await get_product(pid)
recs = await get_recs(uid); reviews = await get_reviews(pid)

# after: ~420 ms (the slowest call)
user, product, recs, reviews = await asyncio.gather(
    get_user(uid), get_product(pid), get_recs(uid), get_reviews(pid))
```

Figure 3. The fix in code: one gather instead of four awaits in a row.

6 Verify, and keep it fixed

Measure again after every change, under realistic load (lesson 44), and compare **percentiles**, not single runs. Stop when you hit the target, because each further gain costs more. Keep the measurement in place, with dashboards and alerts, so a regression doesn't return silently, and write down what changed and why.

COMMON MISTAKES

- Optimising without a target or a baseline.
- Micro-optimising code that's 2% of the request.
- Caching everything before finding the real bottleneck.
- Declaring victory from one fast run on a laptop.

7 Key terms

Critical path: the longest chain of dependent work

Span: one timed operation in a trace

asyncio.gather: run awaitables concurrently, wait for all

Amdahl's Law: speedup limited by the share you improve

Distributed tracing: following one request across services

Precomputation: compute ahead of time, read cheaply

8 Interview questions

Q1. How do you approach a 'make it faster' task? BASIC
Set a measurable target, measure with tracing and profiling, optimise the biggest part of the critical path first, then verify the improvement under realistic load.

Q2. What are the usual big wins? BASIC
Fewer round trips, parallel independent calls, caching and precomputation, indexes, and returning less data.

Q3. A page makes 4 sequential 400 ms calls that don't depend on each other. Fix? SCENARIO
Run them concurrently (asyncio.gather or a thread pool): latency becomes the slowest call, about 400 ms instead of 1.6 s; then cache the slowest if possible.

Q4. Database queries are 10% of request time. Is optimising them worth it? SCENARIO
At most a 10% gain by Amdahl's Law; look first at the span that dominates the critical path.

Q5. Why can parallelising calls make things worse? TRICKY
It multiplies concurrent load on downstream services and the database; if they're near saturation, latency rises. Cap concurrency and watch their utilization.

Q6. When do you stop optimising? TRICKY
When the target is met with headroom; beyond that each gain costs more effort and complexity. Keep monitoring so regressions are caught.

Next: SD-J-015 · Backpressure

Backpressure

let a slow consumer tell fast producers to slow down

Watch the 4-min video

In one line: backpressure is a slow consumer signalling producers to slow down: bound every queue and buffer, and when full block (in-process), reject (429/503 with Retry-After across HTTP) or drop (telemetry); use protocol flow control (TCP windows, HTTP/2 and gRPC, reactive streams, Kafka pull), propagate the signal to the source, and never hide overload in an unbounded queue.

1 No way to say "slow down"

Producers send jobs faster than your workers can handle them. The queue grows to millions of items. Memory fills up, latency is now hours, and finally everything crashes. The system had no way to say: *slow down*. That signal is **backpressure**.

2 What backpressure is

A slow consumer signals fast producers to slow down, so the system stays within its capacity. Without it, an **unbounded queue just hides the overload** until it runs out of memory. With it, overload is visible early and handled on purpose: a hidden failure becomes an explicit, controlled response.

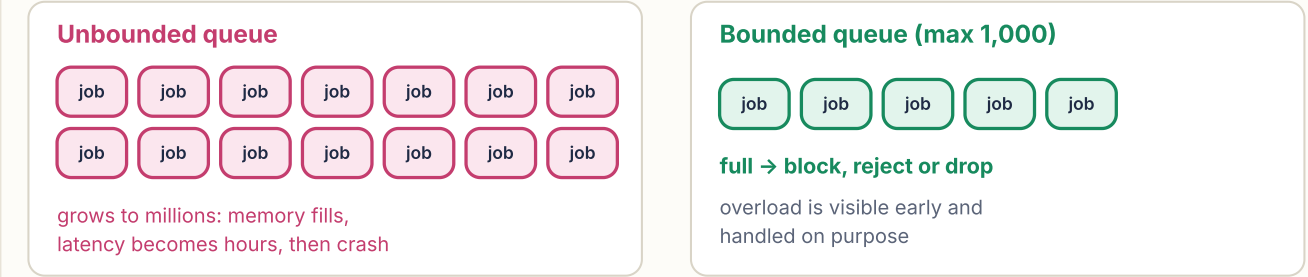


Figure 1. An unbounded queue hides overload until it fails; a bounded one forces a decision while the system is still healthy.

3 Bounded queues and the three choices

Give **every** queue and buffer a maximum size. When it's full, choose:

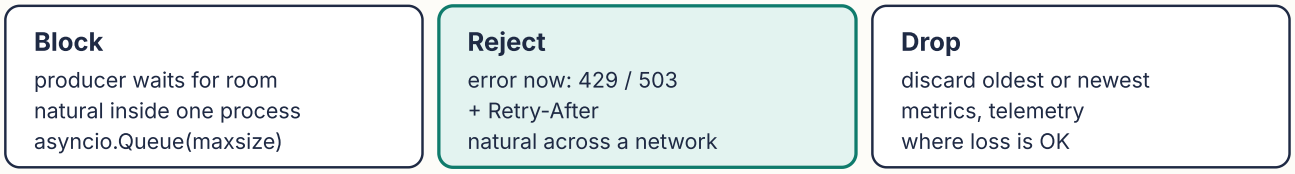


Figure 2. When a bounded queue is full: block the producer, reject the item, or drop data.

Blocking is natural inside one process (an `asyncio.Queue(maxsize=1000)` makes `await q.put()` wait). Rejecting is natural across a network. Dropping fits metrics and telemetry, where the newest data matters most.

4 Over HTTP: 429 and 503

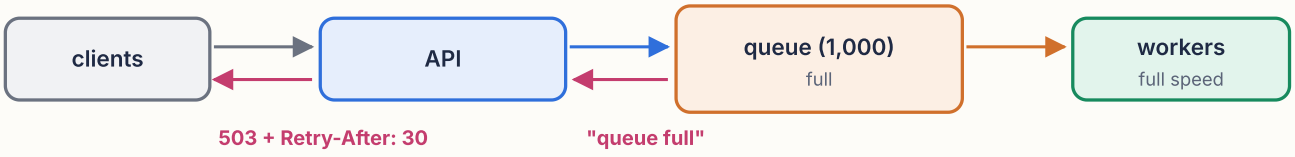
Return **429 Too Many Requests** when a client exceeds its limit, or **503 Service Unavailable** when the server is overloaded. Add a **Retry-After** header so well-behaved clients wait, ideally with exponential backoff and jitter (lesson 10 and Phase 2's API lessons on retries).

5 Built into many protocols

- **TCP flow control**: the receiver advertises a smaller window when it's slow (Phase 2, lesson 10).
- **HTTP/2 and gRPC**: per-stream flow-control windows.
- **Reactive streams**: the consumer requests exactly N items at a time.
- **Kafka**: consumers pull at their own pace; lag grows instead of memory.
- **Python**: `asyncio.Queue(maxsize)`, bounded thread-pool queues, semaphores for concurrency limits.

WORKED EXAMPLE: AN IMAGE JOB POOL

A worker pool processes image jobs from a queue limited to **1,000** items. When the queue is full, the API stops accepting new jobs and returns **503 with Retry-After: 30**. Clients back off and try again later. Workers keep running at full speed, memory stays flat, and accepted jobs still finish in minutes rather than hours.



the signal travels back to the source instead of a middle layer buffering forever

Figure 3. Backpressure end to end: the full queue makes the API reject, and clients slow down.

6 Design it end to end

- **Every hop needs a limit**: request concurrency, connection pools, queues, thread pools.
- **Propagate** the signal back toward the source, rather than letting a middle layer buffer forever.
- **Monitor** queue depth and rejection rate.
- **Combine with autoscaling** (lesson 14) and load shedding (lesson 10), so capacity grows while excess load is shed.

A system that says no early stays healthy under any load.

COMMON MISTAKES

- Unbounded in-memory queues "just in case".
- A middle service that retries and buffers everything, hiding overload from clients.
- Rejecting without Retry-After, so clients retry instantly and make it worse.

7 Key terms

Backpressure: consumer signals producers to slow down

429 / 503: client over limit / server overloaded

Flow control: protocol-level backpressure (TCP, HTTP/2)

Bounded queue: a queue with a maximum size

Retry-After: when the client may try again

Consumer lag: how far a Kafka consumer is behind

8 Interview questions

Q1. What is backpressure and how do you implement it? BASIC
It's a consumer signalling producers to slow down, implemented with bounded queues that block or reject when full, 429 or 503 with Retry-After over HTTP, and protocol flow control.

Q2. Why not just use an unbounded queue? BASIC
It hides overload until memory runs out and latency grows to hours.

Q3. A producer service floods a slow downstream API. What do you add? SCENARIO
A bounded queue or concurrency limit (semaphore) in front of the downstream calls, rejection or blocking when full, retries with backoff and jitter, and alerts on queue depth.

Q4. Block, reject or drop for a metrics pipeline? SCENARIO
Drop (usually the oldest samples or by sampling), because fresh metrics matter more than complete ones and blocking would stall the application.

Q5. How does Kafka provide backpressure? TRICKY
Consumers pull at their own pace, so a slow consumer simply falls behind (lag grows on durable disk) instead of being overwhelmed; producers are throttled by broker quotas or buffer limits.

Q6. Why should the signal reach the original source? TRICKY
If a middle layer absorbs the overload in its own buffer, it just moves the unbounded queue there; only the source can actually reduce the rate.

Next: Phase 4 · SD-H-001 · Distributed System Fundamentals backendroadmap.com